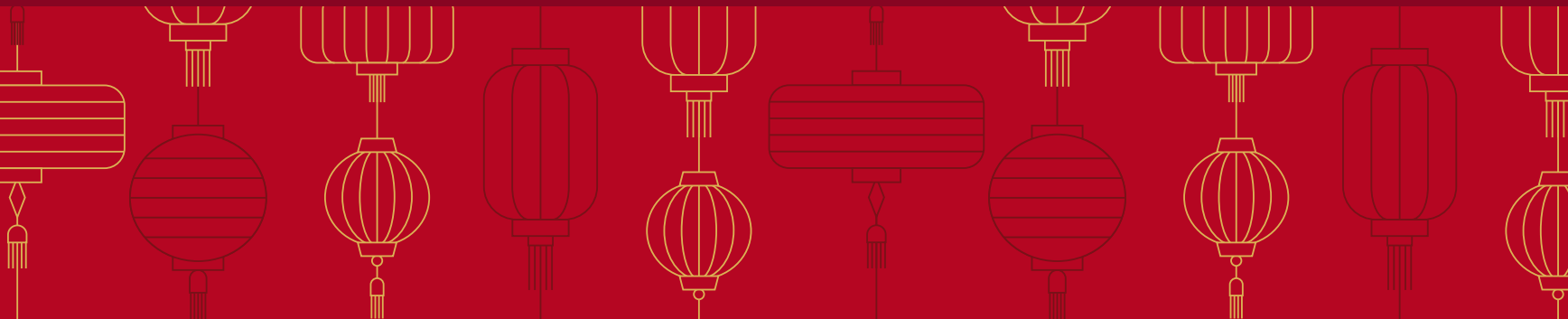




Welcome to Operating Systems

Informazioni generali

- **Docente:** Stefano Iannucci
 - Email: stefano.iannucci@uniroma3.it
 - Orario di ricevimento:
 - Flessibile – mandatemi un'email e decideremo insieme un orario che funzioni per entrambi
- Informazioni sul corso, risorse, pianificazione ed eventuali esercizi possono essere trovati su Moodle

Prove intermedie e appelli

- Durante il corso verranno proposti dei **quiz** in aula
 - Brevi quiz della durata di *pochi minuti*
 - Contenenti domande a *risposta multipla* o *brevi esercizi*
- L'esame può essere superato, in alternativa:
 - Partecipando ad **almeno il 90%** dei quiz ed ottenendo una **media superiore alla sufficienza**

+
 - Svolgendo gli esercizi di programmazione proposti al **primo appello**
 - Superando uno degli **appelli**
- Nel caso di svolgimento della **prima prova d'appello in formato ridotto**, il voto finale sarà determinato da: **40%** voto medio dei **quiz**, **60%** voto negli esercizi di **programmazione**.
 - Se lo studente non dovesse essere soddisfatto del voto medio ottenuto nei quiz, potrà optare per lo svolgimento della prima prova d'appello completa.
- Nel caso di svolgimento della **prima prova d'appello completa**, il voto finale sarà determinato da: **40%** voto ottenuto nelle domande di **teoria**; **60%** voto ottenuto negli esercizi di **programmazione**
- La possibilità di svolgere un esame ridotto è valida solo per il primo appello.
- Il tempo a disposizione per lo svolgimento delle prove verrà comunicato di volta in volta, e sarà commisurato alle domande ed agli esercizi.
- Per rilevare i presenti, in ogni lezione ci sarà un elenco nel quale ciascuno potrà segnalare la propria presenza. L'elenco dei presenti non sarà usato in nessun altro modo.

Buone notizie!

- **Textbook:** “Operating Systems: Three Easy Pieces”
by Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau
- **Buone notizie!** I capitoli chapters del libro sono liberamente scaricabili (gratis) all'indirizzo: <http://pages.cs.wisc.edu/~remzi/OSTEP/>
 - C'è anche una versione PDF dell'intero libro acquistabile per \$10.
- **Cosa non è questo corso:**
 - Un corso per amministratori di sistema.
 - Anche se potrà capitare di eseguire alcuni compiti che normalmente sono parte del lavoro di un amministratore di sistema
- **Quindi cosa studiamo in questo corso?**
 - Una panoramica degli aspetti tipici del kernel di un moderno sistema operativo
- **Cosa richiede questo corso:**
 - Capacità di programmare in C
 - Una macchina con *qualsiasi* distribuzione Linux. A breve vedremo come avviare un container Docker con un sistema Linux.

Orario del corso ed organizzazione lezioni

- Orario del corso:
 - Lunedì 11-14. 3 ore accademiche da 45 minuti, quindi potenzialmente 11-13:15
 - Martedì 15-17. 2 ore accademiche da 45 minuti, quindi potenzialmente 15-16:30
- La compressione dei semestri ha richiesto una riorganizzazione degli orari. I corsi da 6 CFU sono passati da 4h/settimana a 5h/settimana
 - C'è un lato positivo: con 5 ore di lezione a settimana riusciamo gestire delle "pause"
 - Prima pausa: settimana dall'11 al 15 novembre
 - Seconda pausa: dipende dagli impegni accademici.

Quindi... Linux?

- Anche se molti dei concetti che studieremo durante questo corso possono essere applicati a qualsiasi sistema operativo recente, esperimenteremo principalmente con Linux.
 - Sì, ma perché Linux?
 - Perché è gratis
 - Perché è open-source. Quindi, se vogliamo, possiamo scaricarne i sorgenti e studiarlo

Non siete ancora convinti?

- E' uno dei sistemi operativi più utilizzati al mondo:
 - Uno sviluppatore software su due usa Linux
 - E' il sistema operativo usato nella maggior parte dei supercomputer
 - Tra i web server, il 96.3% del primo milione (in termini di traffico) usa Linux
- Non basta?
 - E' installato sull'85% degli smartphone, con circa 1.6B utenti

- Da wikipedia:

A Linux distribution (often abbreviated as distro) is an operating system made from a software collection that is based upon the Linux kernel and, often, a package management system

- There are hundreds (if not thousands) of Linux distributions
- Each distro usually focuses on certain targets and provides certain characteristics

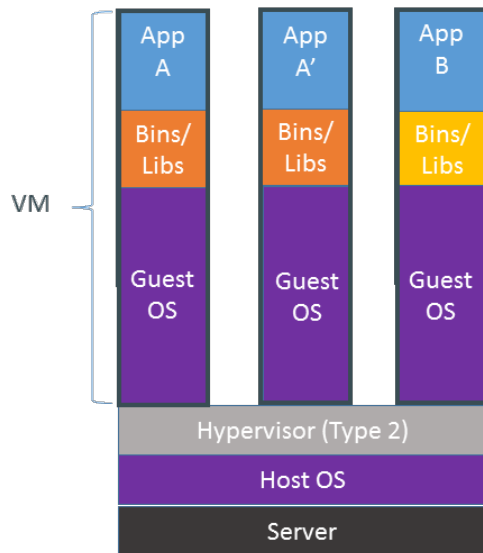
Linux Distributions



Programma del Corso - Macroaree

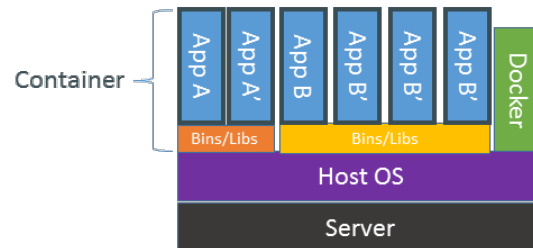
- Introduzione, installazione Docker e immagine del corso, pratica con riga di comando
- Gestione dei processi: introduzione, algoritmi di scheduling e laboratorio
- Gestione della memoria: traduzione degli indirizzi, paging, segmentation, swap, laboratorio
- Gestione della concorrenza: meccanismi di locking, IPC, strutture dati basate su lock e laboratorio
- Gestione dello storage: Ripasso su architetture hardware dei dischi rigidi, RAID ed implementazione software RAID in Linux, filesystem, laboratorio
- Virtualizzazione, virtualizzazione leggera e introduzione a Docker

Virtual Machines vs Containers

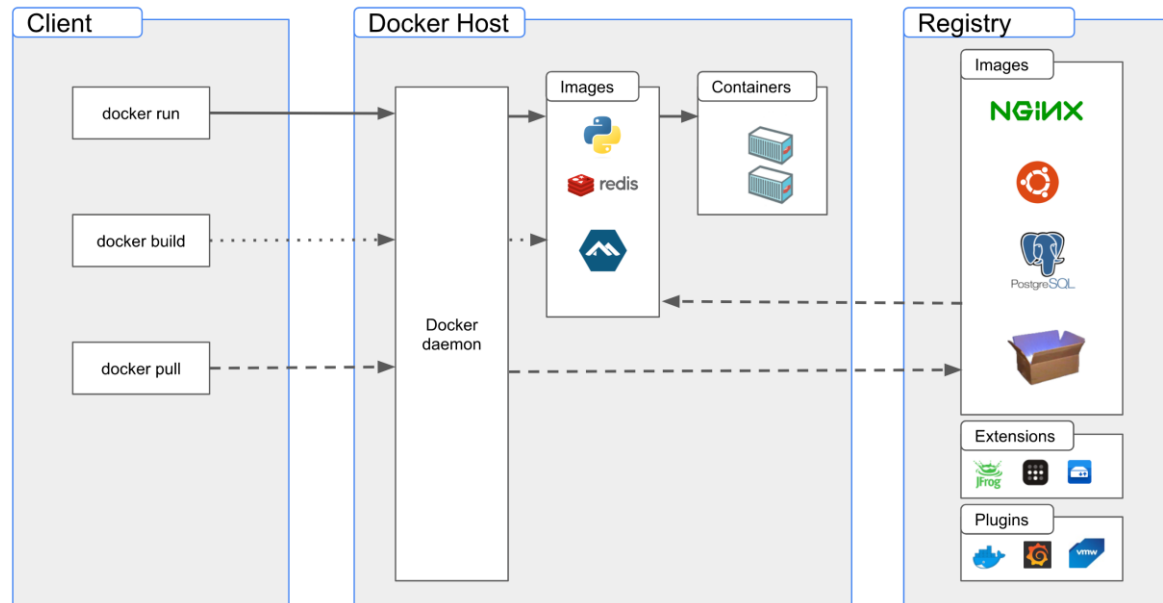


Containers are isolated,
but share OS and, where
appropriate, bins/libraries

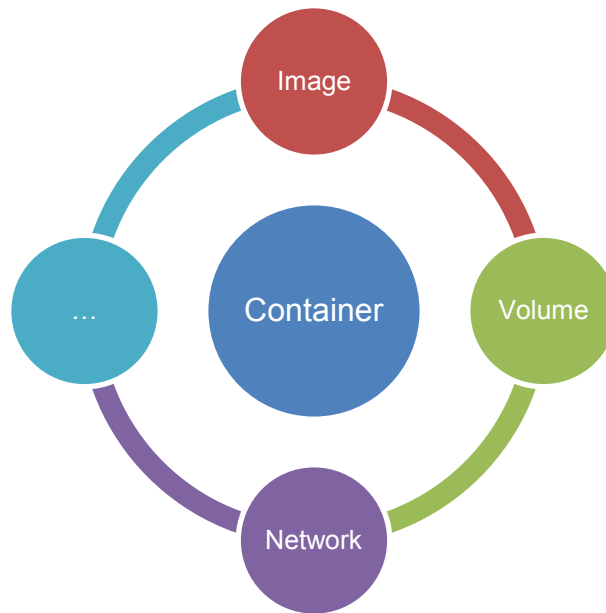
...result is significantly faster deployment,
much less overhead, easier migration,
faster restart



L'Architettura di Docker



I componenti di Docker



I componenti di Docker: Immagini

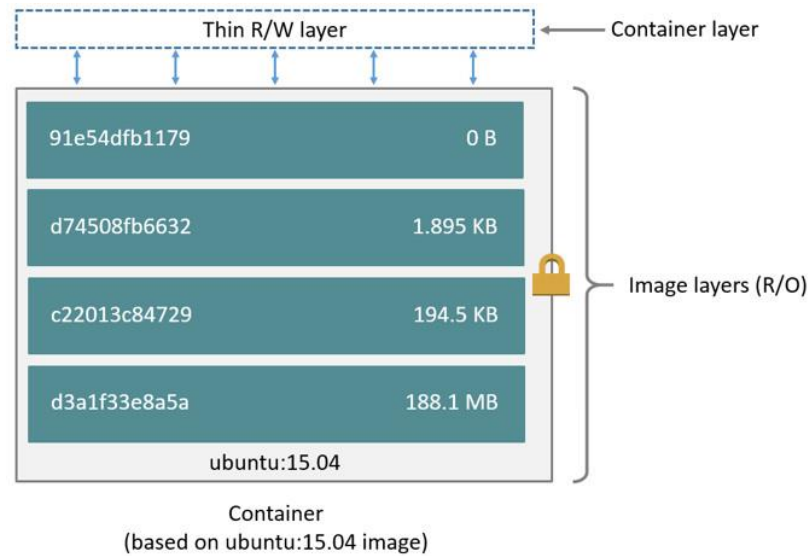
- Template *read-only* contenente:
 - il root filesystem del sistema che si vuole istanziare
 - meta-dati, ad esempio: l'autore dell'immagine, i comandi da eseguire quando il container viene avviato
 - le variabili d'ambiente che devono essere impostate
 - ...
 - È UN PACCHETTO CHE CONTIENE TUTTO CIÒ CHE È NECESSARIO PER ESEGUIRE UN'APPLICAZIONE. UN'IMMAGINE DOCKER È LA BASE DA CUI VENGONO CREATI I CONTAINER.
- Le immagini sono costituite da *layers*
 - Ogni layer può aggiungere, cambiare, rimuovere file e/o metadati
 - Immagini diverse possono condividere dei layer per ottimizzare l'uso del disco, il tempo di trasferimento, e l'utilizzo di memoria
 - Realizzato tramite UnionFS

Esempio di immagine per una webapp Java

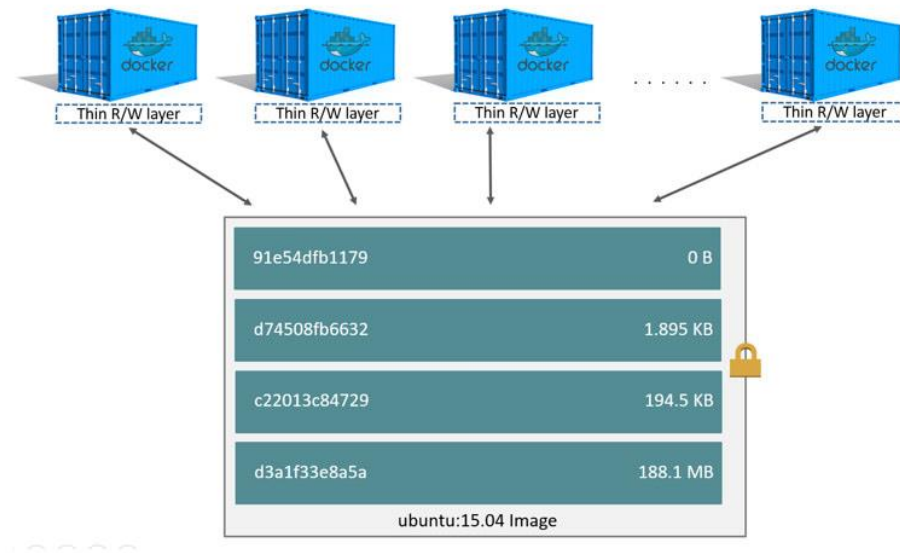
Ognuno dei seguenti elementi corrisponde ad un layer:

- Ubuntu base layer
- Pacchetti e file di configurazione aggiunti dall'Ufficio IT
- JRE
- Tomcat
- Dipendenze dell'applicazione
- Codice dell'applicazione
- Configurazione dell'applicazione

II layer *read-write*



Più container possono usare la stessa immagine



I componenti di Docker: Volumi

- E' buona norma che i container siano *stateless*
 - Disaccoppiamento tra sistema, applicazioni, **e dati**
 - Il disaccoppiamento tra sistema e applicazioni è realizzato tramite layer dell'immagine
 - Il disaccoppiamento rispetto ai dati è realizzato con l'introduzione di un nuovo concetto, quello di **Volume**
- Un volume è una directory che può essere associata ad uno o più container tramite un *mountpoint*
 - E' accessibile in lettura e scrittura
 - Il suo ciclo di vita è **indipendente** dal ciclo di vita dei container

Buona notizia!

- La sintassi dei comandi per l'amministrazione dei container Docker è stata aggiornata e "ripulita" nel 2017. Gestire container docker è ora molto semplice, se sappiamo cosa fare.
- Comando principale: `docker`
 - Per la gestione dei container: `docker container <qualcosa>`
 - Esempio: `docker container ls` mostra l'elenco dei container attivi
 - Per la gestione delle immagini: `docker image <qualcosa>`
 - Esempio: `docker image rm <id_immagine>` rimuove l'immagine con ID `id_immagine`
 - Per la gestione dei volumi: `docker volume <qualcosa>`
 - Esempio: `docker volume inspect <id_volume>` mostra informazioni dettagliate sul volume con ID `id_volume`

Non ci piace imparare a memoria

- Con il tempo, imparerete a memoria i comandi principali
 - Non è però necessario studiarli tutti subito: l'interfaccia a riga di comando ci aiuta molto.
 - Esempio: basta digitare `docker volume` senza ulteriori argomenti e ci verrà restituita una lista di azioni che è possibile eseguire. Stessa cosa vale per `image`, `network`, `container`.
- Disponibile una cheat sheet con alcuni dei comandi di utilizzo più frequente:
 - https://docs.docker.com/get-started/docker_cheatsheet.pdf

Hands-on: Installiamo Docker

- Faremo ora un'installazione passo passo di Docker utilizzando "Docker Desktop".
 - E' una release di docker molto comoda perché include tutto ciò di cui abbiamo bisogno per partire ed è disponibile per Windows, Mac, Linux.

Ricapitolando:

- Creazione (e avvio) di un container con l'immagine del corso:

- `docker container run --name os2425` 
`--hostname sistemi-operativi -v dati:/corso -it chmod84/os2425` 

- Avvio del container precedentemente creato ed avvio della shell

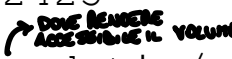
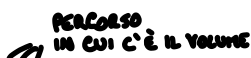
- `docker container start os2425`
- `docker container exec -it os2425 /bin/bash`

- Aggiornamento dell'immagine del container:

- `docker container stop os2425` 

- `docker container rm os2425`

- `docker image pull`  `os2425`

- `docker container run --name os2425` 
`--hostname sistemi-operativi -v dati:/corso -it chmod84/os2425`  

Introduzione alla Riga di Comando

Come interpretare la riga di comando

- Il terminale si presenta in questo modo. Ma cosa vuol dire ciò che c'è scritto?

```
corso@sistemi-operativi:~$
```



nome utente

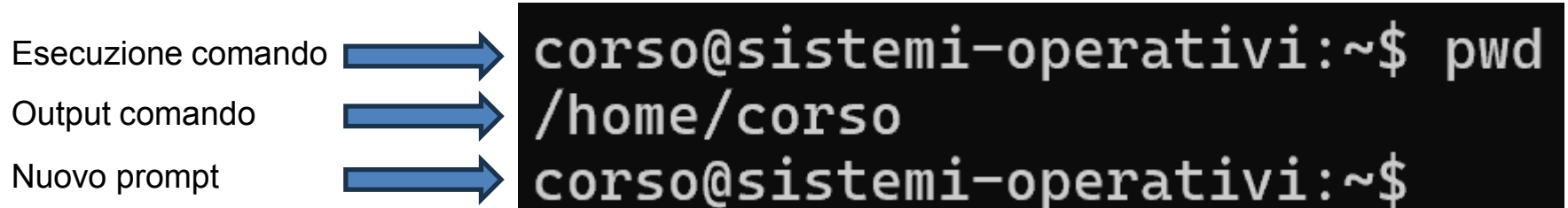


nome computer



Posizione su filesystem

Eseguiamo il nostro primo comando



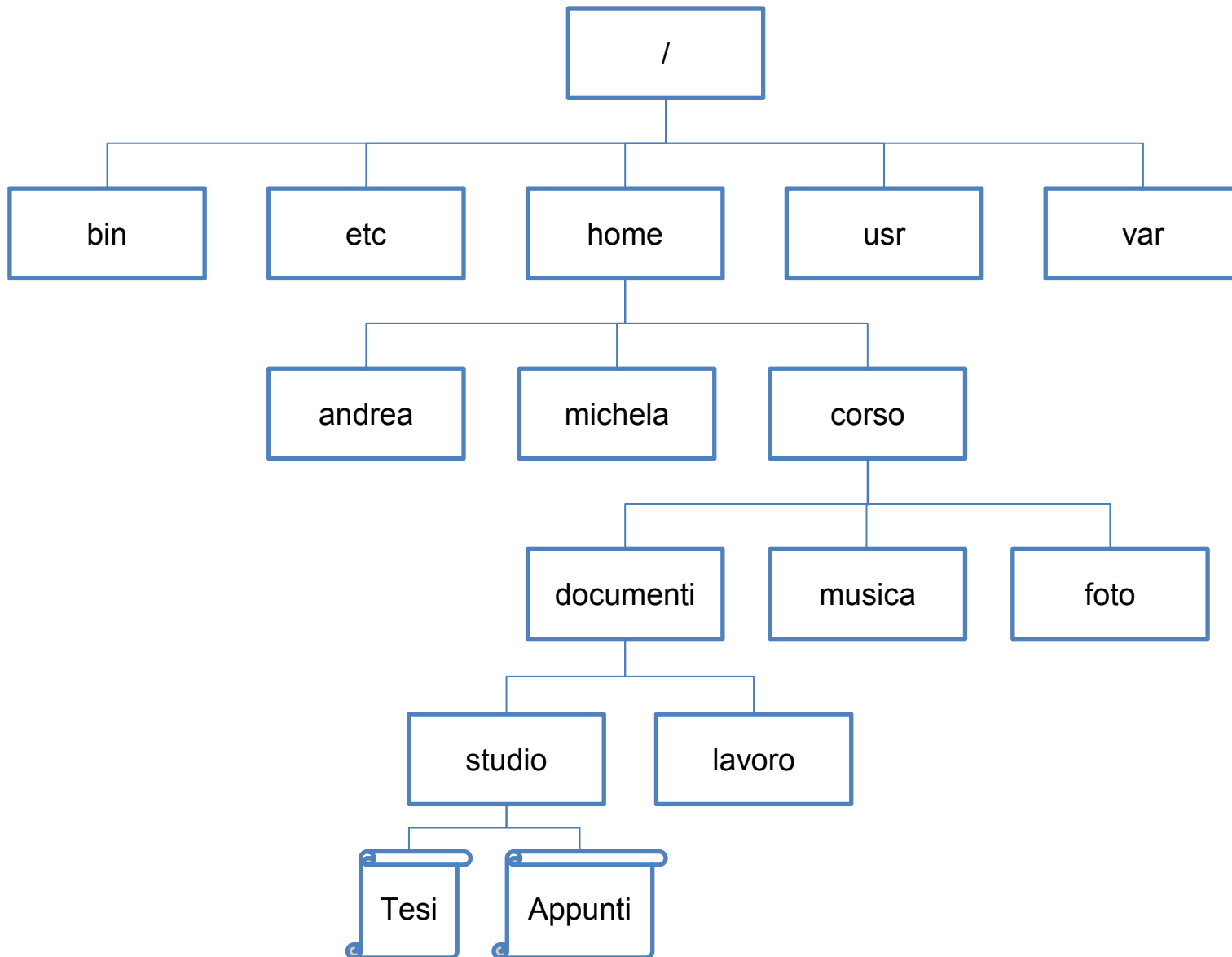
Attenzione: Linux è *case sensitive*. Ciò vuol dire che il comando *pwd* è diverso da *PWD*, e che */home/corso* è diverso da */home/Corso*

Cosa ci dice il nostro primo comando?

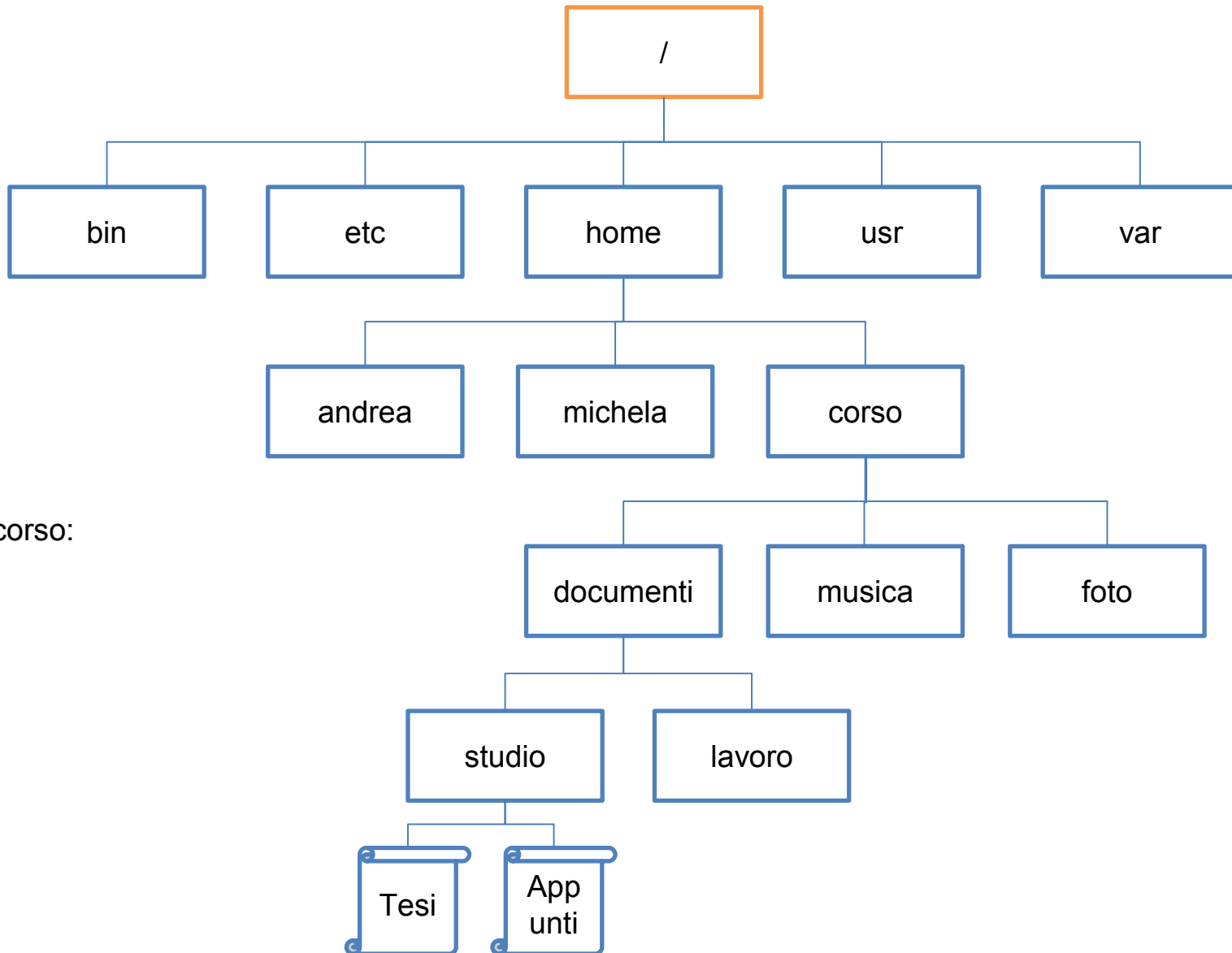
```
corso@sistemi-operativi:~$ pwd  
/home/corso  
corso@sistemi-operativi:~$
```

- *pwd* sta per print working directory
 - Cioè stampa la directory (cartella) in cui stiamo lavorando
- Il concetto di *working directory* è fondamentale
 - E' la directory dove ci troviamo all'interno del *filesystem*
 - E' la directory all'interno della quale effettuiamo le operazioni
 - Creazione, modifica, rimozione, ... di file ed altre directory
- */home/corso* indica la directory *corso* all'interno della directory *home*, all'interno della directory principale del filesystem (*root directory*)

Il Filesystem

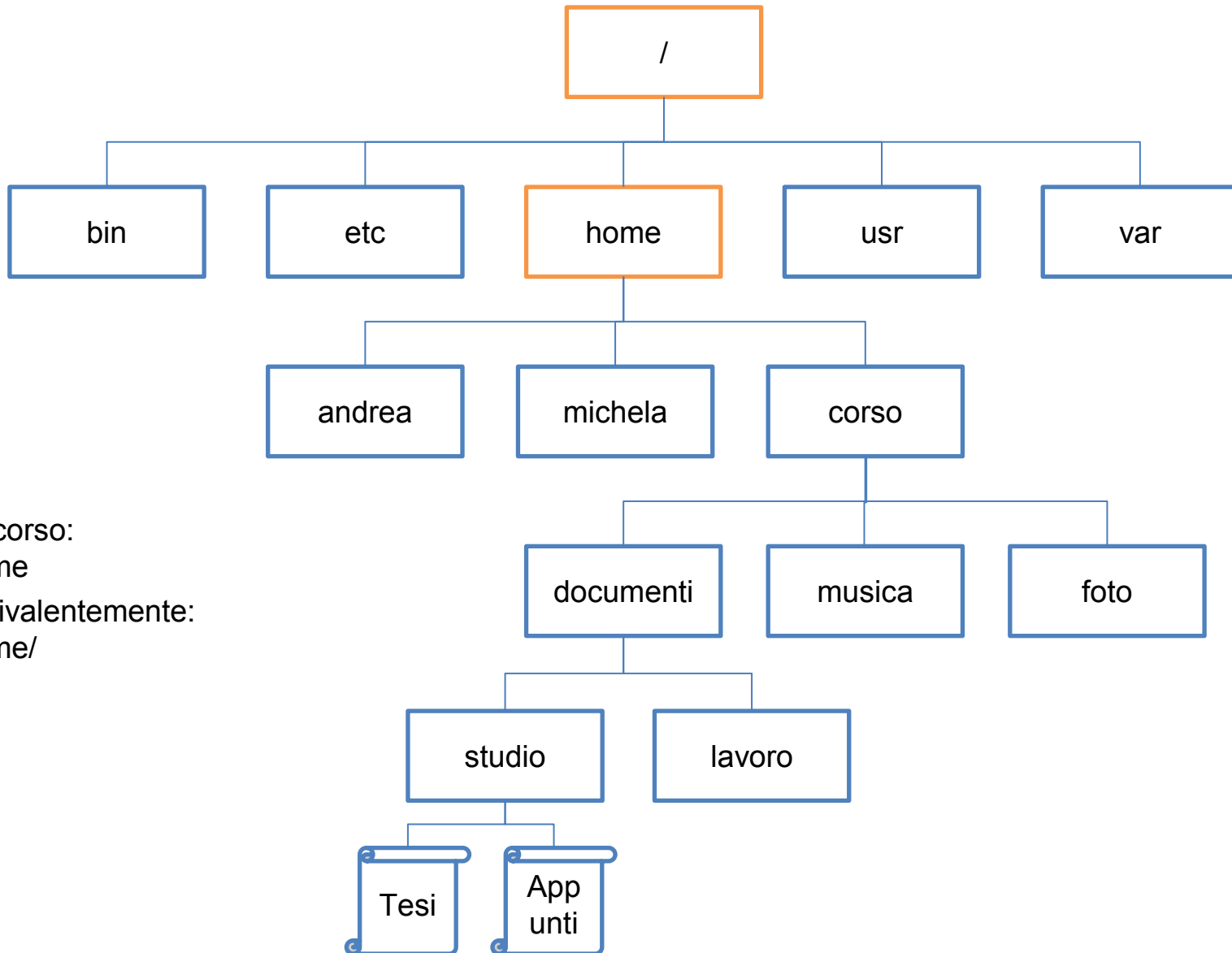


Il Filesystem

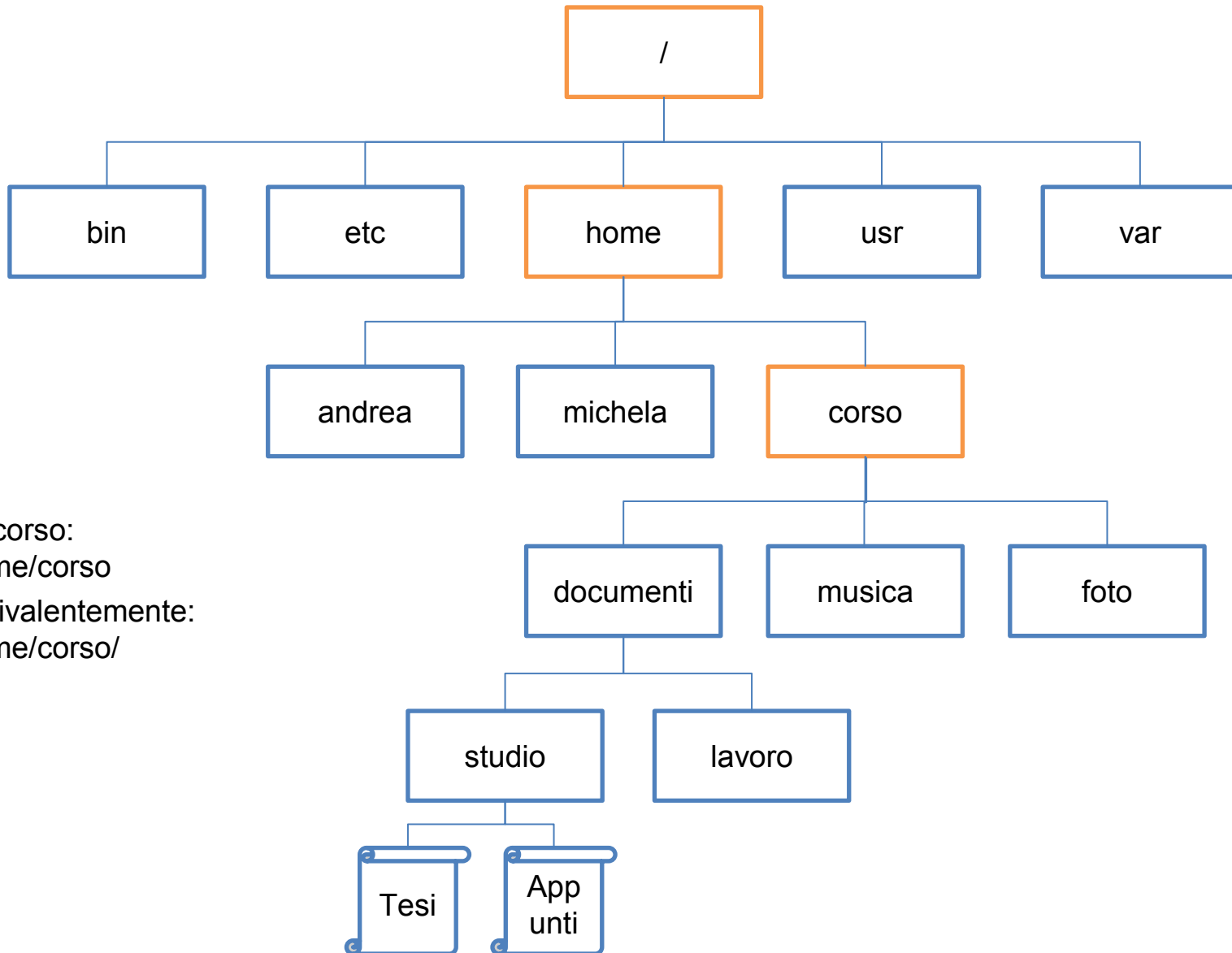


Percorso:
/

Il Filesystem



Il Filesystem



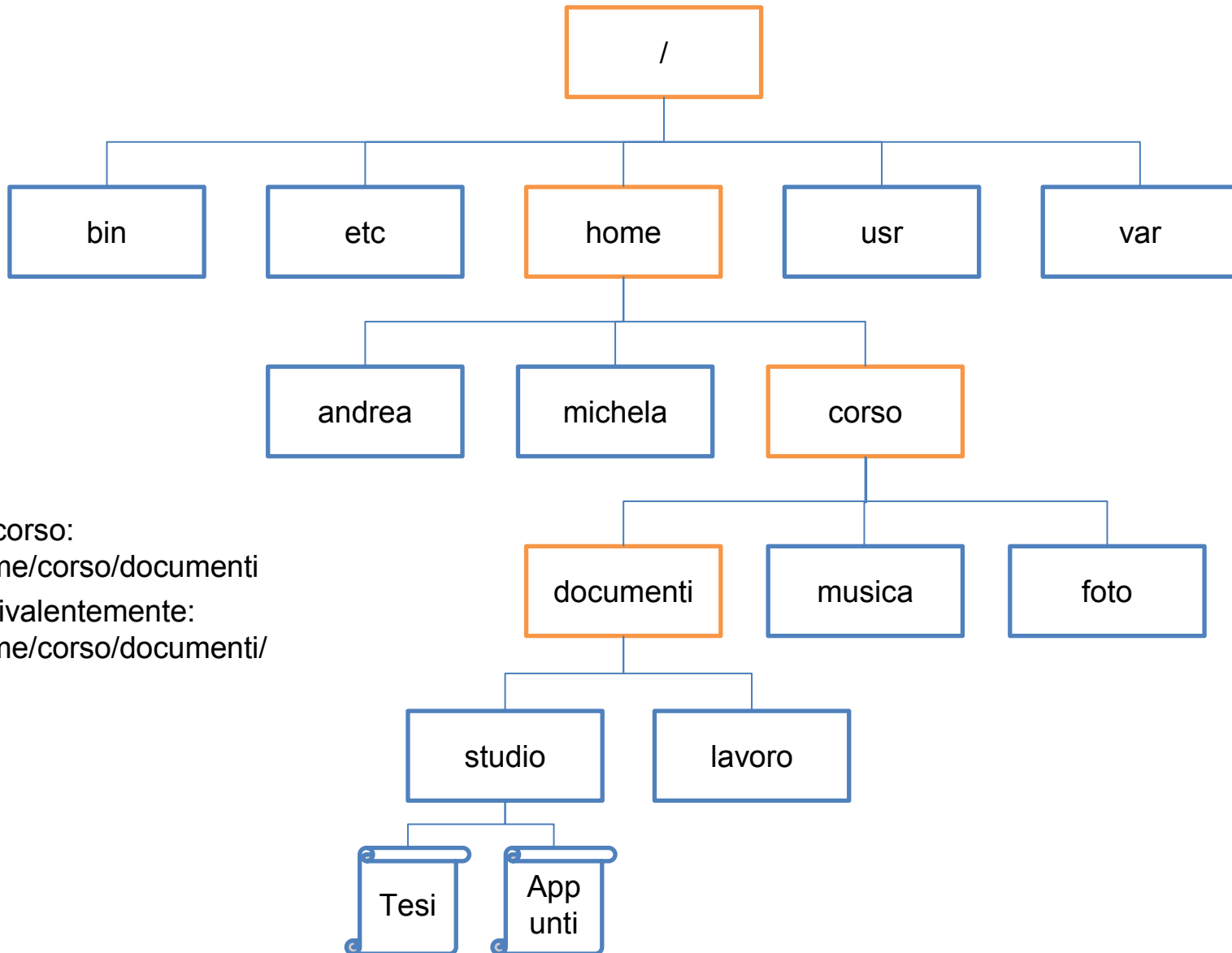
Percorso:

/home/corso

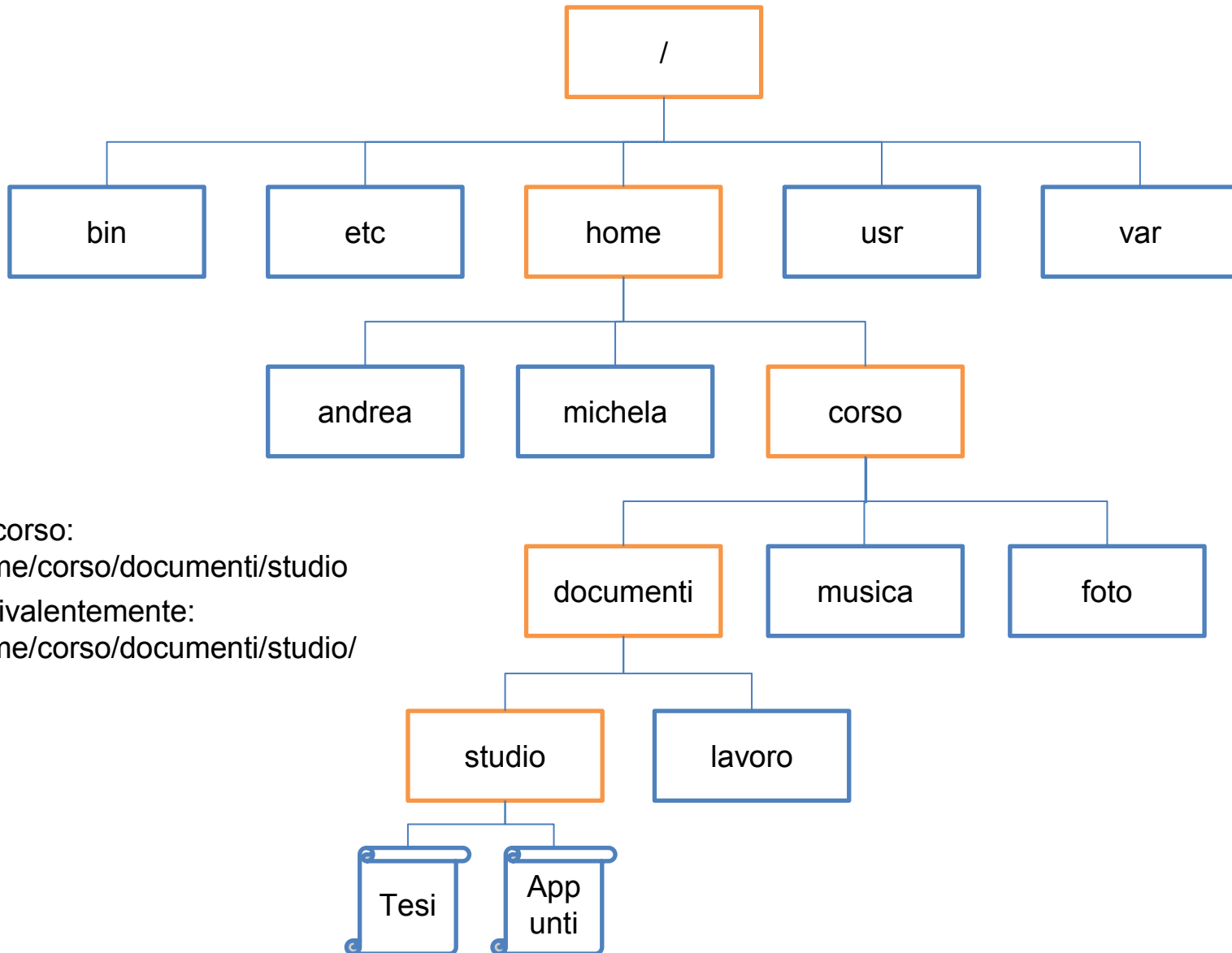
Equivalentemente:

/home/corso/

Il Filesystem



Il Filesystem



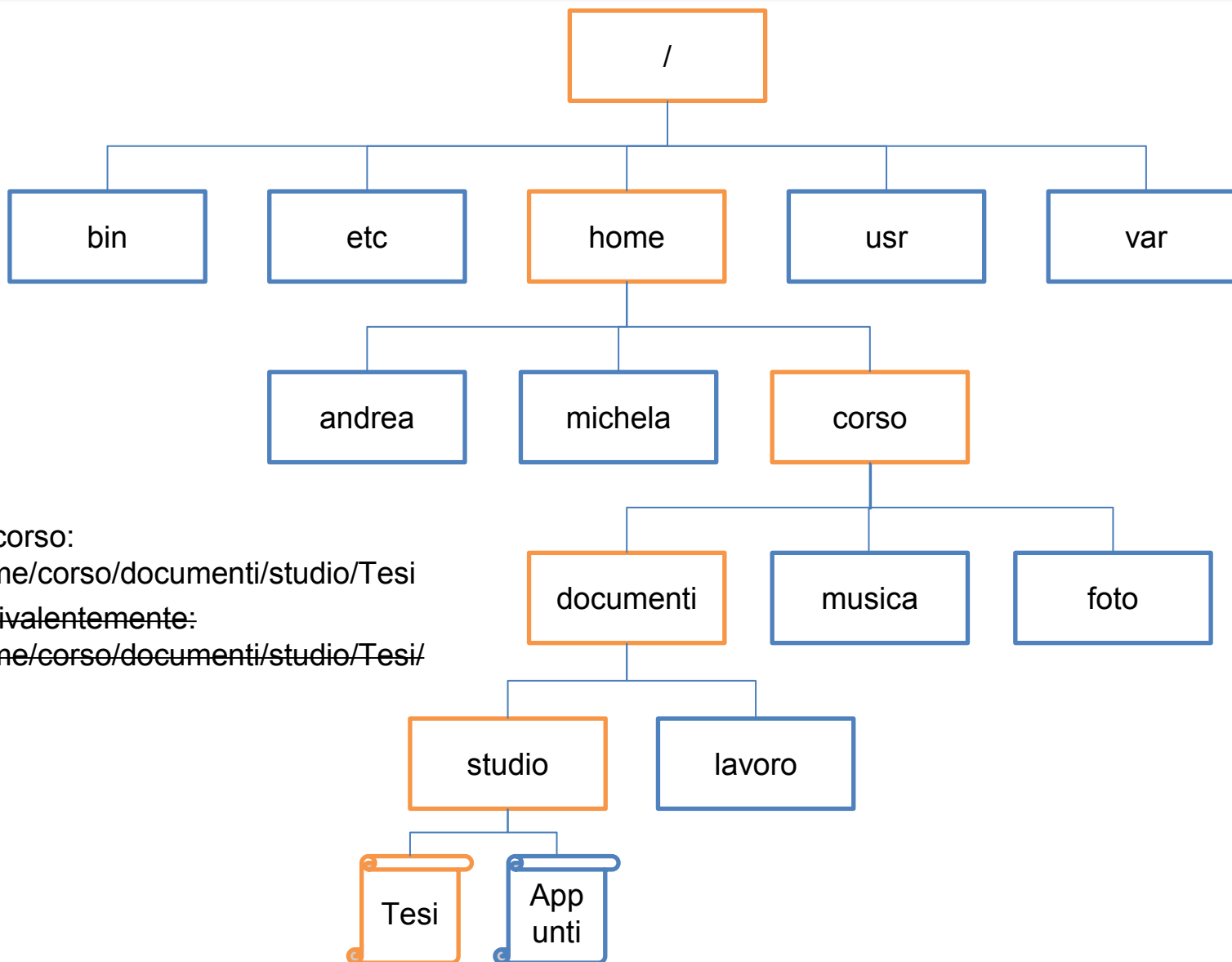
Percorso:

/home/corso/documenti/studio

Equivalentemente:

/home/corso/documenti/studio/

Il Filesystem



Percorso:

/home/corso/documenti/studio/Tesi

Equivalentemente:

/home/corso/documenti/studio/Tesi/

Come cambiare la working directory?

```
corso@sistemi-operativi:~$ pwd
/home/corso
corso@sistemi-operativi:~$ cd /
corso@sistemi-operativi:/$ pwd
/
corso@sistemi-operativi:/$
```

- Il comando *cd* sta per change directory
- Il primo comando mi ha informato del fatto che mi trovo in */home/corso*
- Il secondo comando mi ha fatto spostare nella directory */*
 - Notare che **il comando *cd*, se eseguito con successo, non produce alcun output**
- Il terzo comando mi ha informato del fatto che mi trovo in */*

Come tornare da / in */home/corso*?

- Prima di tutto devo andare in */home*. Da lì poi posso andare in */home/corso*
 - *cd home*
 - *cd corso*

```
corso@sistemi-operativi:/$ cd home
corso@sistemi-operativi:/home$ pwd
/home
corso@sistemi-operativi:/home$ cd corso
corso@sistemi-operativi:~$ pwd
/home/corso
corso@sistemi-operativi:~$
```

Percorsi relativi, percorsi assoluti

- Tutti i percorsi che iniziano con `/` sono detti assoluti
 - Perché specificano il percorso nella sua interezza (ad esempio `/home/corso`)
- Percorsi che non iniziano con `/` sono detti relativi
 - Ne abbiamo già utilizzati due! Quali?
 - *home* e *corso*
- Quando si usa un percorso relativo, Linux lo trasforma in assoluto tramite un'operazione di concatenamento con il percorso rappresentante la working directory
 - Ad esempio, quando abbiamo eseguito `cd corso`:
 - *corso* era il percorso relativo
 - La working directory era `/home/`
 - `/home/ + corso → /home/corso`
 - Quindi è come se avessimo eseguito `cd /home/corso`

Directory speciali

- Ogni directory contiene a sua volta due directory speciali, chiamate:
 - `.`
 - `..`
- La directory `.` rappresenta la directory corrente
 - Quindi `/home/corso/.` è esattamente uguale a `/home/corso`
 - Ma anche `/home/corso/./.` è esattamente uguale a `/home/corso`
 - Sembra strano ed inutile, ma presto ci tornerà comodo
- La directory `..` Rappresenta la directory superiore
 - Quindi `/home/corso/..` è esattamente uguale a `/home`
 - `/home/corso/./..` è esattamente uguale a...?
 - `/`
- Ma perché dovremmo complicarci la vita in questo modo?
 - Se mi trovo in `/home/corso` non è sufficiente scrivere `cd /home` se voglio andare in `/home`?

La directory ..

- Se mi trovo in `/home/corso` non è sufficiente scrivere `cd /home` se voglio andare in `/home`?
 - Sì, è sufficiente. Però è scomodo, perché devo ricordarmi del nome della directory superiore
 - E' molto più facile scrivere `cd ..`

```
corso@sistemi-operativi:~$ pwd
/home/corso
corso@sistemi-operativi:~$ cd ..
corso@sistemi-operativi:/home$ pwd
/home
corso@sistemi-operativi:/home$
```

Navigazione con ..

- Come posso raggiungere / a partire da /home/corso senza utilizzare il simbolo / ?

```
corso@sistemi-operativi:~$ pwd
/home/corso
corso@sistemi-operativi:~$ cd ..
corso@sistemi-operativi:/home$ pwd
/home
corso@sistemi-operativi:/home$ cd ..
corso@sistemi-operativi:/$ pwd
/
corso@sistemi-operativi:/$
```

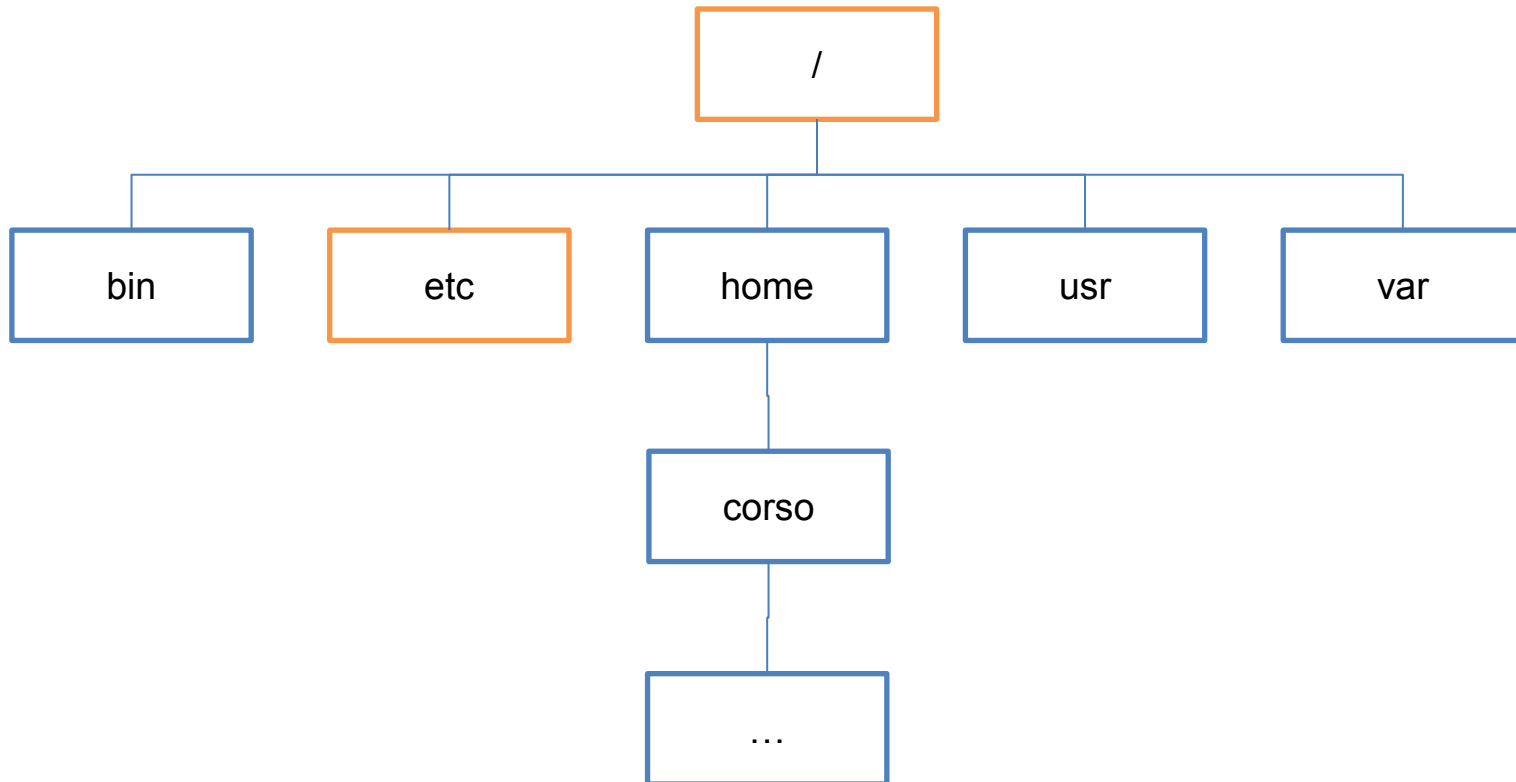
- Non è necessario utilizzare *pwd* ogni volta dopo *cd*

```
corso@sistemi-operativi:~$ cd ..
corso@sistemi-operativi:/home$ cd ..
corso@sistemi-operativi:/$ pwd
/
corso@sistemi-operativi:/$
```



```
corso@sistemi-operativi:~$ cd ../../
corso@sistemi-operativi:/$ pwd
/
corso@sistemi-operativi:/$
```

Domanda



Come arrivare in `/etc` partendo da `/home/corso`?

`cd` con percorso assoluto
`cd /etc`

`cd` con percorso relativo
`cd ../../etc`

La home directory con ~

- La home directory è una directory molto speciale all'interno della quale vengono solitamente archiviati tutti i documenti (testi, musica, video, ...) di un determinato utente.
 - Ogni utente ha la propria home directory
 - Convenzionalmente, ogni utente ha la propria home directory all'interno della directory /home, chiamata con il proprio nome utente.
- Il simbolo ~ è una scorciatoia per rappresentare la home directory
 - Quindi, ovunque io mi trovi, posso utilizzare `cd ~` per spostarmi nella mia home directory
 - Sulle tastiere italiane il simbolo ~ non c'è (bisognerebbe digitarne il codice ASCII)
 - Fortunatamente, c'è un modo ancora più veloce di tornare alla propria home directory: `cd`

Creazione di directory

- Molto spesso ci troviamo a dover creare una directory per meglio organizzare i nostri file.
- La creazione di una directory in Linux si ottiene eseguendo il comando ***mkdir***
 - Uso: *mkdir <percorso>*, dove *<percorso>* indica la directory che si vuole creare, che può essere specificata sia con un percorso assoluto che con un percorso relativo
 - Esempio: *mkdir /home/corso/tutorial* crea una directory tutorial all'interno della directory */home/corso*
 - Avrei potuto alternativamente spostarmi all'interno della directory */home/corso* e creare la directory tutorial con un percorso relativo: *mkdir tutorial*
- *mkdir* permette anche la creazione di più directory allo stesso tempo
 - Esempio: *mkdir dir1 dir2 dir3*
- Cosa succede se provo ad eseguire *mkdir* senza parametri?

Mostrare il contenuto di una directory

- Ovviamente non dobbiamo imparare a memoria tutto l'albero del filesystem. Il comando *ls* (sta per *list*) serve ad elencare il contenuto di una directory.

```
corso@sistemi-operativi:~$ mkdir /home/corso/tutorial
corso@sistemi-operativi:~$ cd /home/corso/tutorial
corso@sistemi-operativi:~/tutorial$ mkdir dir1 dir2 dir3
corso@sistemi-operativi:~/tutorial$ ls
dir1  dir2  dir3
corso@sistemi-operativi:~/tutorial$
```

- Notiamo che il comando *mkdir* non ha creato una directory dentro l'altra, ma tutte e tre allo stesso livello, dentro *tutorial*.
 - E' comunque possibile creare tre directory (diciamo *dir4*, *dir5*, *dir6*) una dentro l'altra eseguendo: *mkdir -p dir4/dir5/dir6*
 - L'opzione *-p* istruisce *mkdir* a creare (se non esistono) tutte le directory specificate all'interno del percorso contenuto nell'argomento
 - Domanda: *dir4/dir5/dir6* è assoluto o relativo?

Il problema degli spazi

- Mio cugino deve inviarmi dei documenti e vorrei archivarli tutti all'interno di una directory a lui dedicata. Proviamo a creare una directory con uno "spazio" tra due parole, chiamata *mio cugino*

```
corso@sistemi-operativi:~/tutorial$ mkdir mio cugino
corso@sistemi-operativi:~/tutorial$
```

- Ha funzionato!!! Proviamo ad utilizzare `ls` per vedere effettivamente questa nuova directory.

```
corso@sistemi-operativi:~/tutorial$ ls
cugino  dir1  dir2  dir3  mio
corso@sistemi-operativi:~/tutorial$
```

- Oops... in realtà ha creato due directory, una chiamata *mio* e l'altra chiamata *cugino*. C'era da aspettarselo!

Gestione degli spazi

- Se possibile, è più semplice sostituire uno spazio con un altro carattere, come ad esempio un *under score* _ o un trattino -
 - Quindi, *mio cugino* diventerebbe *mio_cugino*, che è sintatticamente una sola parola
 - Purtroppo, non è sempre possibile: a volte ci troviamo a lavorare con file o directory creati da altri
 - E comunque vogliamo avere la libertà di poterlo fare!

```
corso@sistemi-operativi:~/tutorial$ mkdir "folder 1"
corso@sistemi-operativi:~/tutorial$ mkdir 'folder 2'
corso@sistemi-operativi:~/tutorial$ mkdir folder\ 3
corso@sistemi-operativi:~/tutorial$ mkdir "folder 4" "folder 5"
corso@sistemi-operativi:~/tutorial$ mkdir -p "folder 6"/"folder 7"
corso@sistemi-operativi:~/tutorial$
```

Creazione di file

- Finora ci siamo occupati soltanto di directory, ma possiamo anche creare dei file
- Ad esempio, supponiamo di voler creare un file avente come contenuto l'output del comando `ls`, eseguito all'interno di `/home/corso/tutorial`

```
corso@sistemi-operativi:~/tutorial$ ls
cugino  dir1  dir2  dir3  'folder 1'  'folder 2'  'folder 3'  'folder 4'  'folder 5'  'folder 6'  mio
corso@sistemi-operativi:~/tutorial$
```

- E' possibile redirigere ***l'output*** di un comando su file tramite l'operatore `>`

```
corso@sistemi-operativi:~/tutorial$ ls > output.txt
corso@sistemi-operativi:~/tutorial$ ls
cugino  dir1  dir2  dir3  'folder 1'  'folder 2'  'folder 3'  'folder 4'  'folder 5'  'folder 6'  mio  output.txt
corso@sistemi-operativi:~/tutorial$
```

Lettura di un file

- Possiamo leggere il contenuto di un file utilizzando il comando **cat** (concatenate)

```
corso@sistemi-operativi:~/tutorial$ cat output.txt
cugino
dir1
dir2
dir3
folder 1
folder 2
folder 3
folder 4
folder 5
folder 6
mio
output.txt
corso@sistemi-operativi:~/tutorial$
```

- Possiamo creare ulteriori file con contenuti testuali arbitrari con il comando **echo**
 - Proviamo ad eseguire *echo "This is a test"*. Cosa succede?
 - Come possiamo creare un file che contenga la frase *This is a test*?

Più files, più letture

- Esercizio: creiamo tre file, chiamati *test_1.txt*, *test_2.txt*, *test_3.txt*, che contengano rispettivamente i seguenti testi:
 - This is a test
 - This is a second test
 - This is a third test

```
corso@sistemi-operativi:~/tutorial$ echo "This is a test" > test_1.txt
corso@sistemi-operativi:~/tutorial$ echo "This is a second test" > test_2.txt
corso@sistemi-operativi:~/tutorial$ echo "This is a third test" > test_3.txt
corso@sistemi-operativi:~/tutorial$
```

- Ora proviamo a leggere in modo concatenato il contenuto dei tre file

```
corso@sistemi-operativi:~/tutorial$ cat test_1.txt test_2.txt test_3.txt
This is a test
This is a second test
This is a third test
corso@sistemi-operativi:~/tutorial$
```

Creiamo un nuovo file che contenga il contenuto dei vari test

- Come possiamo creare un nuovo file con all'interno il contenuto dei vari file di test che abbiamo scritto prima?
 - `cat t* > combined.txt`
- Cosa succede se ripetiamo questo stesso comando?
- E se invece volessimo effettivamente "appendere" un testo ad un file esistente?
 - Utilizziamo l'operatore `>>`
- Proviamo:
 - `cat t* >> combined.txt`
 - `echo "ho appeso una linea" >> combined.txt`
 - `cat combined.txt`
- Ripetiamo i comandi un po' di volte (provate ad utilizzare la freccia in su), fino ad ottenere un file più lungo della dimensione dello schermo. Come possiamo leggere tale file?
 - `less combined.txt` **9 PER USCIRE**

Spostare file

- E' possibile spostare un file o una cartella utilizzando il comando *mv* (move)
 - Il comando *mv combined.txt dir1* sposta il file *combined.txt* all'interno della cartella *dir1*
- Verifichiamo se il file è stato effettivamente spostato

```
corso@corso-os:~$ mv combined.txt dir1
corso@corso-os:~$ ls dir1
combined.txt
corso@corso-os:~$
```

- Ora supponiamo di aver cambiato idea e di non volere il file in *dir1*, ma nella directory dove si trovava inizialmente. Come facciamo? Diverse strade, proviamone alcune:
 - Posso specificare come percorsi di partenza e destinazione dei percorsi assoluti
 - Posso spostarmi nella cartella che attualmente contiene *combined.txt* ed utilizzare dei percorsi relativi per spostare il file una directory sopra
 - Posso rimanere dove mi trovo ed utilizzare i percorsi relativi per spostare il file nella directory corrente
 - ...

Spostamenti multipli di file e cartelle

- *mv* consente di spostare con un solo comando diversi elementi in una cartella di destinazione
 - Esempio: *mv combined.txt test_* dir3 dir2* sposta all'interno di *dir2* gli elementi *combined.txt*, *test_**, *dir3*
 - Notare che gli elementi oggetto dello spostamento possono essere sia file che cartelle, mentre l'elemento di destinazione deve necessariamente essere una cartella (non avrebbe senso altrimenti)

```
corso@corso-os:~$ mv combined.txt test_* dir3 dir2
corso@corso-os:~$ ls dir2
combined.txt  dir3  test_1.txt  test_2.txt  test_3.txt
corso@corso-os:~$
```

- Esercizio: spostare ancora una volta il file *combined.txt* da *dir2* a *dir4/dir5/dir6*

Copiare e rinominare

- Il comando per la copia di un file è molto simile a quello per lo spostamento:
 - `cp <percorso di origine> <percorso di destinazione>`
- Come copiamo il file `combined.txt` che sta dentro `dir4/dir5/dir6` all'interno della directory corrente?
 - `cp dir4/dir5/dir6/combined.txt .`
- A questo punto, supponiamo di voler creare una copia di backup di `combined.txt` all'interno della stessa cartella, chiamata `backup_combined.txt`. Come facciamo?
 - `cp combined.txt backup_combined.txt`
- E se il nome che abbiamo scelto non dovesse piacerci? Supponiamo di voler rinominare il file `backup_combined.txt` in `combined_backup.txt`.
 - `mv backup_combined.txt combined_backup.txt`
- Esercizio: rinominare tutte le cartelle con lo spazio che avevamo creato prima.

Cancellare file e cartelle

- Attenzione! La cancellazione di un file o di una cartella da shell non utilizza il meccanismo del cestino
 - Questo vuol dire che un elemento, una volta cancellato, è perso.
- La cancellazione di un file avviene tramite il comando `rm` (remove), seguito da una lista di file da rimuovere. Esempio:
 - `rm dir4/dir5/dir6/combined.txt combined_backup.txt`
- Proviamo ora a rimuovere anche le cartelle `folder_*` che avevamo creato precedentemente.

```
rm: cannot remove 'folder_1': Is a directory
rm: cannot remove 'folder_2': Is a directory
rm: cannot remove 'folder_3': Is a directory
rm: cannot remove 'folder_4': Is a directory
rm: cannot remove 'folder_5': Is a directory
rm: cannot remove 'folder_6': Is a directory
```

Cancellare cartelle

- Per la cancellazione di cartelle dobbiamo utilizzare il comando *rmdir*.
 - Proviamo di nuovo ad eliminare le cartelle *folder_**

```
rmdir: failed to remove 'folder_6': Directory not empty
```

- *rmdir* richiede che la cartella da rimuovere sia vuota. Non deve contenere ne' sottocartelle ne' files.
- Esiste un modo per rimuovere una cartella e tutto il suo contenuto, incluse le sue sottocartelle e, a loro volta, le loro sottocartelle, e così via
 - Si utilizza l'opzione *-r* di *rm* per indicare una rimozione ricorsiva.

Riusciamo a scrivere di meno?

- Vogliamo eseguire i nostri compiti nel modo più veloce possibile, quindi è naturale cercare di scrivere il meno possibile.
- Possiamo utilizzare dei caratteri speciali, come `*` e `?` per abbreviare il comando
- Il carattere speciale `*` vuol dire: sostituisci con zero o più caratteri qualunque. Esempi:
 - `qualcosa*` corrisponde a tutte le stringhe che hanno `qualcosa` come prefisso
 - `*qualcosa` corrisponde a tutte le stringhe che hanno `qualcosa` come suffisso
 - `*qualcosa*` corrisponde a tutte le stringhe che hanno al loro interno `qualcosa` (compresa la stringa `qualcosa`)
- Il carattere speciale `?` vuol dire: sostituisci un carattere qualunque. Esempi:
 - `qualcosa?` corrisponde a tutte le stringhe che hanno `qualcosa` come prefisso e che hanno un ulteriore carattere (es. `qualcosaa`, `qualcosab`, `qualcosa1`, `qualcosa#`, ...)
 - `?qualcosa` corrisponde a tutte le stringhe che hanno `qualcosa` come suffisso, preceduto da un carattere qualunque (es. `aqualcosa`, `9qualcosa`, ...)
 - `?qualcosa?` corrisponde a tutte le stringhe che hanno al loro interno la sottostringa `qualcosa`, preceduto e seguito da un carattere
- Sia `*` che `?` possono anche comparire in posizioni che non siano alle estremità della stringa. Esempio: `qual*cosa`, `qual?cosa`

Scriviamo di meno

- Come posso utilizzare i caratteri che abbiamo visto precedentemente per scrivere in modo piu' conciso: *cat test_1.txt test_2.txt test_3.txt* ?
 - *cat test_?.txt*
 - *cat test_*.txt*
 - *cat test*.txt*
 - *cat test**
 - *cat t**

Creiamo un nuovo file che contenga il contenuto dei vari test

- Come possiamo creare un nuovo file con all'interno il contenuto dei vari file di test che abbiamo scritto prima?
 - `cat t* > combined.txt`
- Cosa succede se ripetiamo questo stesso comando?
- E se invece volessimo effettivamente "appendere" un testo ad un file esistente?
 - Utilizziamo l'operatore `>>`
- Proviamo:
 - `cat t* >> combined.txt`
 - `echo "ho appeso una linea" >> combined.txt`
 - `cat combined.txt`
- Ripetiamo i comandi un po' di volte (provate ad utilizzare la freccia in su), fino ad ottenere un file più lungo della dimensione dello schermo. Come possiamo leggere tale file?
 - `less combined.txt`

Spostare file

- E' possibile spostare un file o una cartella utilizzando il comando *mv* (move)
 - Il comando *mv combined.txt dir1* sposta il file *combined.txt* all'interno della cartella *dir1*
- Verifichiamo se il file è stato effettivamente spostato

```
corso@corso-os:~$ mv combined.txt dir1
corso@corso-os:~$ ls dir1
combined.txt
corso@corso-os:~$
```

- Ora supponiamo di aver cambiato idea e di non volere il file in *dir1*, ma nella directory dove si trovava inizialmente. Come facciamo? Diverse strade, proviamone alcune:
 - Posso specificare come percorsi di partenza e destinazione dei percorsi assoluti
 - Posso spostarmi nella cartella che attualmente contiene *combined.txt* ed utilizzare dei percorsi relativi per spostare il file una directory sopra
 - Posso rimanere dove mi trovo ed utilizzare i percorsi relativi per spostare il file nella directory corrente
 - ...

Spostamenti multipli di file e cartelle

- *mv* consente di spostare con un solo comando diversi elementi in una cartella di destinazione
 - Esempio: *mv combined.txt test_* dir3 dir2* sposta all'interno di *dir2* gli elementi *combined.txt*, *test_**, *dir3*
 - Notare che gli elementi oggetto dello spostamento possono essere sia file che cartelle, mentre l'elemento di destinazione deve necessariamente essere una cartella (non avrebbe senso altrimenti)

```
corso@corso-os:~$ mv combined.txt test_* dir3 dir2
corso@corso-os:~$ ls dir2
combined.txt  dir3  test_1.txt  test_2.txt  test_3.txt
corso@corso-os:~$
```

- Esercizio: spostare ancora una volta il file *combined.txt* da *dir2* a *dir4/dir5/dir6*

Copiare e rinominare

- Il comando per la copia di un file è molto simile a quello per lo spostamento:
 - `cp <percorso di origine> <percorso di destinazione>`
- Come copiamo il file `combined.txt` che sta dentro `dir4/dir5/dir6` all'interno della directory corrente?
 - `cp dir4/dir5/dir6/combined.txt .`
- A questo punto, supponiamo di voler creare una copia di backup di `combined.txt` all'interno della stessa cartella, chiamata `backup_combined.txt`. Come facciamo?
 - `cp combined.txt backup_combined.txt`
- E se il nome che abbiamo scelto non dovesse piacerci? Supponiamo di voler rinominare il file `backup_combined.txt` in `combined_backup.txt`.
 - `mv backup_combined.txt combined_backup.txt`
- Esercizio: rinominare tutte le cartelle con lo spazio che avevamo creato prima.

Cancellare file e cartelle

- Attenzione! La cancellazione di un file o di una cartella da shell non utilizza il meccanismo del cestino
 - Questo vuol dire che un elemento, una volta cancellato, è perso.
- La cancellazione di un file avviene tramite il comando `rm` (remove), seguito da una lista di file da rimuovere. Esempio:
 - `rm dir4/dir5/dir6/combined.txt combined_backup.txt`
- Proviamo ora a rimuovere anche le cartelle `folder_*` che avevamo creato precedentemente.

```
rm: cannot remove 'folder_1': Is a directory
rm: cannot remove 'folder_2': Is a directory
rm: cannot remove 'folder_3': Is a directory
rm: cannot remove 'folder_4': Is a directory
rm: cannot remove 'folder_5': Is a directory
rm: cannot remove 'folder_6': Is a directory
```

Cancellare cartelle

- Per la cancellazione di cartelle dobbiamo utilizzare il comando *rmdir*.
 - Proviamo di nuovo ad eliminare le cartelle *folder_**

```
rmdir: failed to remove 'folder_6': Directory not empty
```

- *rmdir* richiede che la cartella da rimuovere sia vuota. Non deve contenere ne' sottocartelle ne' files.
- Esiste un modo per rimuovere una cartella e tutto il suo contenuto, incluse le sue sottocartelle e, a loro volta, le loro sottocartelle, e così via
 - Si utilizza l'opzione *-r* di *rm* per indicare una rimozione ricorsiva.

Altri comandi utili: wc

- `wc` (word count) consente di contare il numero di parole all'interno di un file
 - Dal manuale di `wc`: "A word is a non-zero-length sequence of characters delimited by white space"
- Non solo – `wc` consente di contare anche:
 - Il numero di byte in un file (opzione `-c`)
 - Il numero di linee in un file (opzione `-l`)
- Proviamo ad utilizzarlo per contare il numero di linee in `combined.txt` ...

```
corso@corso-os:~$ wc -l combined.txt
54 combined.txt
corso@corso-os:~$
```

- ... Ed il numero di caratteri...

```
corso@corso-os:~$ wc -c combined.txt
1044 combined.txt
corso@corso-os:~$
```

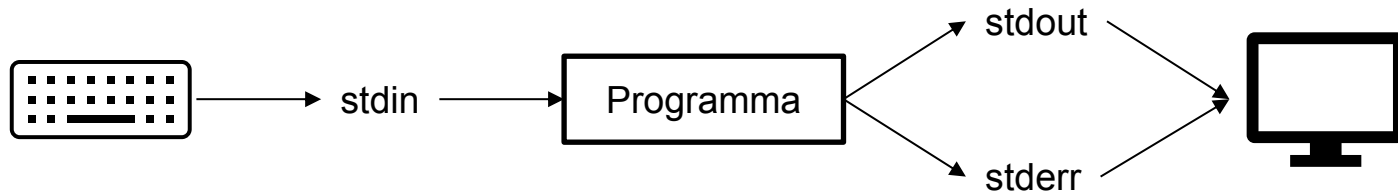
- ... e per finire, senza argomenti

```
corso@corso-os:~$ wc -c combined.txt
54  252  1044 combined.txt
corso@corso-os:~$
```

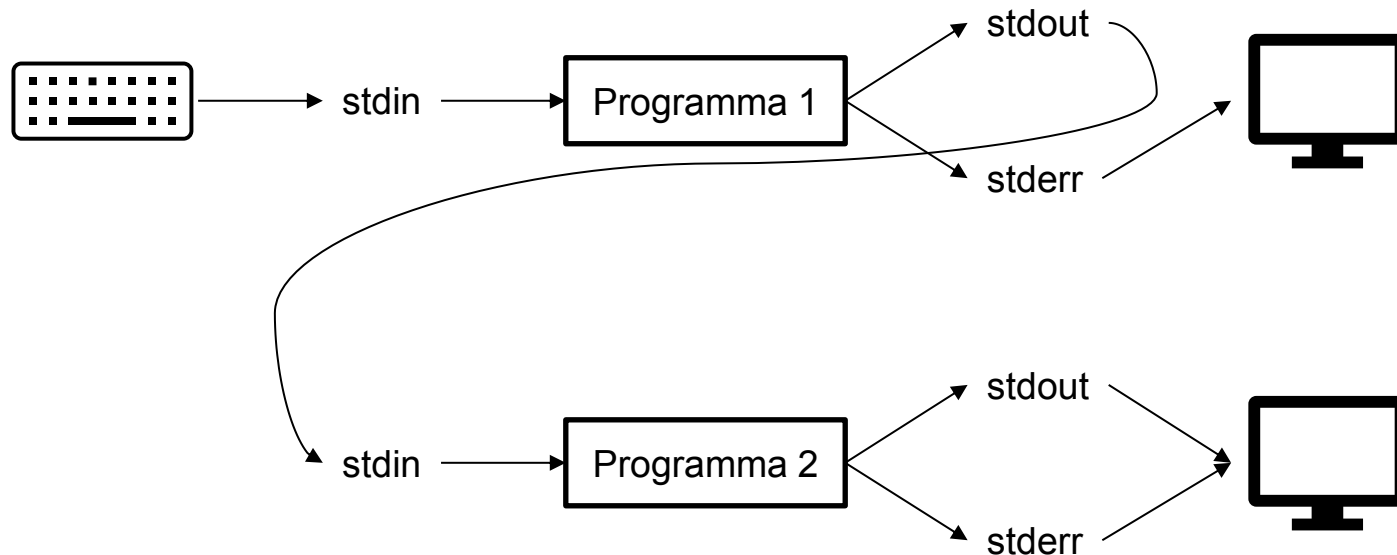
Altri usi di `wc`

- Supponiamo di voler contare gli elementi (files e directory) contenuti nella nostra home directory, andando successivamente a fare un po' di pulizia. Possiamo utilizzare `wc`.
 - `ls ~ > file_list`
 - `wc -l file_list`
 - `rm file_list`
- Funziona, ma è molto scomodo: ho dovuto eseguire tre comandi!

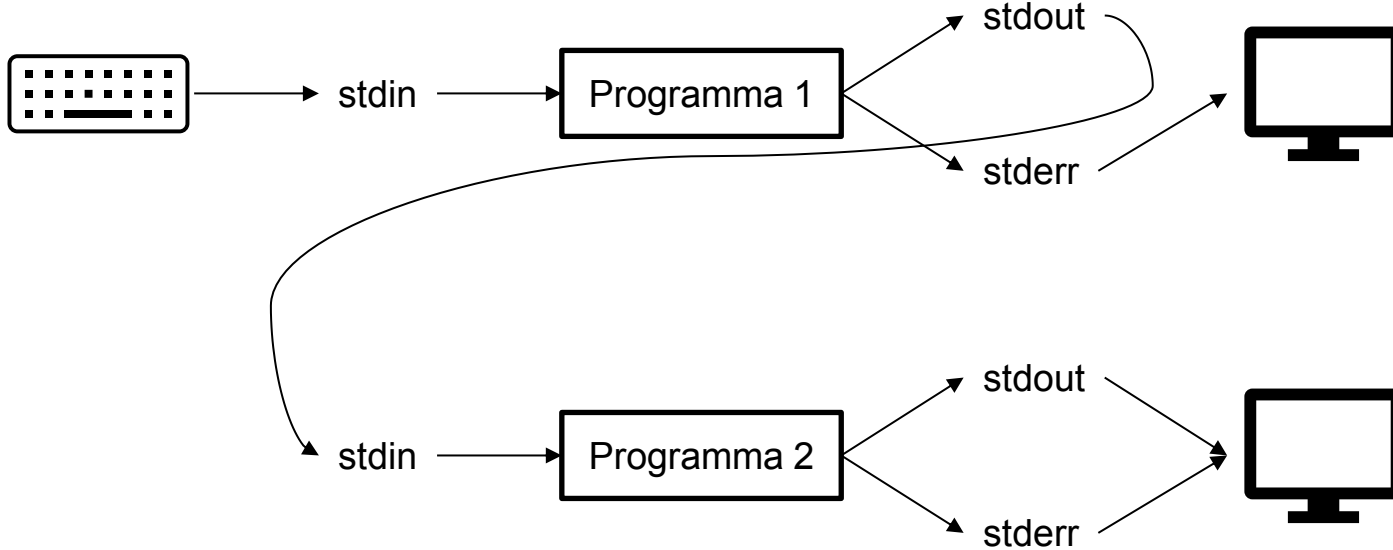
Collegare input e output di comandi



- Abbiamo già visto come redirigere lo standard output su file. Non sarebbe bello poter mandare lo standard output di un primo comando in input ad un secondo comando? Graficamente:

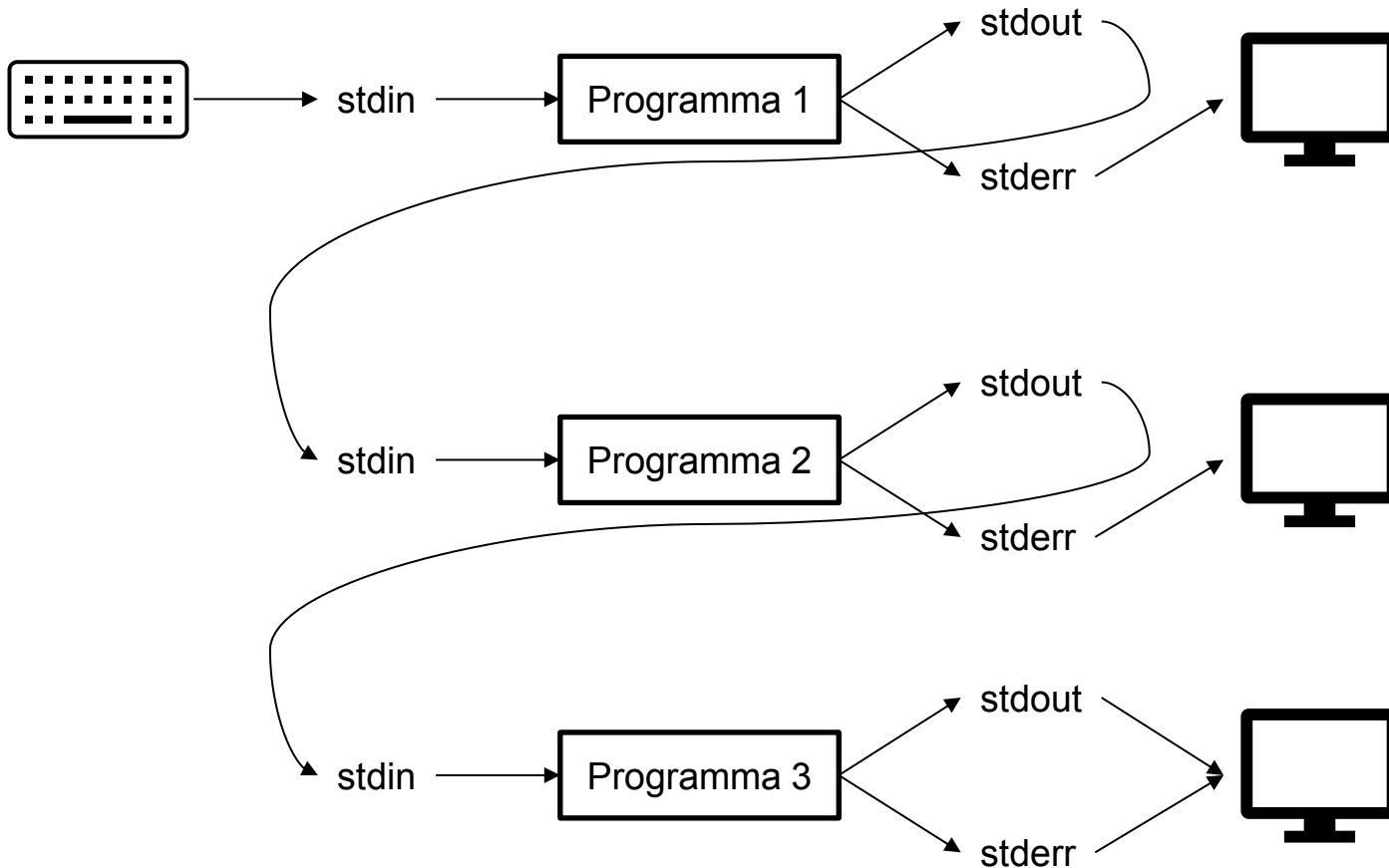


Il meccanismo di piping



- Questo schema è realizzabile tramite un meccanismo di comunicazione chiamato *pipe*.
 - Una *pipe* viene rappresentata dal carattere |
 - Esempio: `ls ~ | wc -l`
 - Altro esempio: `ls /etc | less`

Pipe multiple



- Posso replicare questo schema quante volte voglio.
 - Esempio: supponiamo di voler conoscere il numero di righe identiche nel file *combined.txt*
`cat combined.txt | uniq | wc -l`

sort, uniq

- Qualcosa non ha funzionato nel comando precedente. Cerchiamo di capire perché

```
corso@corso-os:~$ uniq --help
Usage: uniq [OPTION]... [INPUT [OUTPUT]]
Filter adjacent matching lines from INPUT (or standard input),
writing to OUTPUT (or standard output).
```

- Quindi il problema sta nel fatto che le righe uguali non sono adiacenti, quindi *uniq* non è stato in grado di scovarle. Come possiamo fare per renderle adiacenti?
 - Ci può aiutare il comando *sort*. Proviamo ad eseguire: `sort combined.txt | less`
- A questo punto, riscriviamo il nostro comando per contare le righe uniche nel file:

```
corso@corso-os:~$ sort combined.txt | uniq | wc -l
3
corso@corso-os:~$
```

Introduction to Operating Systems

Operating System: Three Easy Pieces

What happens when a program runs?

A running program executes instructions.

1. The processor ^{↑ RECOVER} **fetches** an instruction from memory.
2. **Decode**: Figure out which instruction this is
3. **Execute**: i.e., add two numbers, access memory, check a condition, jump to function, and so forth.
4. The processor moves on to the **next instruction** and so on.

This is also known as the **Von Neumann model** of computing.

Expectations for today's machines are much higher than this. For instance, multiple applications running at the same time.

Operating System (OS)

Responsible for

- ◆ Making it easy to **run** programs
- ◆ Allowing programs to **share** memory
- ◆ Enabling programs to **interact** with devices

**OS is in charge of making sure the system operates
correctly and efficiently.**

Virtualization

The OS takes a **physical resource** and transforms it into a **virtual form** of itself.

- ◆ **Physical resource:** Processor, Memory, Disk ...
- ◆ The virtual form is more general, powerful and easy-to-use.
- ◆ Sometimes, we refer to the OS as a **virtual machine**.

System call

So, how can we use all these nice features provided by an OS?

System calls allow users **to tell the OS what to do**.

- The OS provides some interfaces (APIs, standard library).
- A typical OS exports a few hundred system calls.
 - Run programs
 - Access memory
 - Access devices

The OS is a resource manager.

- The OS **manages resources** such as *CPU*, *memory* and *disk*.
- The OS allows
 - Many programs to run → Sharing the CPU
 - Many programs to *concurrently* access their own instructions and data → Sharing memory
 - Many programs to access devices → For example, sharing disks

Virtualizing the CPU

- A computer system has a certain number of physical/virtual CPUs.
 - For example, my laptop has 20 cores in a single processing unit.
 - A server machine can have tens of cores spread across several processing units
- Usually, the number of applications running on our machines is much larger than the number of available CPUs or cores
- The OS turns a single CPU into a seemingly infinite number of CPUs (it's just an illusion!).
 - Allowing many programs to seemingly run at once → **Virtualizing the CPU**

Virtualizing the CPU (Cont.)

```
1      #include <stdio.h>
2      #include <stdlib.h>
3      #include <sys/time.h>
4      #include <assert.h>
5      #include "common.h"
6
7      int
8      main(int argc, char *argv[])
9      {
10         if (argc != 2) {
11             fprintf(stderr, "usage: cpu <string>\n");
12             exit(1);
13         }
14         char *str = argv[1];
15         while (1) {
16             Spin(1); // Repeatedly checks the time and
17                      // returns once it has run for a second
18             printf("%s\n", str);
19         }
20         return 0;
21     }
```

Annotations:

- Line 10: *CONTROLLA CHE NON VENGA INSERITO NOME DI INSOLITO* (with arrow pointing to `argc != 2`)
- Line 15: *FUNZIONE CHE FA ASPETTARE UN SECONDO* (with arrow pointing to `Spin(1)`)
- Line 18: *STAMPA IL CARATTERE INSERITO* (with arrow pointing to `printf`)

Simple Example(cpu.c): Code That Loops and Prints

Virtualizing the CPU (Cont.)

- Execution result 1.

```
prompt> gcc -o cpu cpu.c -Wall
prompt> ./cpu "A"
A
A
A
^C
prompt>
```

Run forever; Only by pressing "Control-c" can we halt the program

Virtualizing the CPU (Cont.)

- Execution result 2.

```
prompt> ./cpu A & ./cpu B & ./cpu C & ./cpu D &
[1] 7353
[2] 7354
[3] 7355
[4] 7356
A
B
D
C
A
B
D
C
A
C
B
D
...
```

PER LANCIARE PIU' DI UN PROCESSO CONTEMPORANEAMENTE

PER SMETTERE TUTTI I PROCESSI:
KILLALL CPU

Even though we have only **one processor**, all four of programs seem to be running **at the same time!**

Virtualizing Memory

- The physical memory is *an array of bytes*.
- A program keeps all of its data structures in memory.
 - **Read memory** (load):
 - Specify an address to be able to access the data
 - **Write memory** (store):
 - Specify the data to be written to the given address

Virtualizing Memory (Cont.)

- A program that Accesses Memory (mem.c)

```
1      #include <unistd.h>
2      #include <stdio.h>
3      #include <stdlib.h>
4      #include "common.h"
5
6      int
7      main(int argc, char *argv[])
8      {
9          int *p = malloc(sizeof(int)); // a1: allocate some
                                         memory
10         assert(p != NULL); → UNIF CHE
                               SE NON RISPETTA LA CONDIZIONE TERMINIL PROGRAMMA
11         printf("(d) address of p: %08x\n",
12                getpid(), (unsigned) p); // a2: print out the
                                         address of the memmory
13         *p = 0; // a3: put zero into the first slot of the memory
14         while (1) {
15             Spin(1);
16             *p = *p + 1;
17             printf("(d) p: %d\n", getpid(), *p); // a4
18         }
19         return 0;
20     }
```

Virtualizing Memory (Cont.)

- The output of the program `mem.c`

```
prompt> ./mem 0
(2134) memory address of p: 00200000
(2134) p: 1
(2134) p: 2
(2134) p: 3
(2134) p: 4
(2134) p: 5
^C → PCA
      TERMINARE UN SINGOLO PROCESSO
```

- The newly allocated memory is at address 00200000.
- It updates the value and prints out the result.

Virtualizing Memory (Cont.)

- Set to 0 /proc/sys/kernel/randomize_va_space
- Running `mem.c` multiple times

```
prompt> ./mem 0 & ./mem 0 &  
[1] 24113  
[2] 24114  
(24113) memory address of p: 00200000  
(24114) memory address of p: 00200000  
(24113) p: 1  
(24114) p: 1  
(24114) p: 2  
(24113) p: 2  
(24113) p: 3  
(24114) p: 3  
...
```

- It is as if each running program has its **own private memory**.
 - Each running program has allocated memory at the same address.
 - Each seems to be updating the value at `00200000` independently.

Virtualizing Memory (Cont.)

- Each process accesses its own private **virtual address space**.
 - The OS maps **address space** onto the **physical memory**.
 - A memory reference within one running program does not affect the address space of other processes.
 - Physical memory is a shared resource, managed by the OS.

The problem of Concurrency

- The OS is juggling **many things at once**, first running one process, then another, and so forth.
- Modern **multi-threaded programs** also exhibit the concurrency problem.

Concurrency Example (Cont.)

```
16     int
17     main(int argc, char *argv[])
18     {
19         if (argc != 2) {
20             fprintf(stderr, "usage: threads
21                             <value>\n");
22             exit(1);
23         }
24         loops = atoi(argv[1]);
25         pthread_t p1, p2;
26         printf("Initial value : %d\n", counter);
27         pthread_create(&p1, NULL, worker, NULL);
28         pthread_create(&p2, NULL, worker, NULL);
29         pthread_join(p1, NULL);
30         pthread_join(p2, NULL);
31         printf("Final value : %d\n", counter);
32         return 0;
33     }
```

*TUTTO VIENE ESEGUITO
SEGUENDO UN FILO (QUELLO PRINCIPALE)*

CREAZIONE DI NUOVI "FILI"

- The main program creates **two threads**.
 - Thread: a function running within the same memory space. Each thread starts running in a routine called `worker()`.
 - `worker()`: increments a counter

Concurrency Example

A Multi-threaded Program (thread.c)

```
1      #include <stdio.h>
2      #include <stdlib.h>
3      #include "common.h"
4
5      int counter = 0;
6      int loops;
7
8      void *worker(void *arg) {
9          int i;
10         for (i = 0; i < loops; i++) {
11             counter++;
12         }
13         return NULL;
14     }
15     ...
```

Concurrency Example (Cont.)

- `loops` determines how many times each of the two workers will **increment the shared counter** in a loop.

- `loops: 1000.`

```
prompt> gcc -o threads threads.c -Wall -pthread
prompt> ./thread 1000
Initial value : 0
Final value : 2000
```

- `loops: 100000.`

```
prompt> ./thread 100000
Initial value : 0
Final value : 143012 // huh??
prompt> ./thread 100000
Initial value : 0
Final value : 137298 // what the??
```

How many instructions are in *counter++*?

A Multi-threaded Program (thread.c)

```
1      #include <stdio.h>
2      #include <stdlib.h>
3      #include "common.h"
4
5      int counter = 0;
6      int loops;
7
8      void *worker(void *arg) {
9          int i;
10         for (i = 0; i < loops; i++) {
11             counter++;
12         }
13         return NULL;
14     }
15     ...
```

↪ COUNTER = COUNTER + 1
↓
ASSEGNA AL PRIMO DI
MEMORIA COUNTER, COUNTER+1

Why is this happening?

- Increment a shared counter → takes three instructions.
 - Load the value of the counter from memory into register.
 - Increment it
 - Store it back into memory
- These three instructions do not execute **atomically**. → **Concurrency** problem.

NON VENGONO
CONSIDERATE
COME UN'ISTRUZIONE UNICA
MA COME
TRE DIVERSE

Persistence

- Devices such as DRAM store values in a volatile fashion.
- *Hardware* and *software* are needed to store data **persistently**.
 - **Hardware:** I/O device such as a hard drive or solid-state drives (SSDs)
 - **Software:**
 - File system manages the disk.
 - File system is responsible for storing any files the user creates.

Persistence (Cont.)

- Create a file (`/tmp/file`) that contains the string "hello world"

```
1      #include <stdio.h>
2      #include <unistd.h>
3      #include <assert.h>
4      #include <fcntl.h>
5      #include <sys/types.h>
6
7      int
8      main(int argc, char *argv[])
9      {
10         int fd = open("/tmp/file", O_WRONLY | O_CREAT
11                        | O_TRUNC, S_IRWXU);
12         assert(fd > -1);
13         int rc = write(fd, "hello world\n", 13);
14         assert(rc == 13);
15         close(fd);
16         return 0;
17     }
```

`open()`, `write()`, and `close()` system calls are routed to the part of OS called the file system, which handles the requests

↳ MESSE
NEWA DIRECTORY OF
FILE SYSTEM

Persistence (Cont.)

- What does the OS do in order to write to disk?
 - Deals with additional abstraction layers, such as RAID, Logical Volume Manager, and so forth
 - Figures out **where** on disk this new data will reside
 - **Issue I/O** requests to the underlying storage device
- File system handles system crashes during write.
 - **Journaling** or **copy-on-write**
 - Carefully ordering writes to disk

Design Goals

- Build up **abstraction**
 - Make the system convenient and easy to use.
- Provide high **performance**
 - Minimize the overhead of the OS.
 - OS must strive to provide virtualization without excessive overhead.
- **Protection** between applications
 - Isolation: Bad behavior of one does not harm other and the OS itself.

Design Goals (Cont.)

- High degree of **reliability**
 - The OS must also run non-stop.
- Other issues
 - Energy-efficiency
 - Security
 - Mobility

4. The Abstraction: The Process

Operating System: Three Easy Pieces

How to provide the illusion of many CPUs?

- Process: running application
 - An application, or program, is just a bunch of instructions on some storage device
 - The OS is in charge of loading the application/program and start a process
- We usually run many processes on a computer that only has few CPUs. How?
 - **Time sharing**: Running one process, then stopping it and running another
 - The potential cost is **performance**.
 - Stopping a process and running another process is named **context switching**
 - The OS can promote the illusion that many virtual CPUs exist.
- Context Switching is the **mechanism** we use to implement time sharing
 - On top of context switching we can define **policies** to make decisions on *when* to stop a certain process, *which* process to start, for *how long*, ...

→ APPLICAZIONE IN ESECUZIONE

→ ASSEGNO UN UNITA' BEN DEFINITA DI TEMPO A UN PROCESSO QUANDO QUESTO QUANDO DI TEMPO FINISCE POSSO PASSARE A UN ALTRO PROCESSO

→ CAMBIO REPENTINO DI PROCESSO

→ NON INDICA PERO' QUANDO FERMARE E QUANDO PARTIRE

→ DI QUESTO SE NE OCCUPA LA POLITICA

A process is a **running program.**

- Yes, but what does it exactly contain? How can we characterize it?
- A process can be characterized by its machine state
 - The machine state is what a program can read or write when it is running
- Components of a process:
 - Memory (address space)
 - Instructions
 - Data section
 - Registers
 - Program counter
 - Stack pointer

Process API

- The OS is in charge of controlling the life-cycle of processes. To this end, it implements the following functions:
 - **Create**
 - Create a new process to run a program
 - **Destroy**
 - Halt a runaway process
 - **Wait**
 - Wait for a process to stop running
 - **Miscellaneous Control**
 - Some kind of method to suspend a process and then resume it
 - **Status**
 - Get some status info about a process

Process Creation

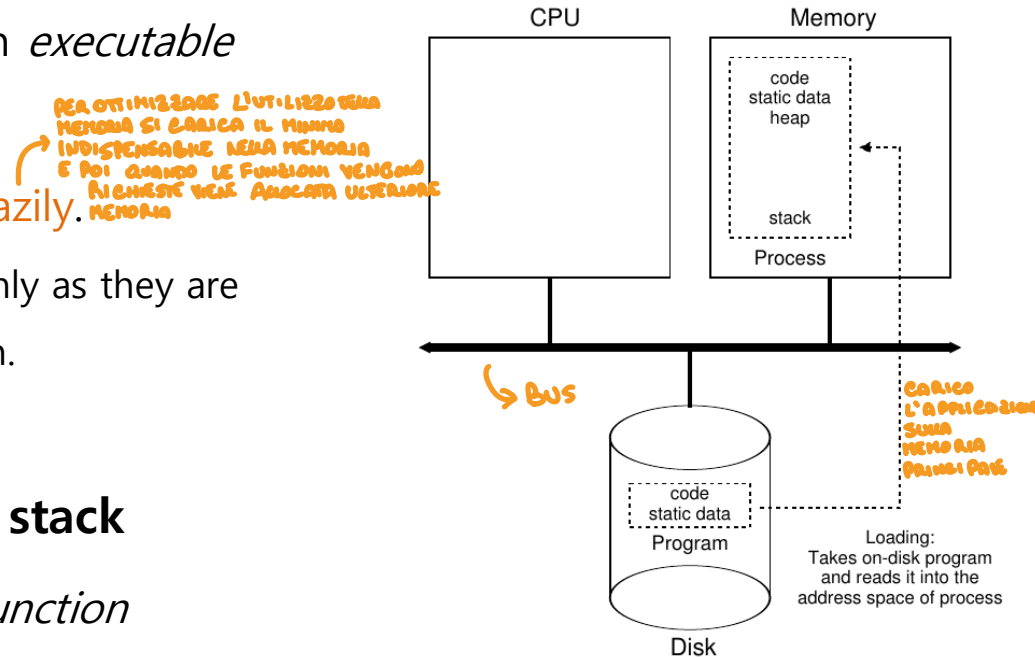
1. **Load** a program code into memory, into the address space of the process.

- Programs initially reside on disk in *executable format*.

- OS perform the loading process **lazily**.
 - Loading pieces of code or data only as they are needed during program execution.

2. **Allocate** the program's run-time **stack**

- stack is used for *local variables*, *function parameters*, and *return address*.
- Initialize the stack with arguments (e.g., `argc` and the `argv` array of `main()` function)



Process Creation (Cont.)

3. **Create** the program's **heap**

- Used for explicitly requested dynamically allocated data.
- Programs request such space by calling `malloc()` and free it by calling `free()`.

4. **Initialize** other stuff

- input/output (I/O) setup
 - Each process by default has three open file descriptors.
 - Standard input, output and error

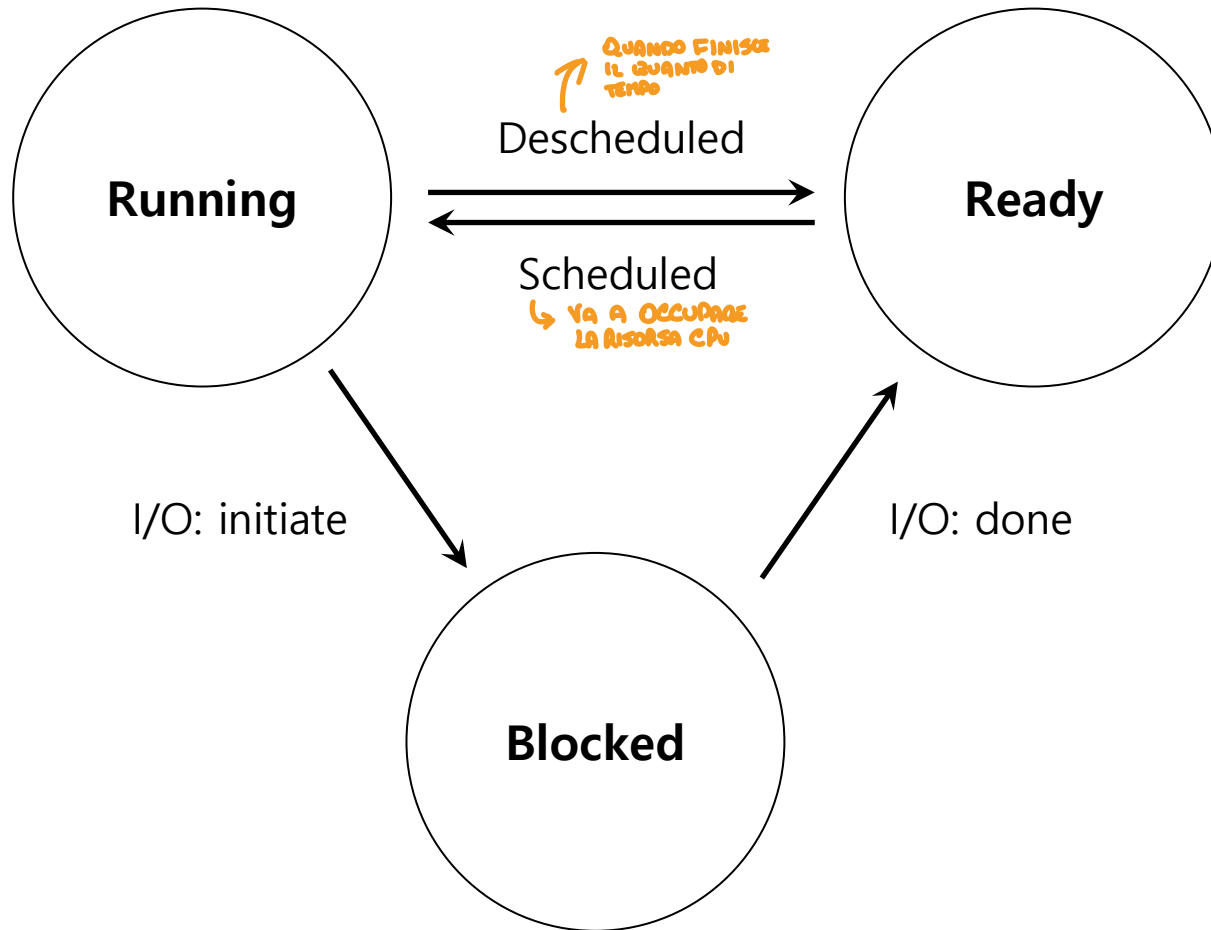
5. **Start the program** running at the entry point, namely `main()`.

- The OS *transfers control* of the CPU to the newly-created process.

Process States

- A process can be in one of three states.
 - **Running**
 - A process is running on a processor.
 - **Ready**
 - A process is ready to run but, for some reason, the OS has chosen not to run it at this given moment.
 - **Blocked**
 - A process has requested to perform some kind of operation.
 - When a process initiates an I/O request to a disk, it becomes blocked and thus some other process can use the processor.
- There are actually more states, but this is a good approximation for now

Process State Transition



Tracing Process State (no I/O)

Time	Process ₀	Process ₁	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	
4	Running	Ready	Process ₀ now done
5	–	Running	
6	–	Running	
7	–	Running	
8	–	Running	Process ₁ now done

IL PROCESSO NON È DETTO CHE FINISCA NEL SUO QUANTO DI TEMPO

Figure 4.3: Tracing Process State: CPU Only

Tracing Process State

Time	Process ₀	Process ₁	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	Process ₀ initiates I/O
4	Blocked	Running	Process ₀ is blocked,
5	Blocked	Running	so Process ₁ runs
6	Blocked	Running	
7	Ready	Running	I/O done
8	Ready	Running	Process ₁ now done
9	Running	–	
10	Running	–	Process ₀ now done

Figure 4.4: Tracing Process State: CPU and I/O

Data structures

- Just like any other program, the OS uses data structures to organize information.
- The OS must track the execution of the processes. There are some data structures that are used for this purpose

Example) The xv6 kernel Proc Structure

```
// the registers xv6 will save and restore
// to stop and subsequently restart a process
struct context {
    int eip;    // Instruction pointer register
    int esp;    // Stack pointer register
    int ebx;    // Called the base register
    int ecx;    // Called the counter register
    int edx;    // Called the data register
    int esi;    // Source index register
    int edi;    // Destination index register
    int ebp;    // Stack base pointer register
};

// the different states a process can be in
enum proc_state { UNUSED, EMBRYO, SLEEPING,
                  RUNNABLE, RUNNING, ZOMBIE };
```

Example) The xv6 kernel Proc Structure (Cont.)

```
// the information xv6 tracks about each process
// including its register context and state
struct proc {
    char *mem; // Start of process memory
    uint sz; // Size of process memory
    char *kstack; // Bottom of kernel stack
                // for this process
    enum proc_state state; // Process state
    int pid; // Process ID
    struct proc *parent; // Parent process
    void *chan; // If non-zero, sleeping on chan
    int killed; // If non-zero, have been killed
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd; // Current directory
    struct context context; // Switch here to run process
    struct trapframe *tf; // Trap frame for the
                        // current interrupt
};
```

Puntuale all'inizio della memoria

6. Mechanism: Limited Direct Execution

Operating System: Three Easy Pieces

How to efficiently virtualize the CPU with control?

- The OS needs to share the physical CPU by **time sharing**.
- Issues:
 - **Performance:** How can we implement virtualization without adding excessive overhead to the system?

REALIZZARE IL CONTEXT SWITCHING
CON MENO OVERHEAD
POSSIBILE → INSIEME DI ISTRUZIONI
CHE POSSO EVITARE DI ESEGUIRE
 - **Control:** How can we run processes efficiently while retaining control over the CPU?
 - If the OS loses control, then a process might run forever, without ever releasing resources
 - Without control, the OS cannot perform security checks (e.g., is that process allowed to access that file?)

Solution for Performance: Direct Execution

- Just run the program directly on the CPU.

OS	Program
1. Create entry for process list 2. Allocate memory for program 3. Load program into memory 4. Set up stack with <code>argc / argv</code> 5. Clear registers 6. Execute call <code>main()</code> ↪ ARGOMENTI DA ALGA DI COMANDO	7. Run <code>main()</code> 8. Execute <code>return from main()</code>
9. Free memory of process 10. Remove from process list	

Without *limits* on running programs, the OS wouldn't be in control of anything (e.g., context switch or security) and thus would be "just a library".

Problem 1: Restricted Operation

- What if a process wishes to perform some kind of restricted operation such as
 - Issuing an I/O request to a disk
 - Gaining access to more system resources such as CPU or memory
- Can we just let the process access disk/CPU/memory?
 - Again, we must think that we don't have only a single running process
 - What if we want to implement a permission mechanism according to which some process cannot access some other process' data?
- **Solution:** we create two working modes. One for the regular processes (user mode), and the other for the kernel (kernel mode)
 - **User mode:** Applications do not have full access to hardware resources.
 - **Kernel mode:** The OS has access to the full resources of the machine
 - **System calls** are exposed to let processes access kernel-mode functionalities

SONO DEI
PROCESSI PARTICOLARI, DIVERSI DA
QUELLI ESEGUITI
DAGLI UTENTI

System Call

→ SALTO DA
UN PIANO DI LAVORO A UN ALTRO PIANO DI LAVORO

- Allow the kernel to **carefully expose** certain key pieces of functionality to user program, such as ...
 - Accessing the file system
 - Creating and destroying processes
 - Communicating with other processes
 - Allocating more memory

System Call (Cont.)

- To execute a system call, a program runs a special **trap** instruction, which performs together:
 - Jump into the kernel → SALTO SU UNA PARTICOLARE ISTRUZIONE DEL KERNEL
 - Raise the privilege level to kernel mode → GARANTIRE LA DISPONIBILITA' DELL'INTERO SET DI ISTRUZIONI
- **Return-from-trap** instruction performs together:
 - Return into the calling user program → ISTRUZIONE SUCCESSIVA ALLA TRAP
 - Reduce the privilege level back to user mode
- Problem: we cannot just jump from user mode to kernel mode at any address
 - This would allow us to run arbitrary pieces of code in the kernel, which could be very dangerous
- The kernel sets up a **trap table** at boot time, where a number is associated to every exposed system call

Limited Direct Execution Protocol

OS @ boot
(kernel mode)

Hardware

initialize trap table

remember address of
syscall handler

OS @ run
(kernel mode)

Hardware

Program
(user mode)

Create entry for process list
Allocate memory for program
Load program into memory
Setup user stack with argv
Fill kernel stack with reg/PC

return-from-trap

populate regs from kernel stack
move to user mode
jump to main

Run main()

...
Call system call
trap into OS

A SEGUITO DELL'AVVIO
DI UN PROGRAMMA E QUINDI
ALLA CREAZIONE DI UN
PROCESSO

PER FARE
ACCESSO IN MEMORIA
O SCRITTURA O
ECC.

Limited Direct Execution Protocol (Cont.)

OS @ run
(kernel mode)

Hardware

Program
(user mode)

(Cont.)

↑
SERVE PER RIPRENDERE
L'ESECUZIONE DEL PROCESSO
ESATTAMENTE DOVE LO
AVEVAMO LASCIATO, SERBANDO
I VALORI CHE AVEVAMO LASCIATO

save regs to kernel stack
move to kernel mode
jump to trap handler → GESTORE TRAP

Handle trap
Do work of syscall
return-from-trap

restore regs from kernel stack
move to user mode
jump to PC after trap

Free memory of process
Remove from process list

...
return from main
trap (via `exit()`)

→ SERVE PER
FINIRE IL PROCESSO
E RIMUOVERE LA STRUTTURA DATI

Problem 2: Switching Between Processes

- Switching between processes should be easy and should introduce as little overhead as possible → it is an operation that is performed multiple times every second
PERFORMANCE → NON METTERE MOLTA TEMPO A PASSARE IL CPU
 - Ideally, the OS should just stop one process, save its CPU state in memory, load the CPU state of another process from memory, and let the latter run.
 - Problem: if a user-mode process is running on a CPU this means, by definition, that the kernel is not running at that time. If the kernel is not running, how can it do anything at all?
- How can the OS **regain control** of the CPU so that it can switch between *processes*?
 - A cooperative Approach: **Wait for system calls**
 - A Non-Cooperative Approach: **The OS takes control**

A cooperative Approach: Wait for system calls

- Processes **periodically give up the CPU** by making **system calls** such as `yield()`.
 - The OS decides to run some other task.
- Applications also transfer control to the OS when they do something illegal.
 - Divide by zero
 - Try to access memory that it shouldn't be able to access
- Using this cooperative approach is often less than optimal:
 - A process is only a single piece of the entire environment, and does not have visibility over its entirety → it cannot make decisions based on the global situation.
 - What if a process is malicious and purposely never runs any system call?
 - What if a process has some bugs that don't allow it to exit an infinite loop?

A Non-Cooperative Approach: OS Takes Control

- The OS needs some collaboration from the hardware to realize the non-cooperative approach.
- **A timer interrupt** *! I PROCESSI NON SANNO DEL TIMER INTERRUPT!*
 - During the boot sequence, the OS starts a timer.
 - The timer raises an interrupt every so many milliseconds.
 - When the interrupt is raised :
 - The currently running process is halted. *↳ Arrestato*
 - The state of the program is saved
 - A pre-configured interrupt handler in the OS runs.

A timer interrupt gives OS the ability to run again on a CPU.

Saving and Restoring Context

- **Scheduler** makes a decision:
 - Whether to continue running the **current process**, or to switch to a **different one**.
 - If the decision is made to switch, the OS executes a context switch.

Limited Direct Execution Protocol (Timer interrupt)

OS @ boot
(kernel mode)

Hardware

initialize trap table

remember address of:

- syscall handler
- timer handler

start interrupt timer

PER NON FAR
MONOPOLIZZARE IL PROCESSORE
DA UN SINGOLO PROCESSO

start timer
interrupt CPU in X ms

OS @ run
(kernel mode)

Hardware

Program
(user mode)

Process A
...

timer interrupt

save regs(A) to k-stack(A)
CPU switch to kernel mode
jump to trap handler

FONDAMENTALE CHE VENGA GESTITO
IL TIMER INTERRUPT DALL' HARDWARE

Limited Direct Execution Protocol (Timer interrupt)

OS @ run
(kernel mode)

Hardware

Program
(user mode)

(Cont.)

Handle the trap
Call switch() routine
 save regs(A) to proc-struct(A)
 restore regs(B) from proc-struct(B)
 switch to k-stack(B)
return-from-trap (into B)

restore regs(B) from k-stack(B)
move to user mode
jump to B's PC

CONTEXT SWITCH
Process B
...

Worried About Concurrency?

- What happens if, during interrupt or trap handling, another interrupt occurs?
- OS handles these situations:
 - **Disables interrupts** during interrupt processing
 - Uses a number of sophisticated **locking** schemes to protect concurrent access to internal data structures.
- We will deep dive into these issues in the second “piece” of the course, which will regard concurrency.

→ PUO' SUCCESSERE DI SCARICARE I TIMER INTERRUPT

Introduzione ai Sistemi Operativi

Un programma in esecuzione esegue istruzioni.

Il processore recupera un'istruzione dalla memoria

La decodifica

La esegue

Successivamente si sposta sulla prossima istruzione (Modello di Von Neumann).

Il sistema operativo è responsabile di rendere semplice l'esecuzione di programmi, permettere ai programmi di condividere la memoria, permettere ai programmi di interagire con gli altri dispositivi. Il sistema operativo ha il compito di controllare che il sistema lavori in modo efficiente e corretto. Il sistema operativo prende una risorsa fisica e la trasforma in una virtuale. Sostanzialmente grazie alle chiamate di sistema permettiamo all'utente di dire cosa fare al sistema operativo. Quindi un sistema operativo gestisce le risorse, condivisione della cpu e condivisione delle memorie. Il sistema operativo virtualizzando la cpu ci dà l'impressione che questa utilizzi un numero infinito di cpu (Guarda cpu.c). Per quanto riguarda la virtualizzazione della memoria la memoria viene letta: specificando un indirizzo dove accedere ai dati e viene scritta: specificando i dati da scrivere all'indirizzo evocato (Guarda mem.c). Nella virtualizzazione della memoria ogni processo accede al suo spazio di indirizzi virtuali privati. Il sistema operativo mappa lo spazio degli indirizzi sulla memoria fisica. La memoria fisica è una risorsa condivisa gestita dal sistema operativo. Un altro problema è quello della concorrenza dato dai programmi multi thread. Si può immaginare che il nostro programma segua un filo principale e che durante l'esecuzione di possano creare dei nuovi fili secondari (Guarda threads.c). Quando in un programma abbiamo il semplice incremento di una variabile, in realtà stiamo eseguendo tre istruzioni: caricare il valore della variabile dalla memoria al registro, incrementarla e rimetterla nella memoria; Queste tre istruzioni non vengono eseguite atomicamente, vengono infatti considerate non un'unica istruzione ma tre diverse. Persistenza, hardware e software devono essere in grado di contenere dati persistentemente (file, dati...), Il sistema operativo si occupa di capire dove andranno messi i nuovi dati, infatti emette delle richieste al dispositivo di archiviazione sottostante. I goal del sistema operativo sono quindi: costruire un'astrazione e quindi rendere il sistema semplice da utilizzare, fornire delle buone performance, protezione tra le applicazioni e essere affidabile.

Il processo di astrazione

Processo: applicazione in esecuzione (Un'applicazione è un insieme di istruzioni su un qualche dispositivo di archiviazione).

Il meccanismo che ci permette di eseguire più programmi con poche cpu è il time sharing, consiste nell'assegnare un'unità di tempo (quanto) a un processo, quando questa porzione di tempo finisce ho due opzioni 1. Finisco il processo che stavo eseguendo e passo a un altro 2. Non sono riuscito a finire il processo lo blocco momentaneamente per passare a un altro. Il meccanismo di stoppare un processo e passare a un altro viene detto context switching. Ovviamente il context switching è basato su alcune policy che prendono la decisione su quando stoppare un processo per quanto ecc... Tornando ai processi un processo è composto da la parte della memoria come istruzioni e dati e la parte dei registri come program counter e stack pointer. Il sistema operativo si occupa di controllare il ciclo di vita dei processi, lo fa con le seguenti funzioni: lo crea, lo distrugge, può metterlo in stato di wait, gli fa i vari controlli e ottiene informazioni sullo stato del processo. Riguardo invece alla creazione di un processo: viene caricato il codice di un programma nella memoria, nello spazio degli indirizzi di un processo, si dice che il sistema operativo carica i processi in maniera "pigra" (Per ottimizzare

l'utilizzo della memoria si carica il minimo indispensabile nella memoria e poi quando le funzioni vengono richieste viene allocata ulteriore memoria). Viene allocato lo stack a tempo di esecuzione, solitamente lo stack è utilizzato per variabili locali, parametri di funzioni e return address. Viene creato l'heap del programma, utilizzato per i dati allocati dinamicamente (malloc ()), vengono inizializzate le altre cose e successivamente viene fatto eseguire il programma. Un processo può avere tre stati: in esecuzione (running) , pronto (ready): il processo è pronto per andare in esecuzione ma per qualche motivo il sistema operativo ha deciso di non eseguirlo in questo momento, bloccato (blocked). Il sistema operativo ha delle strutture dati per organizzare le informazioni es. xv6 kernel proc structure. Ritornando al time sharing l'obiettivo del sistema operativo è quello di realizzare il context switching con meno overhead possibile, ovvero tutto quell'insieme di istruzioni che posso evitare di eseguire. Un altro obiettivo è quello di avere, da parte del SO, il controllo di quello che sta succedendo. Alcuni processi hanno bisogno di effettuare operazioni che non tutti potrebbero fare, per questo motivo nasce la problematica dell'implementare un meccanismo che gestisca i permessi di questi processi. La soluzione è stata creare due modelli di lavoro. Uno per i processi regolari che non hanno accesso a tutte le risorse hardware, detto User Mode e l'altro per dei processi particolari che hanno accesso a tutte le risorse detto Kernel Mode. Le chiamate di sistema fanno sì che i processi abbiano accesso alle funzionalità della kernel-mode. Per eseguire una chiamata di sistema un programma lancia un'istruzione trappola che salta nel kernel e aumenta il livello di priorità a kernel-mode. Dopo quest'istruzione trappola tutto ritorna come prima e il livello di privilegio ridiventa User Mode. Ovviamente dato che saltare da una modalità all'altra sarebbe un pericolo il Kernel ha una Trap table dove un numero è associato a ogni chiamata di sistema. Come fa il sistema operativo a riprendere il controllo della cpu, in modo da poter passare da un processo a un altro: ci sono due approcci, Uno cooperativo (aspettare per le chiamate di sistema) e uno non cooperativo (il sistema operativo prende il controllo). I processi cedono regolarmente la CPU effettuando chiamate di sistema come yield(). Le applicazioni trasferiscono il controllo al sistema operativo quando stanno facendo qualcosa di illegale (divisione per 0). L'utilizzo di questo approccio è spesso meno che ottimale: uno dei motivi potrebbe essere un processo cattivo che propositamente non fa nessuna chiamata di sistema. Per quanto riguarda l'altro approccio al sistema operativo serve che l'hardware sia collaborativo per realizzare un approccio non cooperativo. Timer interrupt: Durante la sequenza di avvio il sistema operativo inizializza un timer, il timer genera un'interruzione ogni tot tempo. Quando viene generata un'interruzione il processo in esecuzione viene arrestato, lo stato del programma viene salvato, un gestore di interruzioni preconfigurato nel sistema operativo viene eseguito, Questo Timer Interrupt consente al sistema operativo di riprendere l'esecuzione su una CPU. Quando si verifica un Timer Interrupt il sistema operativo si occupa di disabilitare le interruzioni date dal timer e si occupa di utilizzare degli schemi che impediscono l'accesso concorrente.

7. Scheduling:

Operating System: Three Easy Pieces

Scheduling: Introduction

- We have seen how a context switch can be practically implemented, but we still have to discuss what policies can be realized on top of that mechanism
- Before we can introduce scheduling policies, we must make some assumption on the workload (the processes that are supposed to run on the machine)
- This assumptions will be initially quite strong and unrealistic, but we will relax them as we proceed.
- Workload assumptions (for now):  ASSUNZIONE (IRREALISTICA)
 - Each job  runs for the **same amount of time**.
 - All jobs **arrive** at the same time.
 - Once started, each job runs to  completion
 - All jobs only use the **CPU** (i.e., they don't perform I/O).
 - The **run-time** of each job is known.

Scheduling Metrics

- Performance metric: **Turnaround time**

- The time at which **the job completes** minus the time at which **the job arrived** in the system: $T_{turnaround} = T_{completion} - T_{arrival}$

TEMPO IN CUI IL
PROCESSO È ARRIVATO
NEL SISTEMA

- Given the assumption that all jobs arrive at the same time, we can consider, for now:

$$T_{arrival} = 0 \rightarrow T_{turnaround} = T_{completion}$$

ASSUNZIONE
CHE ABBIAMO FATTO
ALL'INIZIO

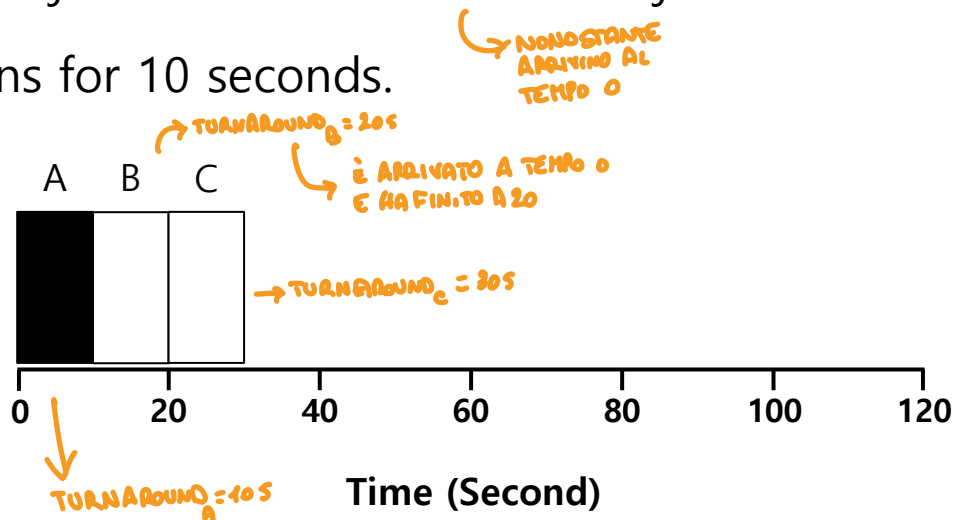
QUANDO EGUALMENTE LE
RISORSE VENGONO DIVISE TRA I PROCESSI

- Another metric is **fairness** (more about it later)

- Performance and fairness are often conflicting in scheduling.

First In, First Out (FIFO)

- First Come, First Served (FCFS)
 - Very simple and easy to implement
- Example:
 - Job A arrived just before B which arrived just before C.
 - Each job runs for 10 seconds.

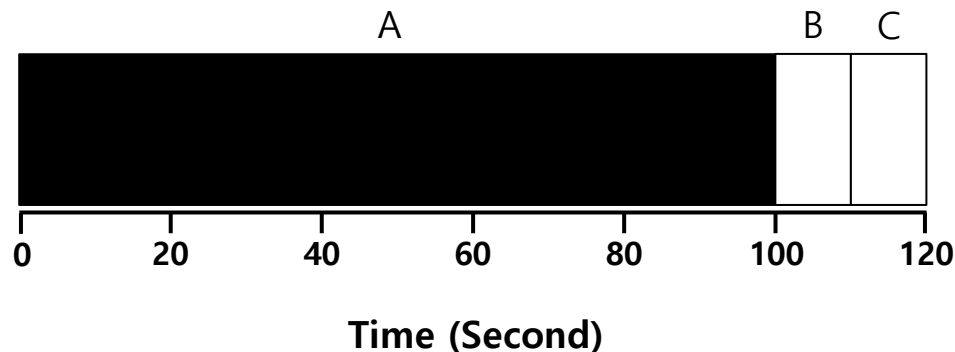


$$\text{Average turnaround time} = \frac{10 + 20 + 30}{3} = 20 \text{ sec}$$

Why is FIFO not that great? – Convoy effect

→ EFFETTO CONVOGLIO
IL VALORE A È DECISAMENTE PIÙ ALTO DI B E C

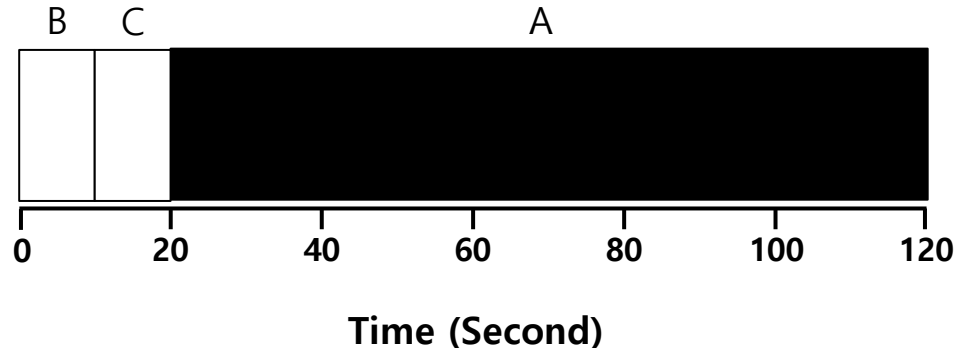
- **Let's relax assumption 1:** Each job **no longer** runs for the same amount of time.
- Convoy effect: a number of relatively short potential consumers gets queued behind a heavyweight resource consumer
- Example:
 - A arrived just before B which arrived just before C.
 - A runs for 100 seconds, B and C run for 10 each.



$$\text{Average turnaround time} = \frac{100 + 110 + 120}{3} = \mathbf{110 \text{ sec}}$$

Shortest Job First (SJF)

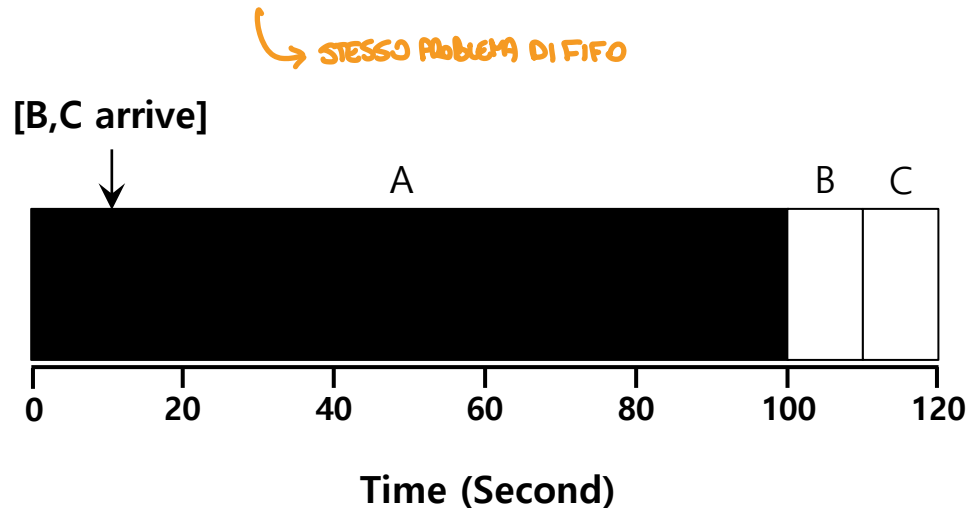
- Run the shortest job first, then the next shortest, and so on (remember, we assumed we know the run-time of each job!)
 - Example:
 - A, B, C arrive at the same time.
 - A runs for 100 seconds, B and C run for 10 each.



$$\text{Average turnaround time} = \frac{10 + 20 + 120}{3} = 50 \text{ sec}$$

SJF with Late Arrivals from B and C

- **Let's relax assumption 2:** Jobs can arrive at any time.
- Example:
 - A arrives at $t=0$ and needs to run for 100 seconds.
 - B and C arrive at $t=10$ and each need to run for 10 seconds

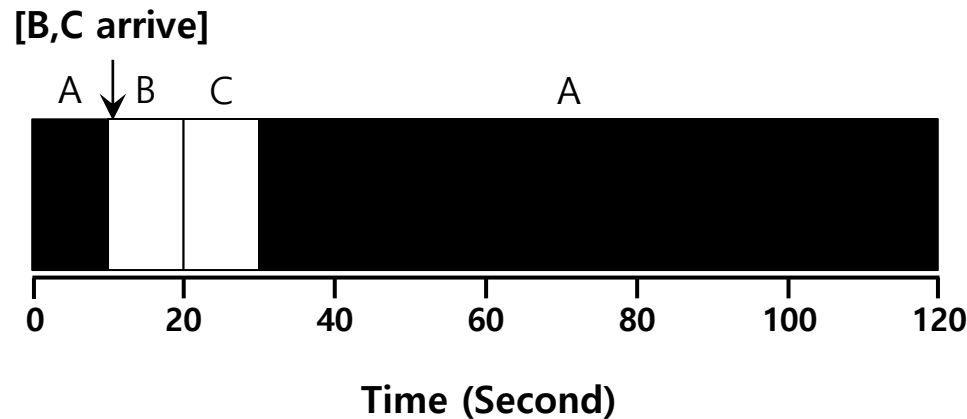


$$\text{Average turnaround time} = \frac{100 + (110 - 10) + (120 - 10)}{3} = 103.33 \text{ sec}$$

Shortest Time-to-Completion First (STCF)

- STCF adds **preemption** to SJF
 - Also known as Preemptive Shortest Job First (PSJF)
- When a new job enters the system, it determines which job has the least time left and schedules it.
- Example:
 - A arrives at $t=0$ and needs to run for 100 seconds.
 - B and C arrive at $t=10$ and each need to run for 10 seconds

SI BASA
NON SUL TEMPO DI ESECUZIONE DEL
PROCESSO MA SUL TEMPO DI ESECUZIONE
RIPASTO DEL PROCESSO



$$\text{Average turnaround time} = \frac{(120 - 0) + (20 - 10) + (30 - 10)}{3} = 50 \text{ sec}$$

New scheduling metric: Response time

- Shortest time to completion first is good for *batch* systems. However, we work today with *interactive* systems, so we need to introduce a new metric:

response time

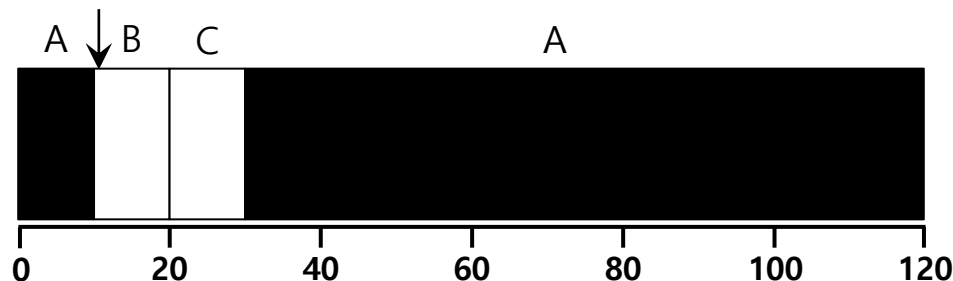
$$T_{response} = T_{firstrun} - T_{arrival}$$

← TEMPO CHE QUEL JOB È STATO
IN CODA PRIMA DI ESEGUIRE LA PRIMA ISTRUZIONE

- The time from **when the job arrives** to the **first time it is scheduled**.

**How can we build a scheduler that is
sensitive to response time?**

[B,C arrive]



What is the response time (rt) of A, B, C?

rt(A)=0

rt(B)=0

rt(C)=10

What is the average response time?

$$\frac{0 + 0 + 10}{3} = \frac{10}{3} s$$

Round Robin (RR) Scheduling

- Time slicing Scheduling → BASATA SU FETTE TEMPORALI
 - Run a job for a **time slice** and then switch to the next job in the **run queue** until the jobs are finished.
 - Time slice is sometimes called a scheduling quantum.
 - The length of a time slice must be *a multiple of* the timer-interrupt period (e.g., if the interrupt is triggered every 10ms, the time slice could be 10ms, 20ms, ...)

→ ESEGUIRE I PROCESSI
PER TOT TEMPO, PASSARE A UN ALTRO ECC...

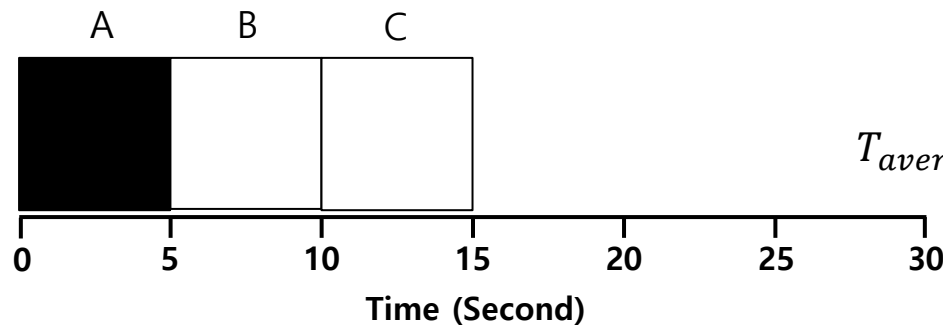
**RR is fair, but performs poorly on metrics
such as turnaround time**

→ ASSEGNA LA STESSA QUANTITA' DI RISORSA
A TUTTI

→ HA OTTIME TURNAROUND TIME PESSIMO

RR Scheduling Example

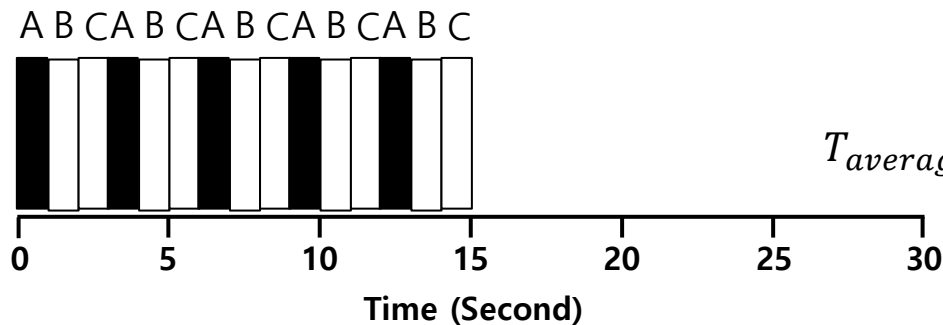
- A, B and C arrive at the same time.
- They each wish to run for 5 seconds.



SJF (Bad for Response Time)

$$T_{average\ response} = \frac{0 + 5 + 10}{3} = 5sec$$

$$T_{average\ turnaround} = \frac{5 + 10 + 15}{3} = 10sec$$



RR with a time-slice of 1sec (Good for Response Time)

$$T_{average\ response} = \frac{0 + 1 + 2}{3} = 1sec$$

$$T_{average\ turnaround} = \frac{13 + 14 + 15}{3} = 14sec$$

The length of the time slice is critical.

- The shorter the time slice
 - The better the response time, but...
 - ... the cost of context switch will dominate overall performance.
- The longer the time slice
 - the more the cost of switching is amortized, but...
 - ... at the cost of a worse response time

**Deciding on the length of the time slice presents
a **trade-off** to a system designer**

Observation on Turnaround time

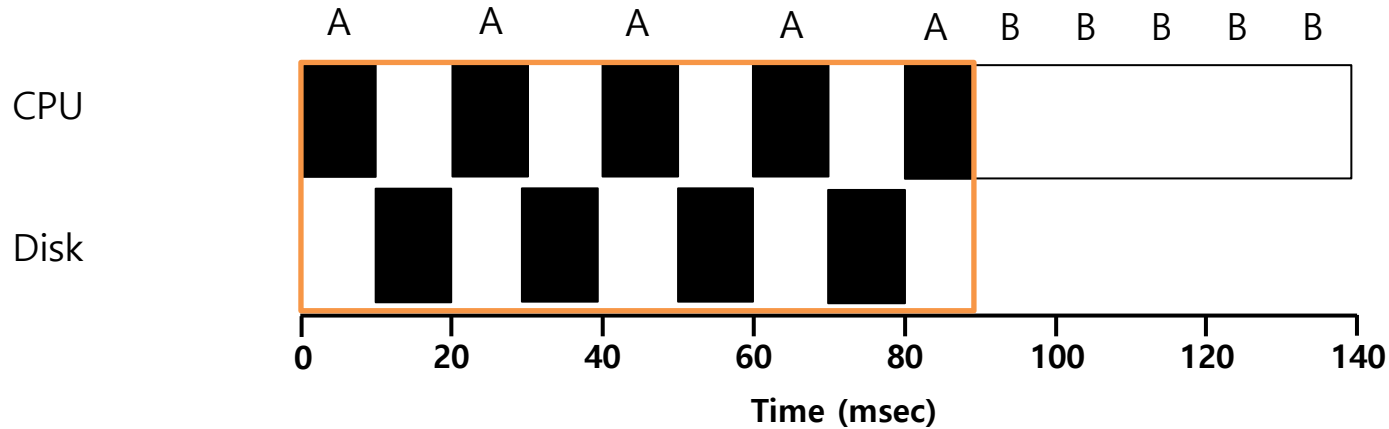
- RR is one of the worst policies for turnaround time
 - To have a good turnaround time, processes must finish as soon as possible
 - RR stretches the execution of the processes to minimize the response time, so it obtains very poor results in terms of turnaround time
- In general, every ***fair*** policy (i.e., a policy that evenly divides the CPU among active processes) will perform poorly on turnaround time.

↓ QUALCHE FA IL RR

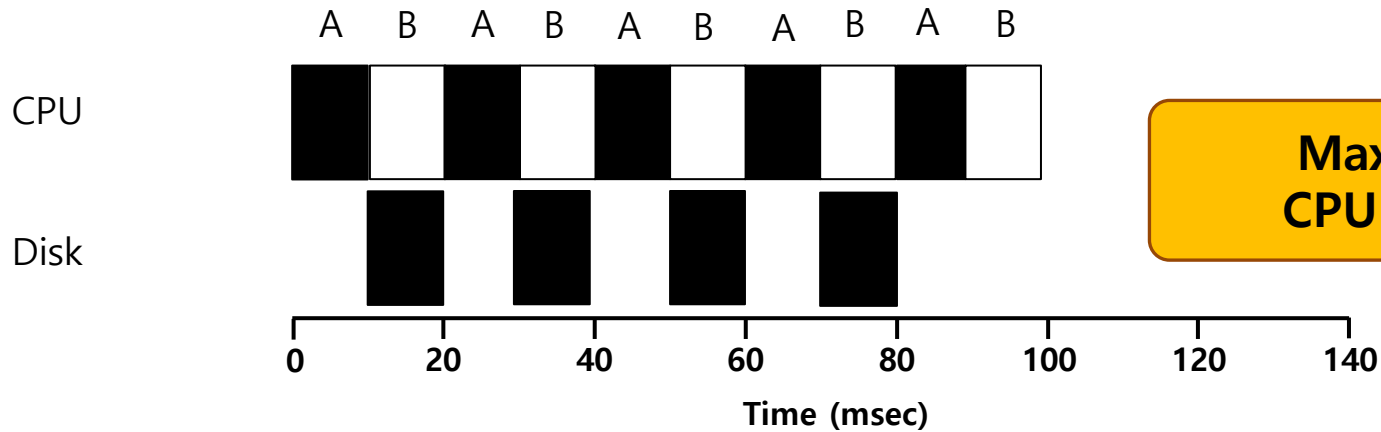
Incorporating I/O

- Let's relax assumption 4: processes can perform I/O
- When a job initiates an I/O request:
 - The job is blocked waiting for I/O completion.
 - The scheduler should schedule another job on the CPU.
- When the I/O completes
 - An interrupt is raised.
 - The OS moves the process from blocked back to the ready state.
- Example:
 - A and B need 50ms of CPU time each.
 - A runs for 10ms and then issues an I/O request
 - I/Os each take 10ms
 - B simply uses the CPU for 50ms and performs no I/O
 - The scheduler runs A first, then B

Incorporating I/O (Cont.)



Poor Use of Resources



Overlap Allows Better Use of Resources

**Maximize the
CPU utilization**

Scheduling

Introduzione

Abbiamo visto come viene implementato il context switch ma dobbiamo ancora discutere delle policy che si trovano al di sopra di questo meccanismo. Per farlo facciamo delle assunzioni che si discostano dalla realtà: (Queste assunzioni verranno modificate per capire meglio le politiche)

Ogni job (processo) ci mette lo stesso tempo a essere eseguito,

Ogni job arriva nello stesso momento

Una volta iniziato il job dev'essere completato

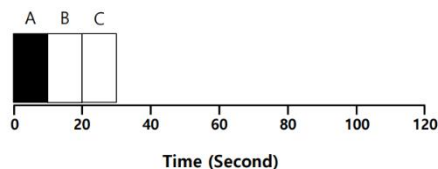
Tutti i job utilizzano solo la cpu e non I/O

Il tempo di esecuzione di ogni job è noto.

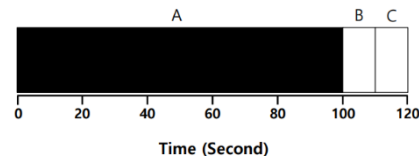
Una metrica con cui giudicheremo le varie politiche di scheduling è il Turnaround Time che sarebbe quanto tempo il processo è rimasto nel sistema ovvero $T_{\text{turnaround}} = T_{\text{Completion}} - T_{\text{arrival}}$ con le assunzioni che abbiamo fatto adesso che ogni processo arriva a tempo 0 il Turnaround è uguale al tempo di completamento del processo. Un'altra metrica è "fairness" quanto equamente le risorse vengono divise tra i processi.

FIFO/First-In-First-Out o FCFS/First-Come-First-Served

Implementare una strategia fifo può essere conveniente solo con le nostre assunzioni, infatti se i processi non arrivassero tutti a tempo zero e avessero un tempo di esecuzione differente tra loro il turnaround medio si alzerebbe troppo e si crea quello che viene detto effetto convoglio, se arriva prima il processo più lungo, quelli dopo ne devono aspettare il completamento



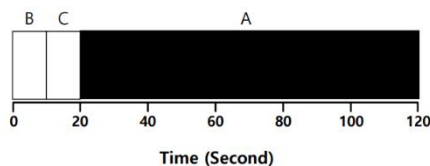
$$\text{Average turnaround time} = \frac{10 + 20 + 30}{3} = 20 \text{ sec}$$



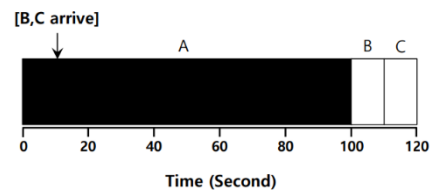
$$\text{Average turnaround time} = \frac{100 + 110 + 120}{3} = 110 \text{ sec}$$

SJF/Shortest Job First

Come fa intuire il titolo viene eseguito prima il processo più corto poi il secondo più corto e così via. La criticità di questo è simile alla FIFO, in questo caso se è presente un solo processo che è anche il più lungo e ne viene iniziata l'esecuzione e poco dopo arrivano gli altri processi, nuovamente si ritrovano bloccati dal processo più lungo.



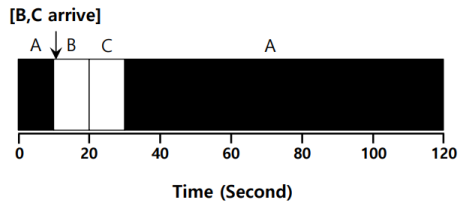
$$\text{Average turnaround time} = \frac{10 + 20 + 120}{3} = 50 \text{ sec}$$



$$\text{Average turnaround time} = \frac{100 + (110 - 10) + (120 - 10)}{3} = 103.33 \text{ sec}$$

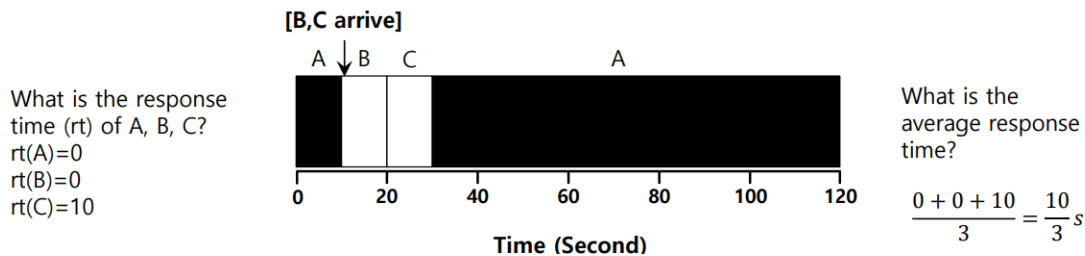
STCF/Shortest-Time-to-Completion-First

Questa politica è simile al SJF solo che non si basa sul tempo di esecuzione del processo ma sul tempo di esecuzione rimasto del processo.



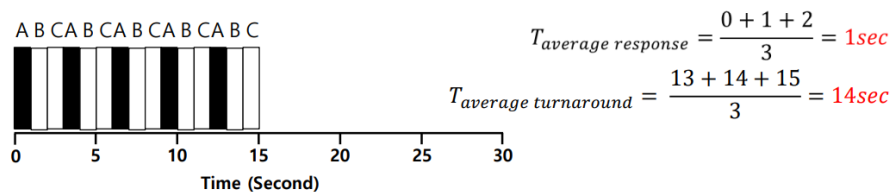
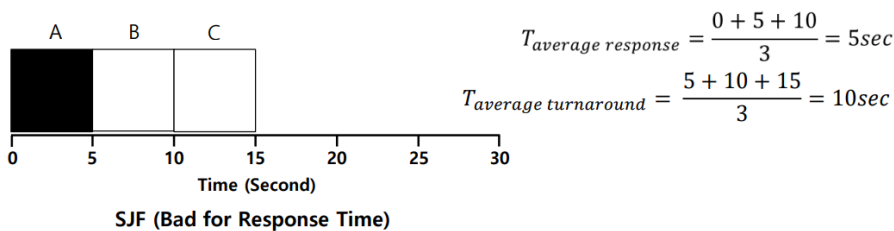
$$\text{Average turnaround time} = \frac{(120 - 0) + (20 - 10) + (30 - 10)}{3} = 50 \text{ sec}$$

Aggiungiamo un'altra metrica di giudizio che è il tempo di risposta, che consiste nel tempo che quel job è stato in coda prima di eseguire la prima istruzione ed è uguale a $T_{\text{response}} = T_{\text{firstrun}} - T_{\text{arrival}}$



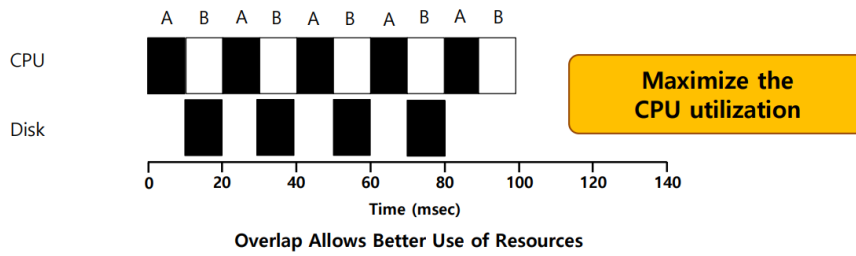
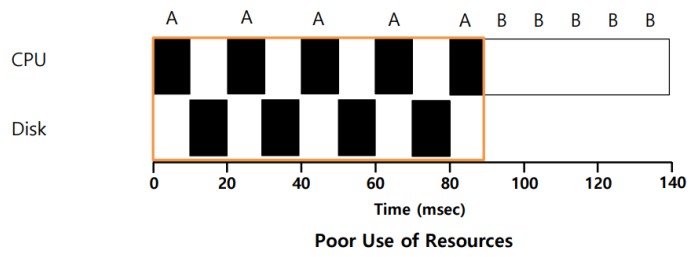
RR/Round Robin

Questa politica è basata su fette temporali, ovvero un job va in esecuzione per un certo quanto di tempo, successivamente viene mandato in esecuzione un altro job fino a che non c'è più nulla in coda. Questa politica è fair ma così facendo ottiene un turnaround pessimo.



Questa politica sia con un quanto di tempo lungo che corto ha sempre delle criticità. Più il quanto di tempo è piccolo e più il costo del context switch dominerà le performance. Più il quanto di tempo è grande e peggiore sarà il tempo di risposta. Generalmente ogni politica fair avrà un cattivo turnaround time. Se volessimo incorporare anche l'I/O dovremmo modificare qualche assunzione, del tipo i processi possono usare I/O, Quando un job inizia una richiesta di I/O: Il job rimane bloccato e aspetta

che l'I/O finisca, il gestore dovrebbe gestire un altro job sulla cpu. Quando l'I/O finisce viene sollevato un Interrupt e il sistema operativo modifica lo stato del processo da bloccato a pronto



8: Scheduling: The Multi-Level Feedback Queue

Operating System: Three Easy Pieces

Multi-Level Feedback Queue (MLFQ)

- We will study now a scheduler that tries to optimize the turnaround time, while making the system feel responsive.
- In general, the Multi-Level Feedback Queue (MLFQ) scheduler tries to:
 - **Optimize turnaround time** → Run shorter jobs first
 - **Minimize response time** without *a priori knowledge of job length*.
 - **Learn** the characteristics of the running jobs to make better scheduling decisions

Quando siamo andati a ottimizzare il turnaround time aumentavamo il tempo di risposta e quando andavamo a ottimizzare il tempo di risposta ottenevamo un turnaround time più elevato. Esistono dei sistemi detti batch che vogliono ottimizzare il tempo di esecuzione di un processo

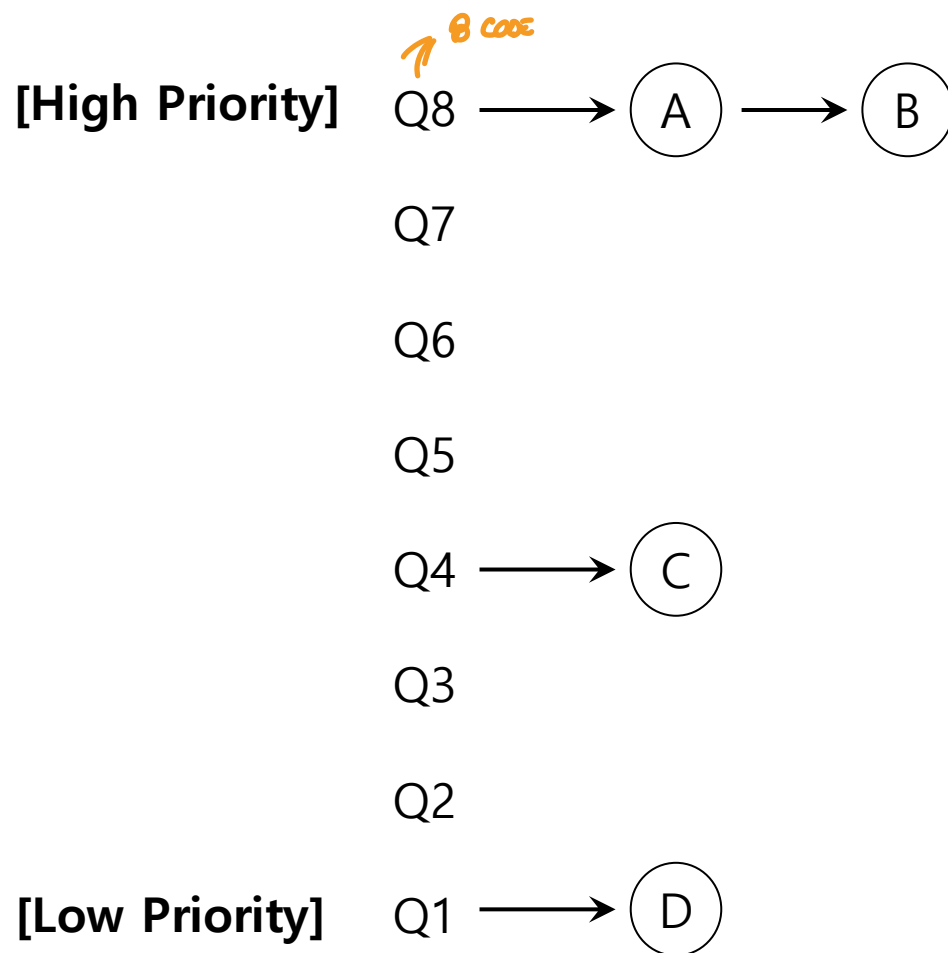
MLFQ: Basic Rules

- MLFQ has a number of distinct **queues**.
 - Each queue is assigned a different priority level.
 - A job on a **higher** queue is **always** chosen to **run**.
 - Use **round-robin** scheduling among jobs in the **same queue**

Rule 1: If $\text{Priority}(A) > \text{Priority}(B)$, A runs (B doesn't).

Rule 2: If $\text{Priority}(A) = \text{Priority}(B)$, A & B run in RR.

MLFQ Example



With the rules we have just defined, A and B will be executed in a round robin fashion. Jobs C and D will never run, as long as A and B don't terminate

MLFQ: Basic Rules (Cont.)

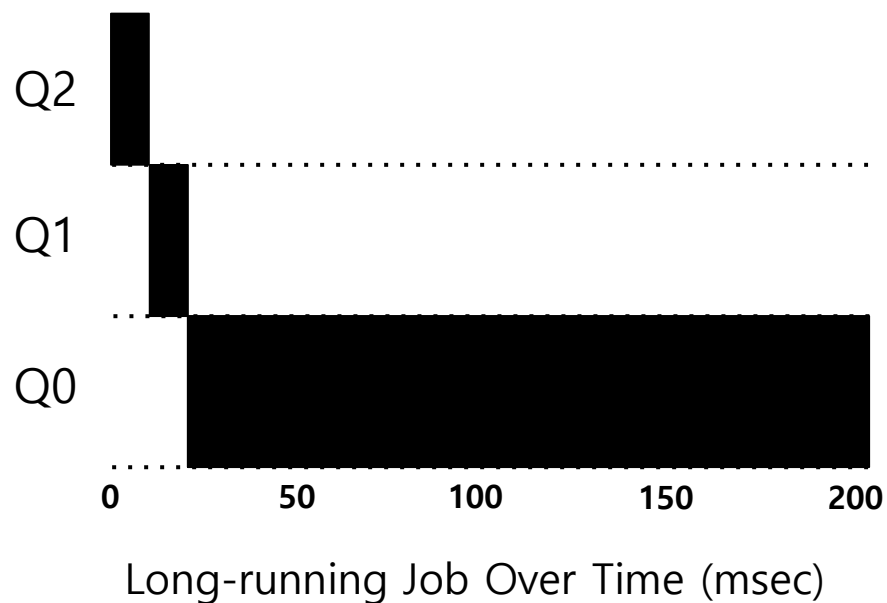
- MLFQ varies the priority of a job based on its observed behavior.
- Example:
 - A job repeatedly relinquishes the CPU while waiting IOs → Keep its priority high  Questo perché probabilmente il job è un job interattivo e quello che voglio fare è diminuire il tempo di risposta
 - This is how usually interactive processes behave. And we want interactive processes to be very responsive
 - A job uses the CPU intensively for long periods of time → Reduce its priority.  Questo perché mi sto accorgendo che il job si sta comportando come un job batch, es. avvio un calcolo la sera e la mattina dopo vedo il risultato
 - The end-user is less likely to notice a delay for a long-running process

MLFQ: How to Change Priority

- MLFQ priority adjustment algorithm:
 - **Rule 3:** When a job enters the system, it is placed at the highest priority
 - **Rule 4a:** If a job uses up an entire time slice while running, its priority is reduced (i.e., it moves down on queue). 
 - **Rule 4b:** If a job gives up the CPU before the time slice is up, it stays at the same priority level 

Example 1: A Single Long-Running Job

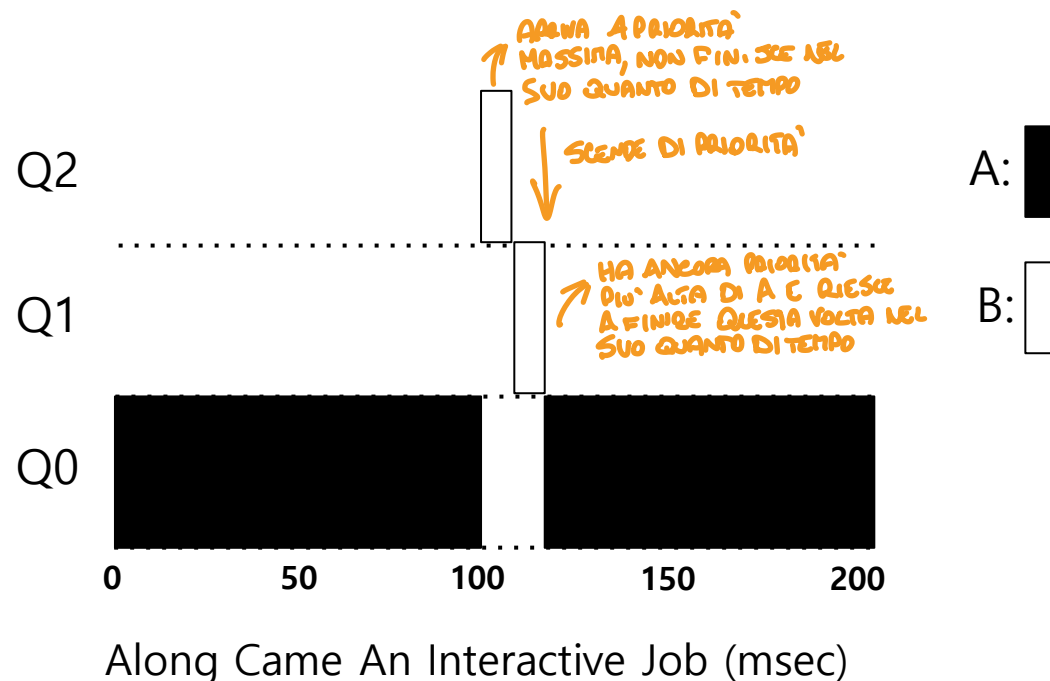
- A three-queues scheduler with time slice 10ms



Example 2: Along Came a Short Job

- Assumption:

- Job A:** A long-running CPU-intensive job
- Job B:** A short-running interactive job (20ms runtime)
- A has been running for some time, and then B arrives at time $T=100$.

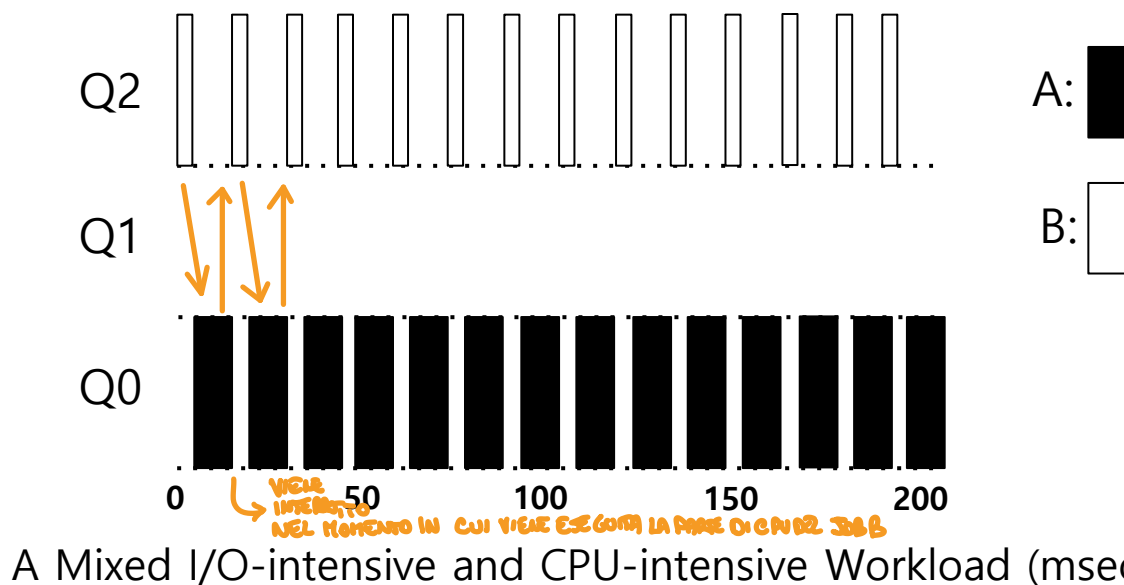


Observation on MLFQ and SJF

- SJF needs to know the length of each job to schedule the shortest first
- With the set of rules that we have defined, MLFQ approximates SJF.
 - MLFQ does not assume to know the length of a process, so:
 - when a new process is run, it assumes it has a short length and puts it in the highest queue
 - if the assumption is right, the job will complete in the highest priority queue, therefore mimicking SJF
 - if the assumption is wrong, the process quickly goes to the lowest priority queue, leaving room for shorter (and higher priority) jobs.

Example 3: What About I/O?

- Assumption:
 - Job A:** A long-running CPU-intensive job
 - Job B:** An interactive job that needs the CPU only for 1ms before performing an I/O



The MLFQ approach keeps an interactive job at the highest priority

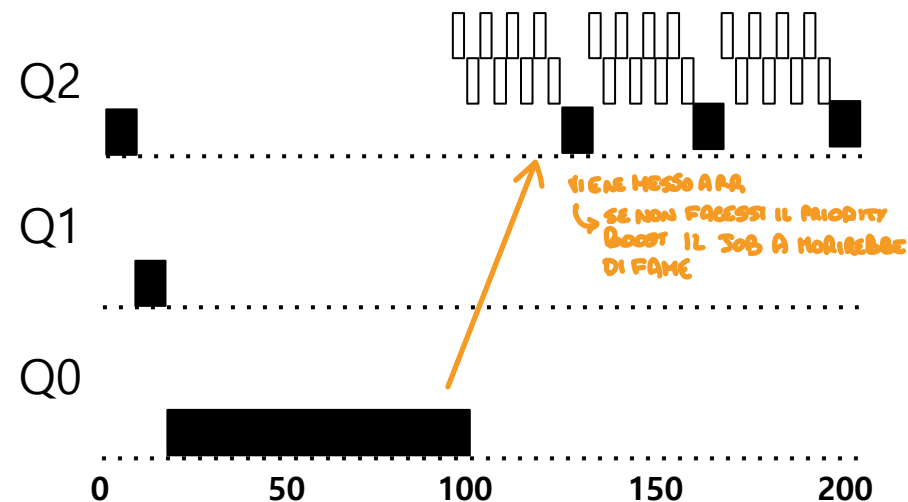
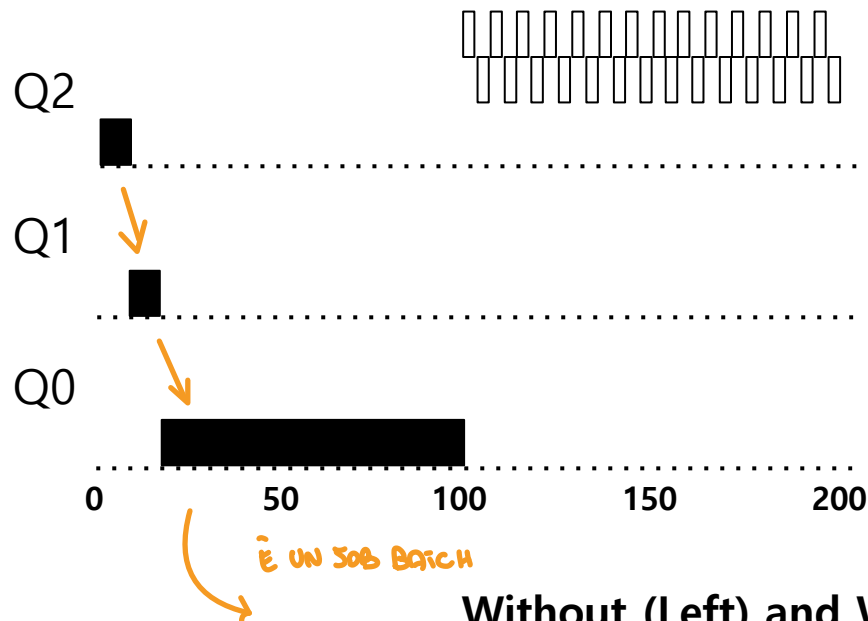
Problems with the Basic MLFQ

- Starvation → I SOB A PRIORITA' PIU' BASSA POTREBBERO ANDARE IN "STARVATION": MOLTO DIFANE
 - If there are "too many" interactive jobs in the system, long-running jobs will never receive any CPU time.
- Game the scheduler
 - After running 99% of a time slice, issue an I/O operation.
 - The job gains a higher percentage of CPU time.

Quello che potrebbe fare un job malevolo, usare il 99% del quanto di tempo e nell'ultimo momento fare una richiesta I/O per non perdere priorità
- A program may change its behavior over time.
 - CPU bound process $\leftarrow \rightarrow$ I/O bound process

The Priority Boost

- **Rule 5:** After some time period S , move all the jobs in the system to the topmost queue. —→ "GLI DAIAMO UN ALTRA CHANCE, RISOLVO IL PROBLEMA DELLA STARVATION"
 - Example: A long-running job (A) with two short-running interactive jobs (B, C)
 - How to properly set S ? Unfortunately, there is no "one size fits all" answer.

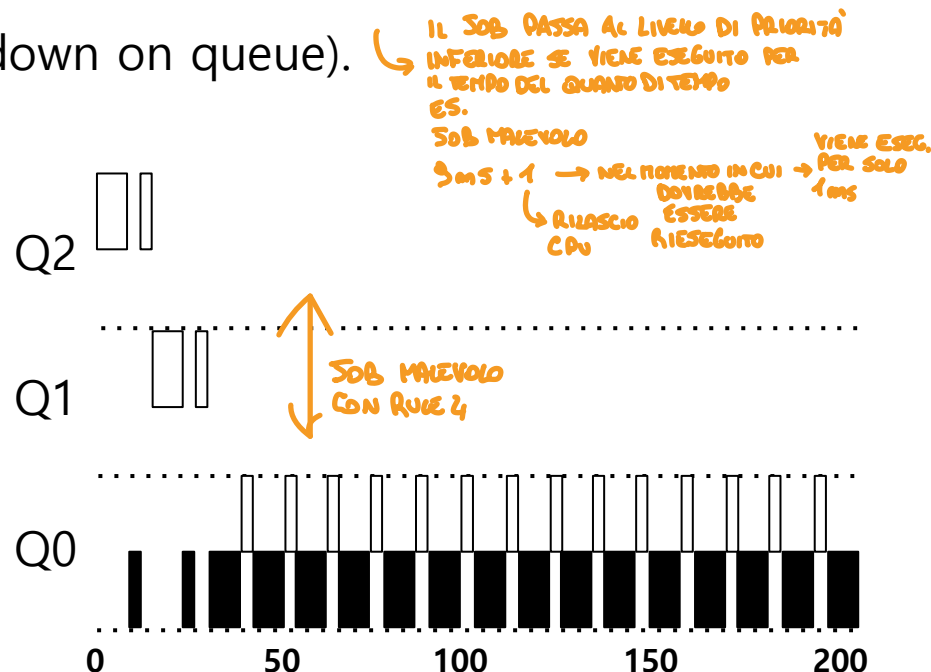
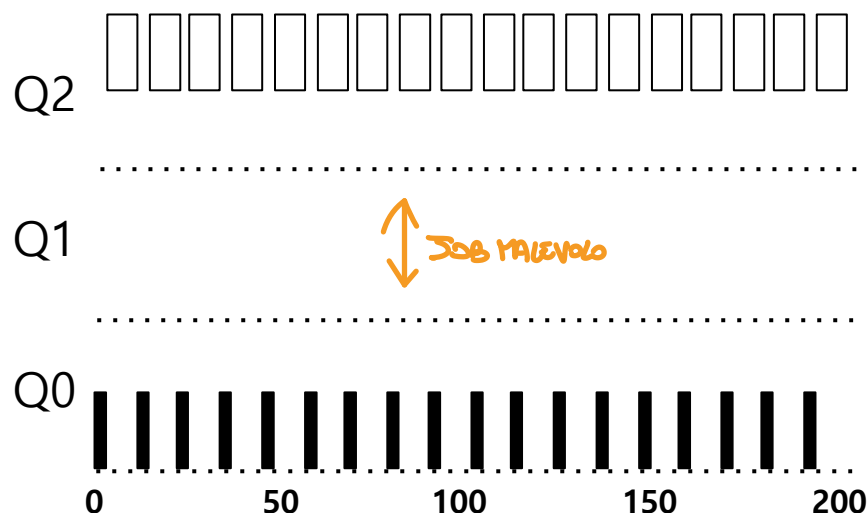


Without (Left) and With (Right) Priority Boost

A:  B:  C: 

Better Accounting

- How to prevent gaming of our scheduler?
- Solution:
 - **Rule 4** (Rewrite Rules 4a and 4b): Once a job **uses up its time allotment** at a given level (regardless of how many times it has given up the CPU), **its priority is reduced** (i.e., it moves down on queue).

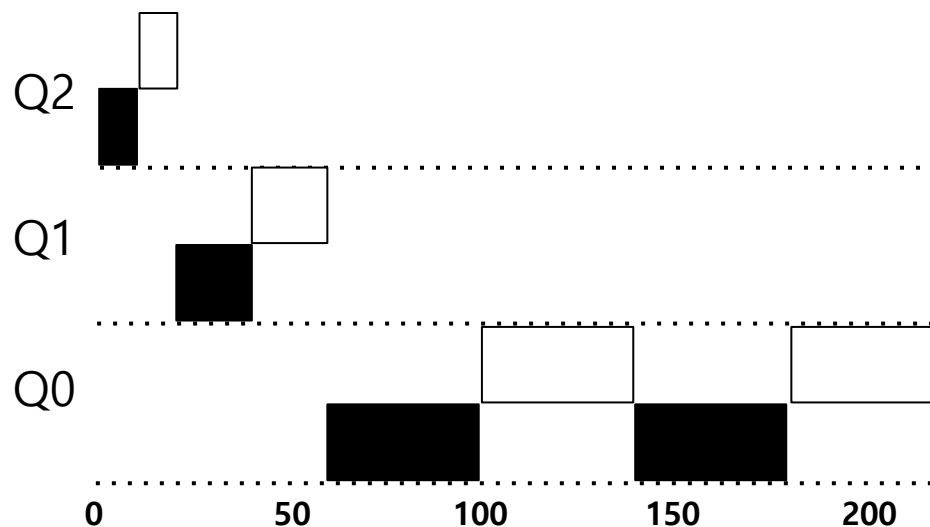


Without (Left) and With (Right) Gaming Tolerance

Tuning MLFQ And Other Issues

Lower Priority, Longer Quanta

- Most MLFQ variants implement the following tricks:
 - High-priority queues → Short time slices
 - E.g., 10 or fewer milliseconds
 - Low-priority queue → Longer time slices
 - E.g., 100 milliseconds



Example: 10ms for the highest queue, 20ms for the middle, 40ms for the lowest

The Solaris MLFQ implementation

- 60 Queues
- Slowly increasing time-slice length
 - The highest priority: 20ms
 - The lowest priority: A few hundred milliseconds
- Priorities boosted around every 1 second or so.

MLFQ: Summary

- The refined set of MLFQ rules:
 - **Rule 1:** If $\text{Priority}(A) > \text{Priority}(B)$, A runs (B doesn't).
 - **Rule 2:** If $\text{Priority}(A) = \text{Priority}(B)$, A & B run in RR.
 - **Rule 3:** When a job enters the system, it is placed at the highest priority.
 - **Rule 4:** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves down on queue).

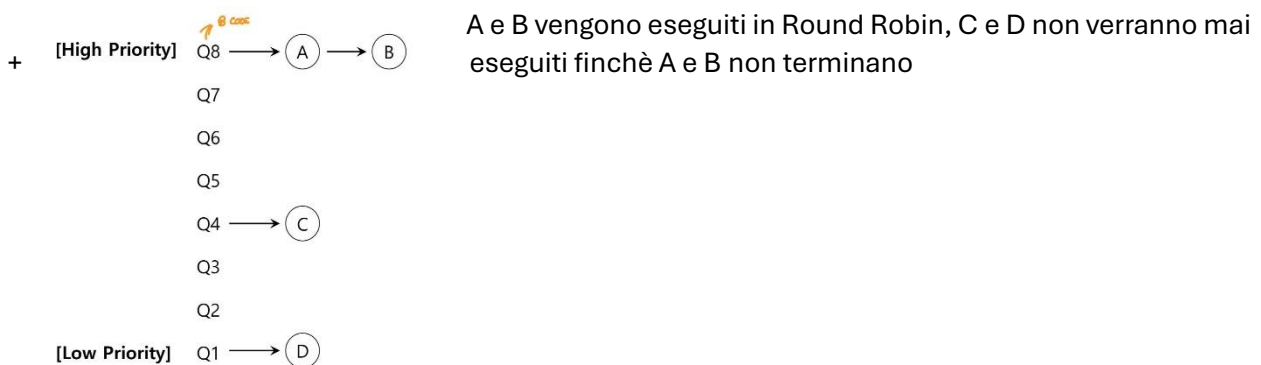
 - **Rule 5:** After some time period S, move all the jobs in the system to the topmost queue.

Multi-Level Feedback Queue (MLFQ)

Con le altre politiche di scheduling abbiamo notato che per migliorare il turnaround time ottenevamo un tempo di risposta pessimo mentre per migliorare il tempo di risposta ottenevamo un turnaround time più elevato. Quello che tenta di fare il MLFQ è di ottimizzare il turnaround time e minimizzare il tempo di risposta, senza sapere a priori il tempo di esecuzione del processo. Questa politica di scheduling impara a run-time le caratteristiche dei job da eseguire, in modo tale da prendere delle decisioni migliori. MLFQ ha un numero di code distinte, ogni coda ha un livello di priorità.

RULE 1: se la $PRIORITA(a) > PRIORITA(b)$ viene eseguito a

RULE 2: se la $PRIORITA(a) = PRIORITA(b)$ a e b vengono eseguiti in RR



With the rules we have just defined, A and B will be executed in a round robin fashion.
Jobs C and D will never run, as long as A and B don't terminate

MLFQ cambia la priorità di un job osservandone il comportamento:

Se un job rilascia spesso la CPU per fare un'operazione di I/O gli lascia la priorità alta dato che probabilmente è un job interattivo e vogliamo che i job interattivi siano reattivi. Se un job utilizza la CPU per un lungo periodo di tempo gli riduce la priorità, questo perché si sta comportando come un job batch (es. avvio un calcolo la sera e la mattina ne vedo il risultato). Aggiungiamo delle regole al cambio di priorità:

RULE 3: Quando un job entra nel sistema gli viene attribuita la massima priorità

RULE 4a: Quando un job usa un intero quanto di tempo la sua priorità viene ridotta, questo perché potrebbe essere un job batch che richiede la cpu e non esegue operazioni di I/O

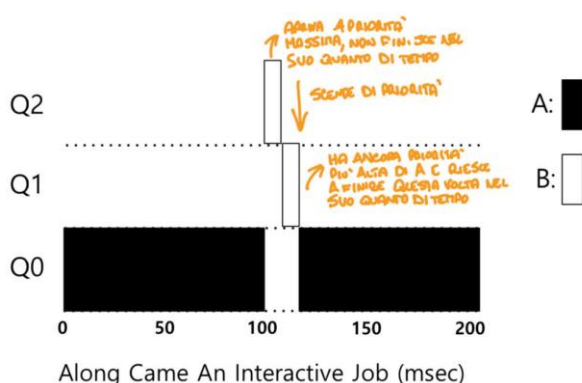
RULE 4b: Se un job rilascia la CPU prima che finisca il quanto di tempo la sua priorità rimane allo stesso livello, questo perché potrebbe essere un job interattivo

Esempio:

Il job A ha già avuto modo di dimostrare che non è un job interattivo

Il job B è un job interattivo che ha runtime 20ms

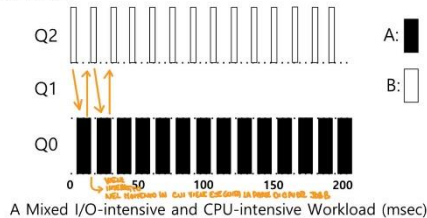
A è stato eseguito per una certa quantità di tempo, successivamente arriva B al tempo $T=100ms$



B arriva a massima priorità e non riesce a finire la sua esecuzione, gli viene scalato un livello di priorità, ha ancora priorità più alta di A e riesce a finire questa volta nel suo quanto di tempo

Comparando MLFQ con SJF (Short Job First) SJF doveva sapere a priori la lunghezza di ogni processo, con il set di regole che abbiamo messo MLFQ approssima SJF. La differenza è che MLFQ non gli serve sapere a priori la lunghezza del job, quando arriva un nuovo processo viene messo alla priorità massima, infatti se fosse un processo corto resterebbe alla massima priorità e lavorerebbe come SJF, se invece il processo fosse lungo, quest'ultimo, calerebbe di priorità.

- o **Job A:** A long-running CPU-intensive job
- o **Job B:** An interactive job that needs the CPU only for 1ms before performing an I/O



Quest'esempio ci fa notare delle debolezze:

-**Starvation (morire di fame):** Se ci fossero troppi job interattivi, i job a priorità più bassa potrebbero andare in starvation.

-**Ingannare lo scheduler:** Un job malevolo, potrebbe, usare il 99% del quanto di tempo e successivamente fare una richiesta I/O in modo tale da non perdere di priorità

RULE 5 (Priority Boost): Dopo un certo periodo di tempo, sposto tutti i job a priorità massima ("gli do una seconda possibilità"), in poche parole dopo un po' un job batch verrebbe messo a RR con i job interattivi.

RULE 4: (Riscrivo 4a e 4b) Il job passa a un livello di priorità inferiore se viene eseguito per il tempo del quanto di tempo. Es: Ho un job malevolo e ho quanti di tempo da 10ms, utilizza 9ms e poi fa una richiesta di I/O quando verrà poi rieseguito, avrà a disposizione solo 1ms, dopodichè scenderebbe di priorità

MLFQ solitamente attribuisce ai livelli di priorità più alti quanti di tempo minori e viceversa.

L'implementazione Solaris di MLFQ ha 60 code, e incrementa lentamente i quanti di tempo: Livello di priorità massimo: 20ms, Livello di priorità minimo qualche centinaia di ms. Il priority boost viene fatto ogni secondo.

9: Scheduling: The Linux Completely Fair Scheduling (CFS)

The Linux Completely Fair Scheduling (CFS)

IO TROVANO
SOME MACCHINE LINK DI
AREZZO

- Recent studies have shown that nearly 5% of the CPU time in Google datacenters is spent in scheduling
 - ◆ Designing an efficient scheduler is key to improve the overall performance
- Completely Fair Scheduling (CFS)
 - ◆ The current CPU scheduler in Linux
 - ◆ Main characteristics:
 - Non-fixed timeslice.
 - Supports priority → MOLTO DIVERSO DA MLFA
 - Enables control over priority by using nice value. "GENTILEZZA" INDICA QUANTO IL NOSTRO PROCESSO DEBEA ESSERE GENTILE CON IL NOSTRO PROCESSO
 - Makes use of efficient data structures for efficient search, insertion and deletion of a process.

Basic parameters

▣ Virtual runtime (*vruntime*)

- ◆ Denote how long the process has been executing.
- ◆ Per-process variable
- ◆ Increase in **proportion with physical (real) time** when it runs.
- ◆ CFS will pick the process with the **lowest vruntime** to run next.

→ RAPPRESENTA
LA QUANTITÀ
DI CPU UTILIZZATA DAL PROCESSO
↓
QUANTI MS

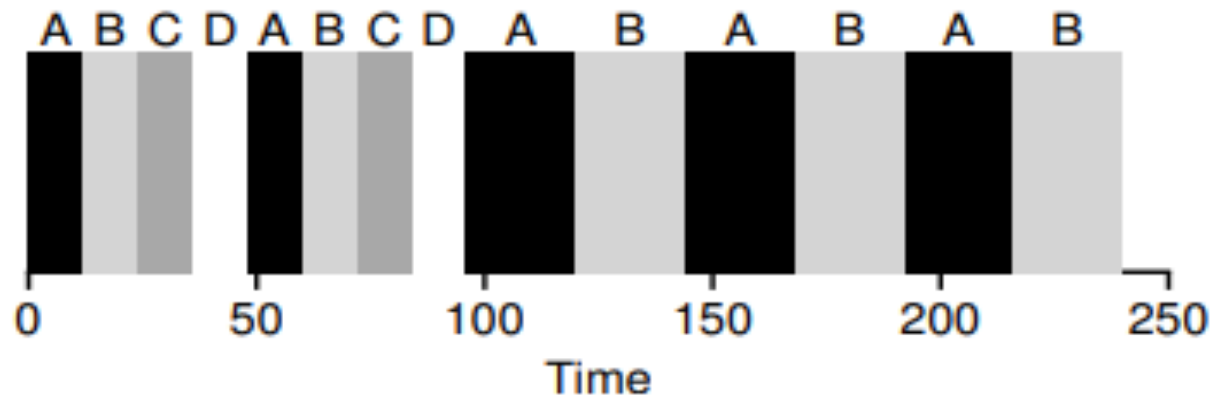
▣ sched_latency

LA DIMENSIONE

- ◆ Parameter that defines[✓] the time window that has to be divided in slices.
 - Observation: by defining this time window, we implicitly guarantee bounds on the response time
- ◆ A typical value is 48 (milliseconds)
- ◆ $\text{timeslice of a process} = \text{sched_latency} / (\text{the number of process})$

Example

- ◆ Simple Example → PER CAPIRE IL QUANTO DI TEMPO VARIABILE
 - 4 processes (A,B,C,D) and then 2 processes(C,D) complete.



- ◆ min_granularity
 - The minimum timeslice (6ms) → CON TROPPI PROCESSI ANDREMO DEI PROBLEMI SE DIVIDESSIMO SEMPRE I 48 ms
QUINDI AUMENTEREMMO I 48ms
 - Ensure that not too much time is spent in scheduling overhead, when there are too many processes running.

Weight

- ◆ Nice value → VIENE TRADOTTO CON DEI PESI NUMERICI
 - CFS enables controls over process priority.
 - Nice parameter is integer value and can be set from -20 to +19.
 - The nice value is mapped to a weight

```
static const int prio_to_weight[40] = {  
    /* -20 */ → 88761,      71755,      56483,      46273,      36291,  
    /* -15 */      29154,      23254,      18705,      14949,      11916,  
    /* -10 */      9548,      7620,      6100,      4904,      3906,  
    /*  -5 */      3121,      2501,      1991,      1586,      1277,  
    /*   0 */ → 1024,      820,      655,      526,      423,  
    /*   5 */      335,      272,      215,      172,      137,  
    /*  10 */      110,      87,      70,      56,      45,  
    /*  15 */      36,      29,      23,      18,      15,  
};
```

Weighting (Niceness)

- ◆ New timeslice formula

$$time_slice_k = \frac{weight_k}{\sum_{i=0}^{n-1} weight_i} \cdot sched_latency$$

↑ PESO CHE IL PROCESSO DEVE AVERE

↘ IN RELAZIONE AGLI ALTRI PROCESSI

- ◆ Simple Example

- Assign Process 'A' a nice value of -5 and process 'B' a nice value of 0.

Process	nice value	weight	Time slice
A	-5	3121	36 ms
B	0	1024	12 ms

*→ GUARDA
LA TABELLA
DI PRIMA*

*→ B VERAMENTE PREFERITO
DATO CHE OGGI LA
SUA ESECUZIONE AVRA' VANTAGGIO PIU'
BASSO E QUINDI B RAGGIUNGERA'
A
QUINDI NON ABBIAMO
RISOLTO NULLA*

vruntime with weight

Weighting (Niceness)

vruntime formula

- Calculate the actual run time. Scales it inversely by the weight of process.

$$vruntime_i = vruntime_i + \frac{weight_0}{weight_i} \cdot runtime_i$$

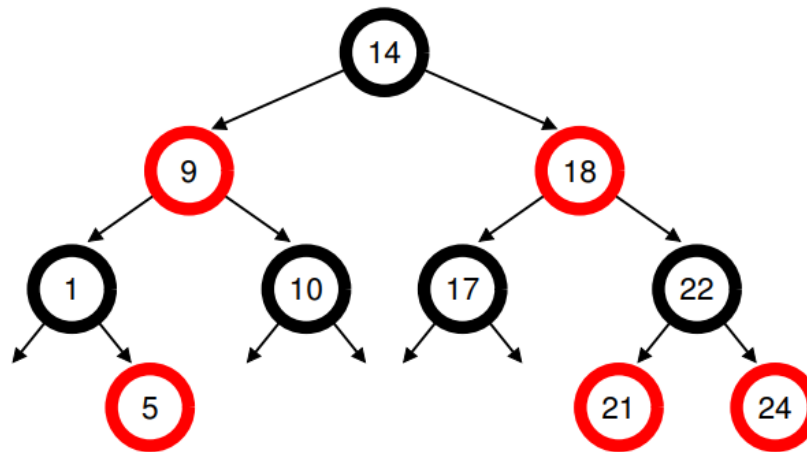
Simple Example

Process	nice value	weight	Accumulated value
A	-5	3121	$\frac{1}{3} * runtime$
B	0	1024	$1 * runtime$

Tree-based structure of ready queue

▣ Red-Black Tree

- ◆ Balanced binary tree (can address worst-case insertion)
- ◆ Efficiently find the process with minimum virtual runtime.
- ◆ Only running (or ~~runnable~~^{READY}) processes are kept therein.



I/O and sleeping process

COSA SUCCIDE SE UN PROCESSO VUOLE FARE UN'OPERAZIONE DI I/O

▣ Dealing with I/O and Sleeping processes

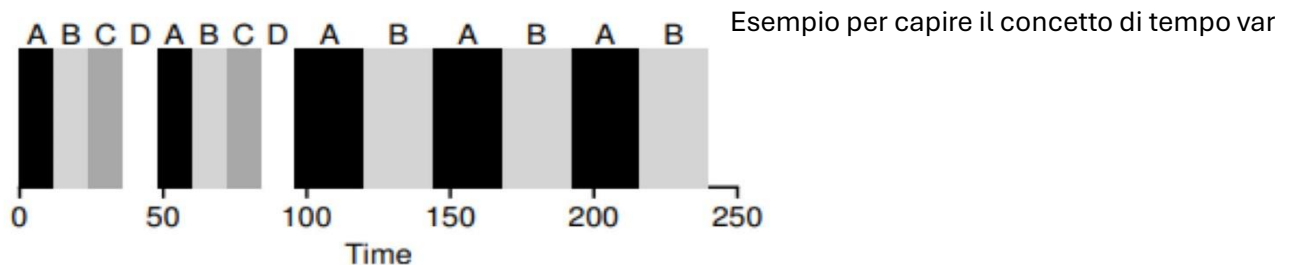
- ◆ Avoid the situation where some process monopolizes the CPU, if process have significantly small vruntime after sleeping.
- ◆ Set the vruntime of process to the minimum value found in tree when it wakes up.
↓ VALORE MINIMO DEL VRUNTINE DEGLI ALTRI PROCESSI
- ◆ Process that sleep for short periods of time frequently do not ever get their fair share of the CPU.

The Linux Completely Fair Scheduling (CFS)

È la politica di scheduling che troviamo sulle macchine di adesso. Le sue principali caratteristiche sono: quanti di tempo variabili, supporta la priorità, controlla la priorità secondo un parametro di "gentilezza" (quanto il nostro processo debba essere gentile con gli altri processi) , utilizza delle strutture dati che sono più efficienti per ricerca, inserzione e cancellazione di un processo. I parametri base sono:

-Virtual Runtime (Vruntime): Denota per quanto tempo un processo è stato eseguito, ovvero la quantità in ms di cpu utilizzata dal processo. Incrementa proporzionalmente durante l'esecuzione di un job. Il CFS prenderà il processo avente runtime più basso possibile.

-Sched_Latency: è il parametro che definisce la dimensione della finestra temporale da dividere in quanti di tempo (definendo questo parametro guadagniamo in termini di tempo di risposta), un valore tipico è di 48 ms. Il quanto di tempo di un processo = sched_latency / numero di processi.



Ora c'è bisogno di fare delle precisazioni: il quanto di tempo minimo è di 6ms, infatti se riducessimo troppo il tempo dei quanti di tempo il SO impiegherebbe più tempo a fare il context switching che a eseguire i processi stessi. Il parametro di gentilezza va da -20 a +1 e viene tradotto in pesi numerici.

```
static const int prio_to_weight[40] = {
/* -20 */ 88761, 71755, 56483, 46273, 36291,
/* -15 */ 29154, 23254, 18705, 14949, 11916,
/* -10 */ 9548, 7620, 6100, 4904, 3906,
/* -5 */ 3121, 2501, 1991, 1586, 1277,
/* 0 */ 1024, 820, 655, 526, 423,
/* 5 */ 335, 272, 215, 172, 137,
/* 10 */ 110, 87, 70, 56, 45,
/* 15 */ 36, 29, 23, 18, 15,
};
```

New timeslice formula

$$time_slice_k = \frac{weight_k}{\sum_{i=0}^{n-1} weight_i} \cdot sched_latency$$

Nota: weight_k è il peso del processo k, e sched_latency è la latenza di scheduling.

Simple Example

- Assign Process 'A' a nice value of -5 and process 'B' a nice value of 0.

Process	nice value	weight	Time slice
A	-5	3121	36 ms
B	0	1024	12 ms

Weighting (Niceness)

vruntime formula

- Calculate the actual run time. Scales it inversely by the weight of process.

$$vruntime_i = vruntime_i + \frac{weight_0}{weight_i} \cdot runtime_i$$

Nota: weight_0 è il peso del processo 0, e runtime_i è il tempo di esecuzione del processo i.

Simple Example

Process	nice value	weight	Accumulated value
A	-5	3121	4/3 * runtime
B	0	1024	1 * runtime

Questa è la formula per il quanto di tempo, ovvero peso del processo / somma dei pesi dei processi * sched_latency. In questo esempio il processo B verrà preferito dato che B avrà, dopo la prima esecuzione, Vruntime più basso e quindi B raggiungerà A. Per ovviare a questo viene aggiustato il calcolo del Vruntime che sale inversamente al peso del processo.

Una struttura basata su questo tipo di coda, sono gli alberi rosso-neri. L'ultima osservazione da fare è chiederci cosa succede in caso un processo voglia fare un'operazione di I/O. Innanzitutto bisogna evitare che gli altri processi aventi Vruntime basso monopolizzino la cpu, infatti quello che viene fatto è mettere al minimo il vruntime di tutti i processi.

5. Interlude: Process API

Operating System: Three Easy Pieces

What interfaces should the OS present for process creation and control?

The fork() System Call

p1.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]){
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {    // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) { // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
    } else {          // parent goes down this path (main)
        printf("hello, I am parent of %d (pid:%d)\n",
            rc, (int) getpid());
    }
    return 0;
}
```

- One moment before the end of the execution of fork(), there will be two copies of p1 running, and each of them will be about to return from its execution
 - The newly created process does not start from the first instruction of main(), but from after fork() itself
- The two processes will almost be an exact copy of each other
 - The newly-created process has its own copy of the **address space, registers, and PC**.
 - The value returned by fork(), and therefore the value of the variable rc, will be different

Calling fork() example (Cont.)

Result (Not deterministic)

```
prompt> ./p1
hello world (pid:29146)
hello, I am parent of 29147 (pid:29146)
hello, I am child (pid:29147)
prompt>
```

or

```
prompt> ./p1
hello world (pid:29146)
hello, I am child (pid:29147)
hello, I am parent of 29147 (pid:29146)
prompt>
```

The wait() System Call

- The wait() system call suspends execution of the calling process **until one of its children terminates**.
 - What if we have multiple children? How do we handle it?
- p2.c**

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[]){
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {    // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) { // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
    } else {          // parent goes down this path (main)
        int wc = wait(NULL);
        printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
            rc, wc, (int) getpid());
    }
    return 0;
}
```

The wait() System Call (Cont.)

Result (Deterministic)

```
prompt> ./p2
hello world (pid:29266)
hello, I am child (pid:29267)
hello, I am parent of 29267 (wc:29267) (pid:29266)
prompt>
```

The exec() System Call

- Run a program that is different from the calling program

p3.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {                // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {        // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
        char *myargs[3];
        myargs[0] = strdup("wc");           // program: "wc" (word
count)
        myargs[1] = strdup("p3.c");         // argument: file to count
        myargs[2] = NULL;                  // marks end of array
        ...
    }
```

The exec() System Call (Cont.)

p3.c

(Cont.)

```
    execvp(myargs[0], myargs); // runs word count
    printf("this shouldn't print out");
} else {                                // parent goes down this path (main)
    int wc = wait(NULL);
    printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
           rc, wc, (int) getpid());
}
return 0;
}
```

Result

```
prompt> ./p3
hello world (pid:29383)
hello, I am child (pid:29384)
29 107 1030 p3.c
hello, I am parent of 29384 (wc:29384) (pid:29383)
prompt>
```

fork(), exec(), and realizing a simple shell

- You might wonder why we need a combination of fork() and exec()
 - Wouldn't it be better to just have an exec() system call that automatically performs fork()?
- The separation of fork() and exec() provides more flexibility
 - For instance, we can run some other instruction after fork() and before exec()
- Think about this feature of the linux shell:
 - `wc filename > newfile.txt`
 - The ">" is a redirection operator, which routes all the data that is supposed to be written on the standard output to a file named *newfile.txt* in this example
 - How do we implement such a feature?

All of the above with redirection

p4.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>
#include <sys/wait.h>

int
main(int argc, char *argv[]){
    int rc = fork();
    if (rc < 0) {                // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) { // child: redirect standard output to a file
        close(STDOUT_FILENO);
        open("./p4.output", O_CREAT|O_WRONLY|O_TRUNC, S_IRWXU);
        ...
    }
```

All of the above with redirection (Cont.)

p4.c

```
...
// now exec "wc"...
char *myargs[3];
myargs[0] = strdup("wc");           // program: "wc" (word count)
myargs[1] = strdup("p4.c");         // argument: file to count
myargs[2] = NULL;                   // marks end of array
execvp(myargs[0], myargs);          // runs word count
} else {                             // parent goes down this path (main)
    int wc = wait(NULL);
}
return 0;
}
```

Result

```
prompt> ./p4
prompt> cat p4.output
32 109 846 p4.c
prompt>
```

13. The Abstraction: Address Space

Operating System: Three Easy Pieces

Memory Virtualization

- So far we have seen how CPU can be virtualized to accommodate multiple running processes.

OGNI PROCESSO
HA L'ILLUSIONE DI AVERE LA CPU
TUTTA PER SE

- CPU is not the only resource of a computer. What about memory?

VOGHIAMO OTTENERE
QUESTO ANCHE PER
LA MEMORIA

- What is **memory virtualization**?

- OS virtualizes its physical memory.
 - OS provides an **illusory memory space** for each process.
 - It seems to be seen like **each process uses the whole memory**.

- Benefits:

- Ease of use in programming
 - Memory efficiency in terms of **time** and **space**
 - The guarantee of isolation for processes as well as OS

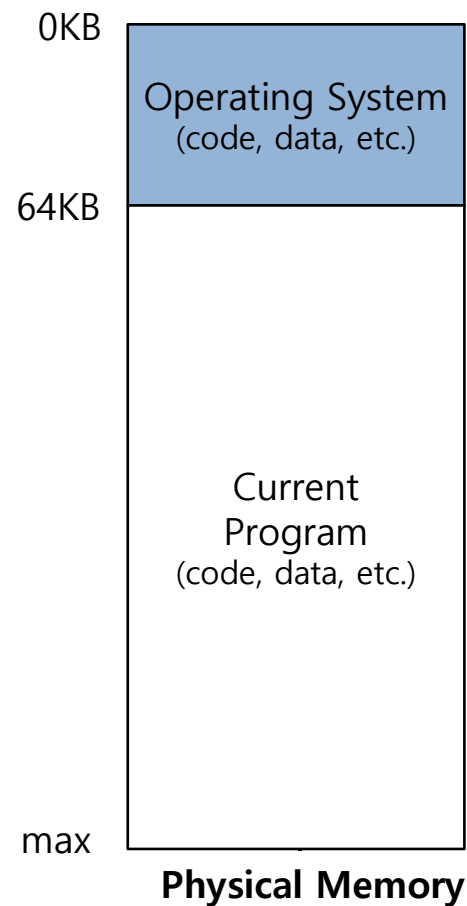
- Protection from **errant accesses** of other processes

NON VOGLIAMO
CHE UN PROCESSO ANCHE
SE BENEVOLO, CAMBI QUALCOSA
QUESTO PERCHÉ ANCHE IL MINIMO
CAMBIAMENTO POTREBBE FAR
CRASHARE IL SISTEMA

OS in The Early Systems

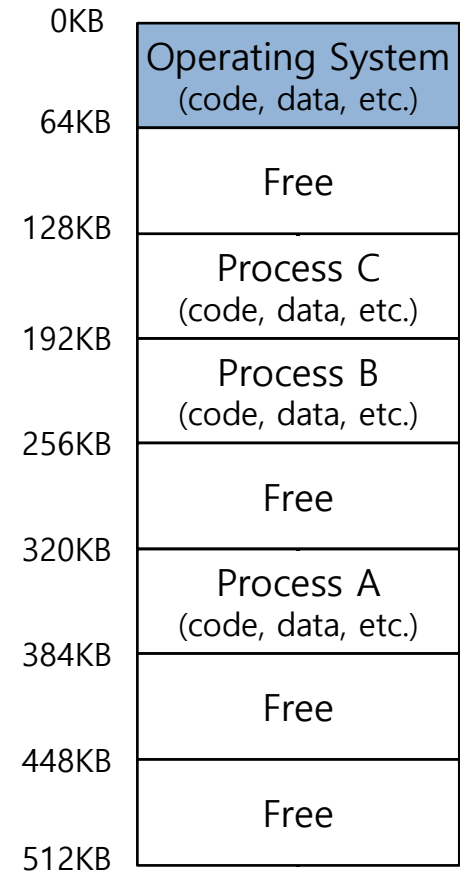
- Load only one process in memory.
 - Poor utilization and efficiency.
 - Batch systems vs interactive systems

— PRO: PRESTAZIONI INEGUAGLIABILI PER L'ACCESSO IN MEMORIA



Multiprogramming and Time Sharing

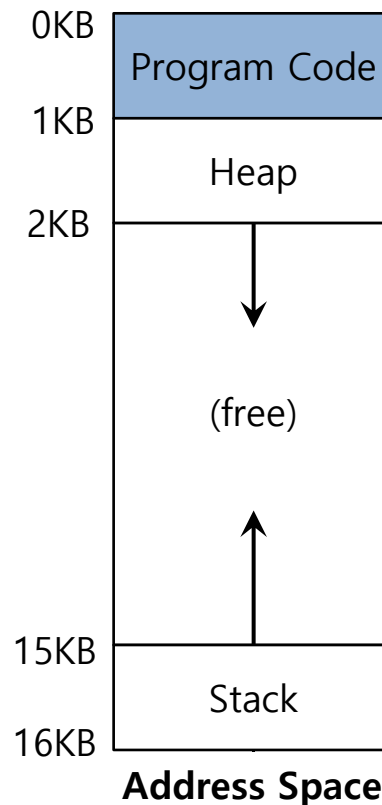
- **Load multiple processes** ^{7 LA MAGGIORA PARTE INTERATTIVI} in memory.
 - Execute one for a short while.
 - Switch between processes in memory.
 - Increase utilization and efficiency.
- Causes an important **protection issue**.
 - Errant memory accesses from other processes



Physical Memory

Address Space

- OS creates an **abstraction** of physical memory.
 - The address space contains all about a running process.
 - That consists of program code, heap, stack, etc.

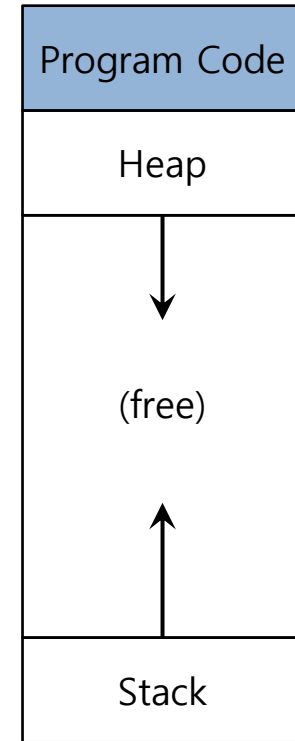


Ogni processo avrà un proprio spazio degli indirizzi, infatti avremo tanti spazi degli indirizzi che in termini di indirizzi stessi sono sovrapposti, il concetto è che questo spazio degli indirizzi è virtuale, ogni processo crede di avere il proprio spazio degli indirizzi e essendo indirizzi virtuali viene gestito tutto dal SO che li convertirà in indirizzi fisici.

Address Space(Cont.)

Porzioni:

- Code
 - Where instructions live
- Heap
 - Dynamically allocate memory.
 - `malloc` in C language
 - `new` in object-oriented language
- Stack
 - Store return addresses or values.
 - Contain local variables arguments to routines.



Address Space

Virtual Address

- **Every address** in a running program is virtual.
 - OS translates the virtual address to physical address

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main(int argc, char *argv[]){
```

```
    printf("location of code   : %p\n", (void *) main);
```

```
    printf("location of heap   : %p\n", (void *) malloc(1));
```

```
    int x = 3;
```

```
    printf("location of stack : %p\n", (void *) &x);
```

```
    return x;
```

```
}
```

LANCIANDO DIVERSE
VOLTE SETTEBRA' UTILIZZARE
SEMPRE GLI STESSI INDIRIZZI,
QUESTO PERCHÉ SONO VIRTUALI

INDIRIZZO DEL MAIN
DOVEREBBE AVERE
L'INDIRIZZO PIÙ BASSO

ALLOCA 1 BYTE DI MEMORIA
INDIRIZZO DEL HEAP

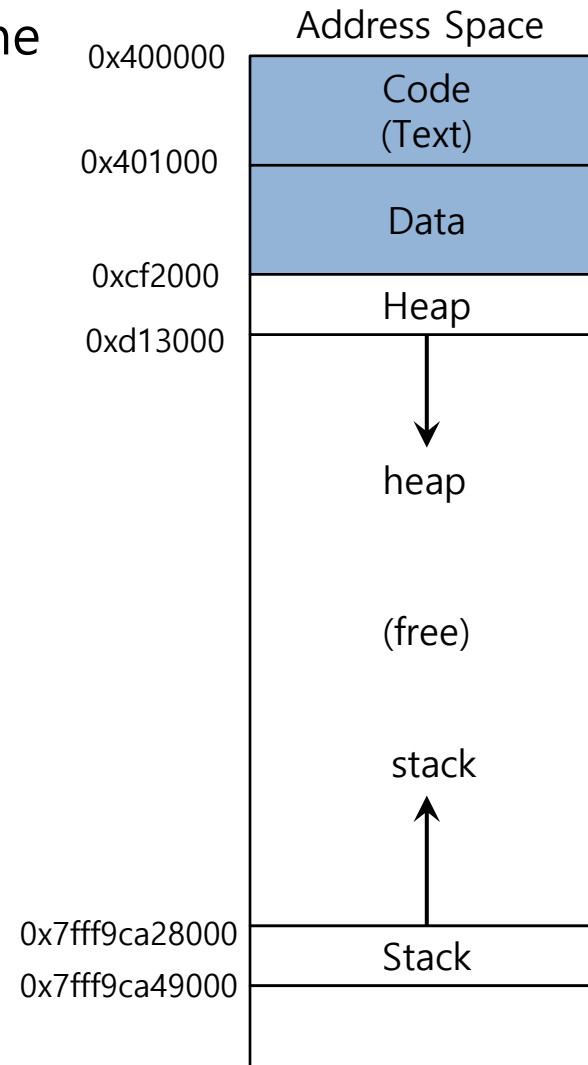
PUNTERE ALLA
BASE DELLO STACK

A simple program that prints out addresses

Virtual Address(Cont.)

- The output on a 64-bit Linux machine

```
location of code   : 0x40057d
location of heap   : 0xcf2010
location of stack  : 0x7fff9ca45fcc
```



15. Address Translation

Operating System: Three Easy Pieces

Memory Virtualizing with Efficiency and Control

- Why memory virtualization?
 - Two main reasons: Flexibility and Access Limitation.
- In Limited Direct Execution (LDE) for CPU virtualization, we used some hardware support to realize an efficient mechanism so that the OS could always be in control
- We want to obtain the same results (efficiency+control) in memory virtualization
 - Similarly to CPU virtualization, we will need some support from the hardware
 - Very simple at the beginning, but will grow fairly complex then.
- We want to study here mechanisms to realize an efficient virtualization of memory.

Address Translation

- The hardware support consists in the transformation of a **virtual address** to a **physical address**.
 - The desired information is actually stored in a physical address.
- The hardware, by itself, only provides mechanisms for the translation.
 - The OS is the entity that drives the hardware in the translation process
- Just like CPU virtualization, we want to create an **illusion**: the illusion that every process has all the memory for itself

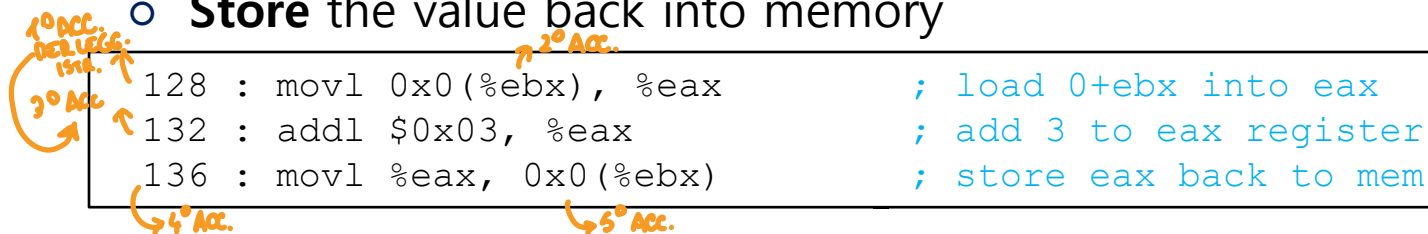
3 Assumptions

- We will start the analysis of memory virtualization with 3 (unrealistic) assumptions:
 - The address space of a process must be placed contiguously in physical memory
 - The size of the address space is smaller than the size of the physical memory
 - Each address space has the same size
- Let's start with something simple, then we will relax these assumptions.

Example: Application Accessing Memory

```
void func() {  
    int x;  
    ...  
    x = x + 3; // this is the line of code we are interested in  
}
```

- In C code:
 - **Load** a value from memory
 - **Increment** it by three
 - **Store** the value back into memory



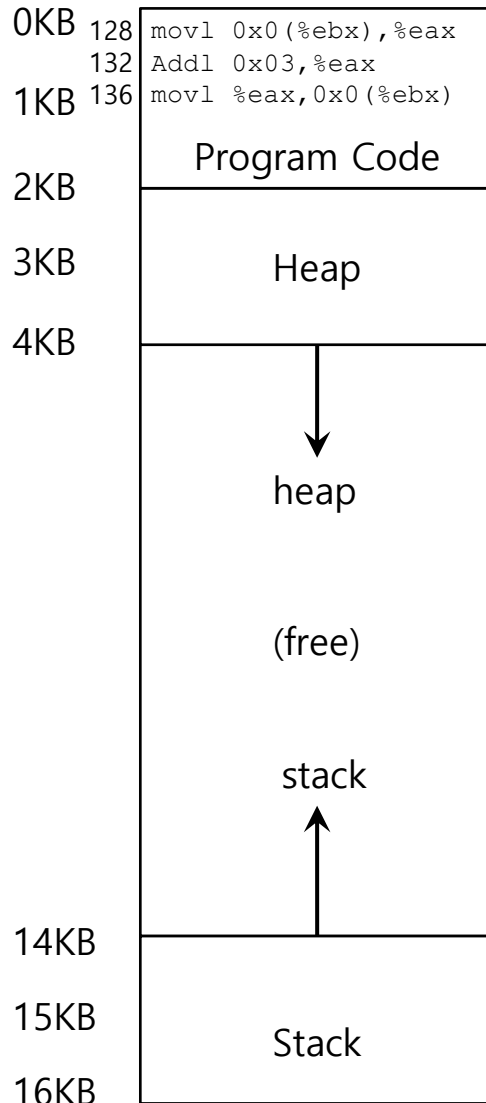
Handwritten annotations in orange:

- 10 Acc. Dec. Left: 1312. (with an arrow pointing to the first instruction)
- 30 Acc. (with an arrow pointing to the second instruction)
- 40 Acc. (with an arrow pointing to the third instruction)
- 20 Acc. (with an arrow pointing to the first instruction)
- 50 Acc. (with an arrow pointing to the third instruction)

```
128 : movl 0x0(%ebx), %eax      ; load 0+ebx into eax  
132 : addl $0x03, %eax         ; add 3 to eax register  
136 : movl %eax, 0x0(%ebx)     ; store eax back to mem
```

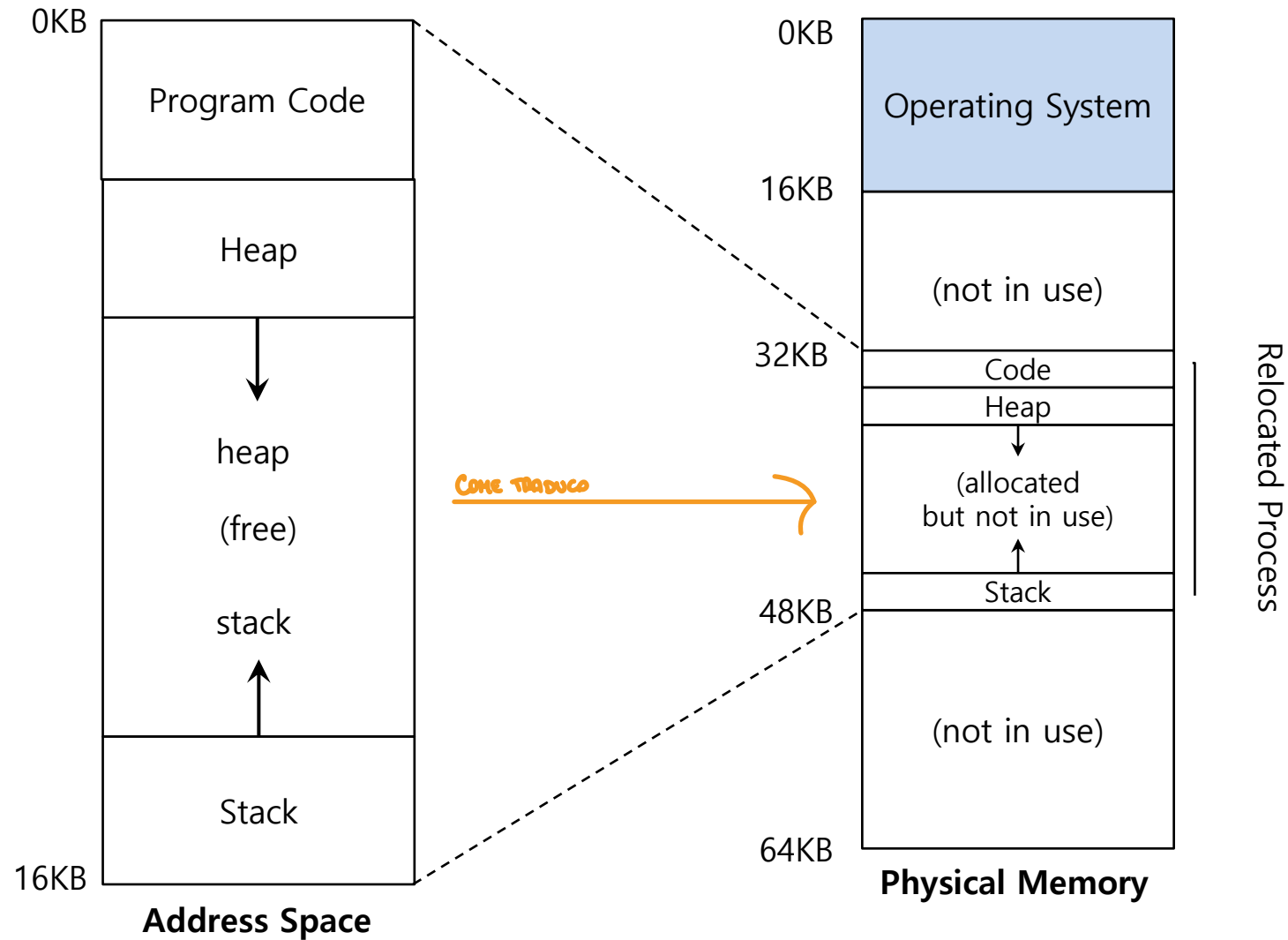
- In Assembly code:
 - **Load** the value at that address into `eax` register.
 - **Add** 3 to `eax` register.
 - **Store** the value in `eax` back into memory.

Example: Application Accessing Memory (Cont.)



- Fetch instruction at address 128
- Execute this instruction (load from address 15KB)
- Fetch instruction at address 132
- Execute this instruction (no memory reference)
- Fetch the instruction at address 136
- Execute this instruction (store to address 15 KB)

A Single Relocated Process



Base and Bound Registers

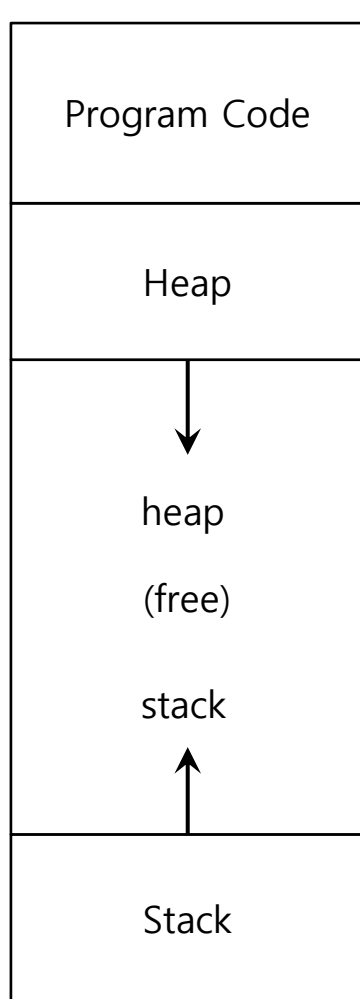
Approccio
BASE AND
BOUND: AGGIUNGO
DUE PEZZI DI HW
PER I REGISTRI
BASE E BOUND

Nel base ce l'indirizzo
iniziale della memoria
fisica, nel bound ce la
dimensione dello spazio
degli indirizzi che sto
tenendo in
considerazione

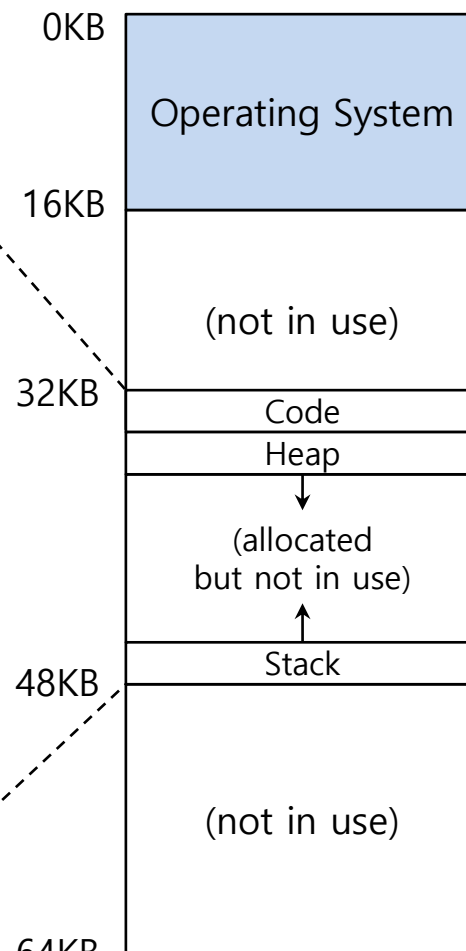
bound register

16KB

16KB



Address Space



Physical Memory

base register

32KB

Ora il problema è che non
vogliamo che qualcuno
acceda ad un determinato
indirizzo perché conosce
questa formuletta, grazie al
registro bound controllo
se la richiesta di traduzione
dell'indirizzo non sia più
grande della dimensione
allocata per il processo

TRADUZIONE

$$\text{Physical address} = \text{virtual address} + \text{base}$$

Relocation and Address Translation

128 : movl 0x0(%ebx), %eax

→ 16DEDE
L'IND101220 F 15110

- **Fetch** instruction at address 128

$128 \rightarrow 128 + 32768 = 32896$

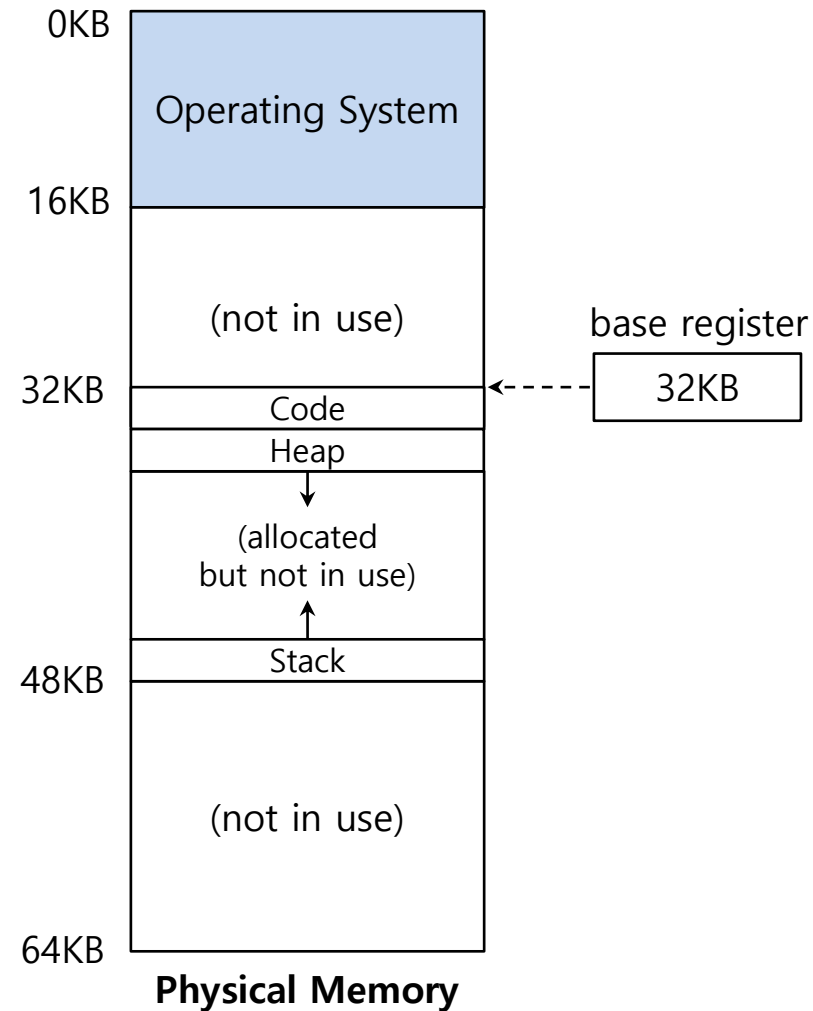
→ 32KB

- **Execute** this instruction

- Load from address 15KB

$15K \rightarrow 15K + 32K = 47K$

→ DESTINO
EAX



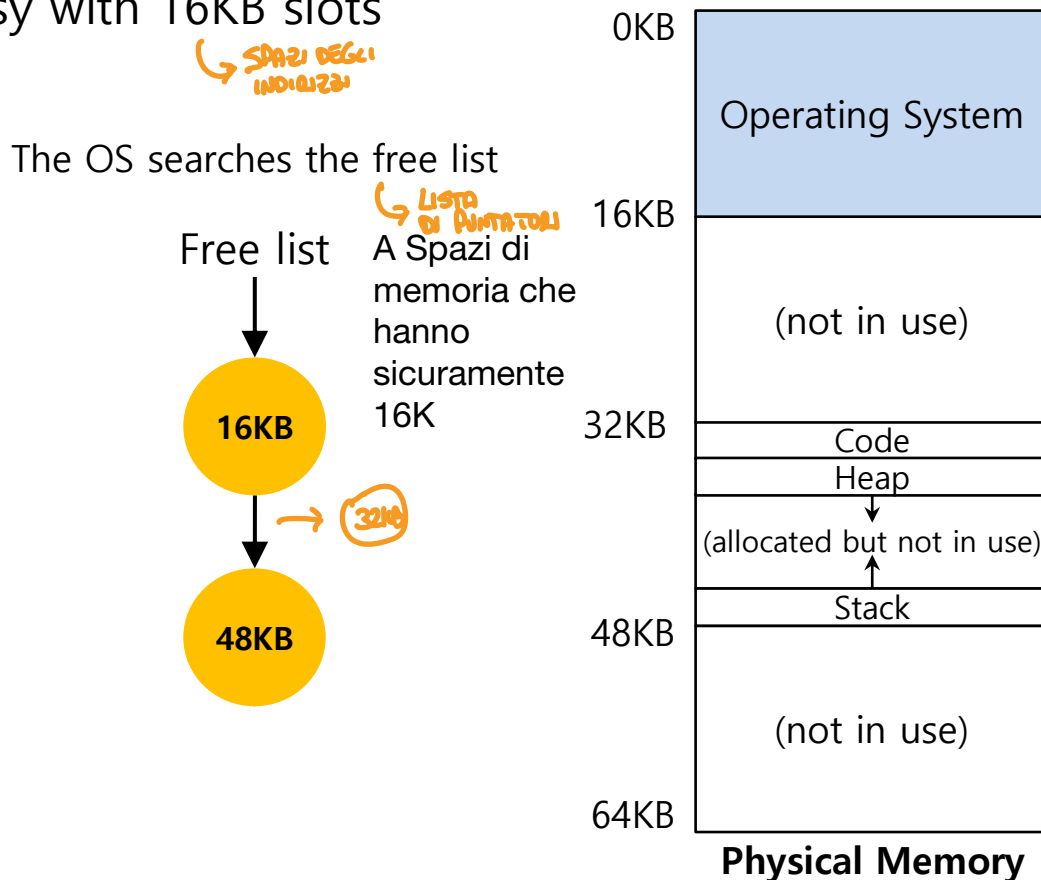
OS Issues for Memory Virtualizing

- Hardware support is good, but we also need some OS implementation
- The OS must **take action** to implement **base-and-bounds** approach.
- Three critical junctures:
 - ↳ FUNT. DIGITIZATION
 - When a process **starts running**:
 - Finding space for address space in physical memory
 - When a process is **terminated**:
 - Reclaiming the memory for use
 - When context **switch occurs**:
 - Saving and storing the base-and-bounds pair

↑ SUFFICIENTLY
GRANT PER TO SPACE
DEGR. INDUZZI
CREATO

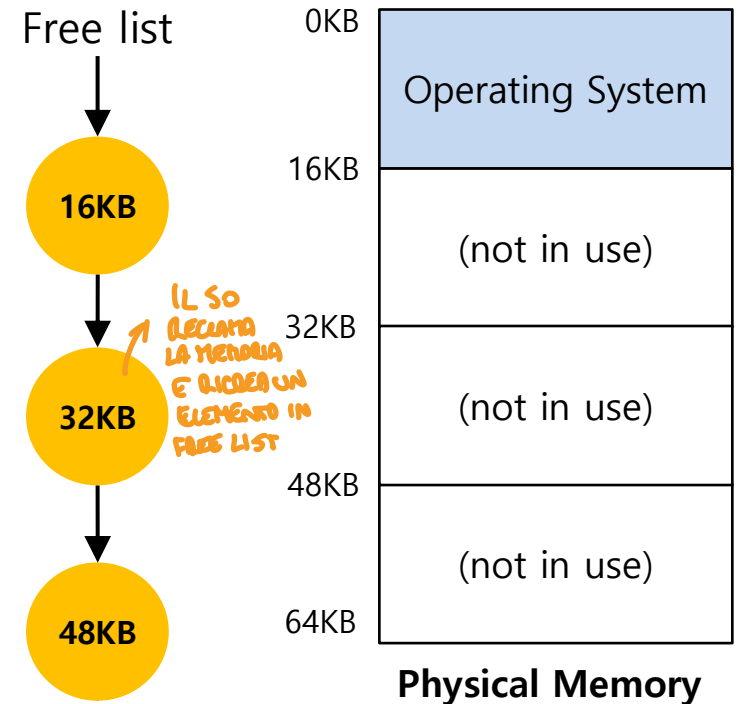
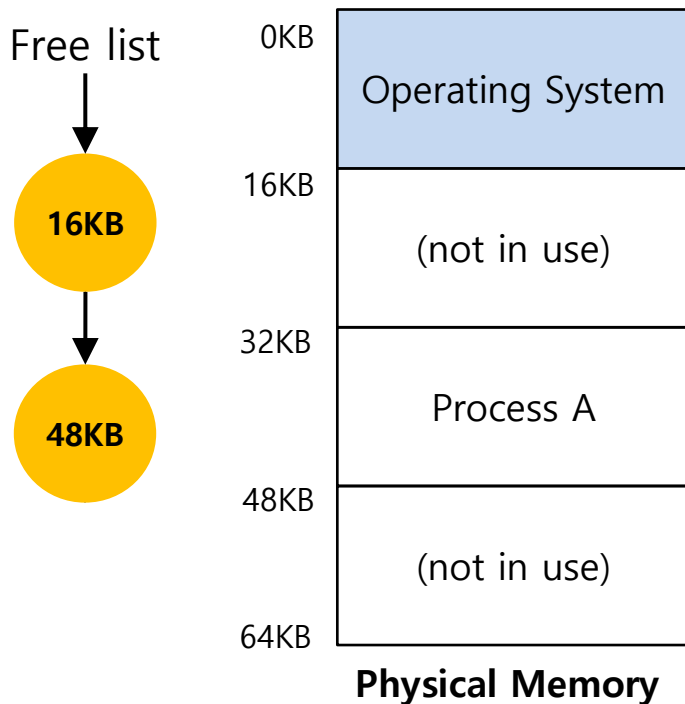
OS Issues: When a Process Starts Running

- The OS must **find room** for a new address space.
 - free list : A list of the ranges of the physical memory which are not in use.
 - Easy with 16KB slots



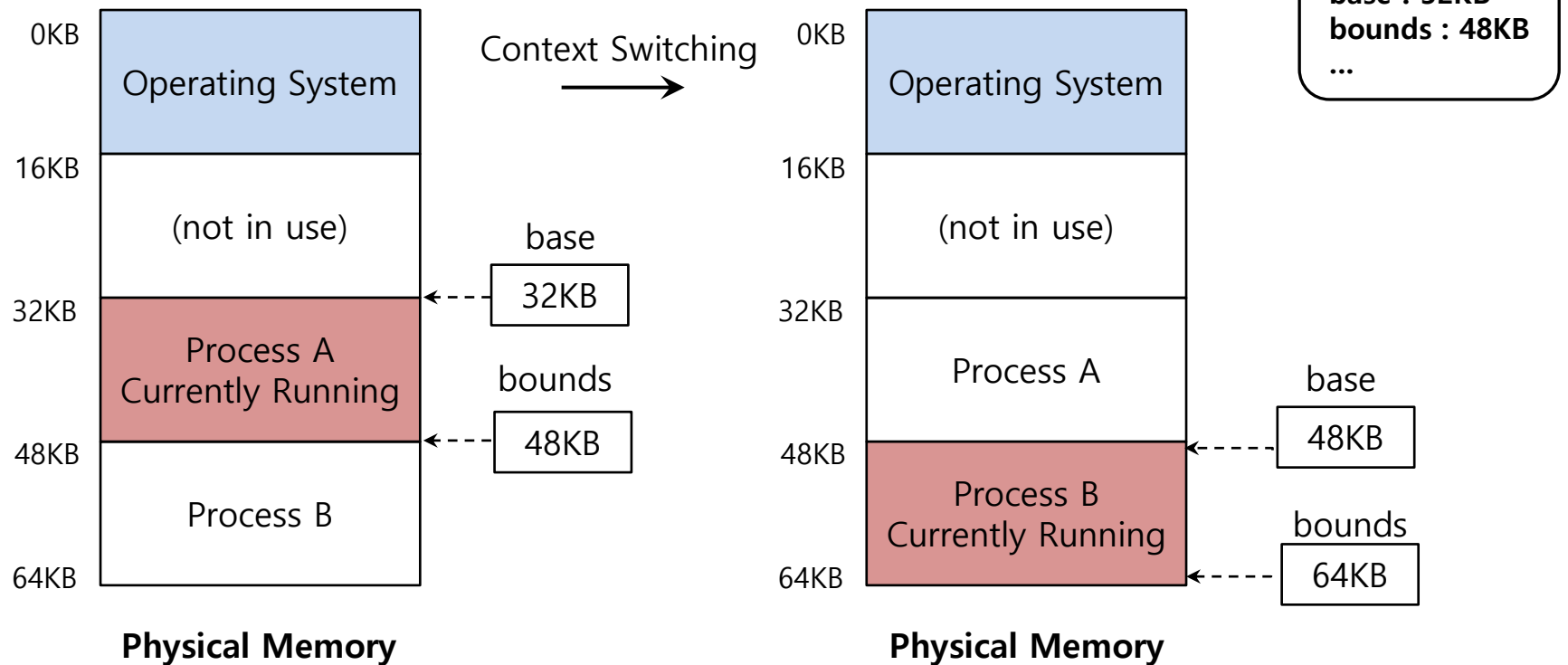
OS Issues: When a Process Is Terminated

- The OS must **put the memory back** on the free list.



OS Issues: When Context Switch Occurs

- The OS must **save and restore** the base-and-bounds pair.
 - In **process structure** or **process control block (PCB)**



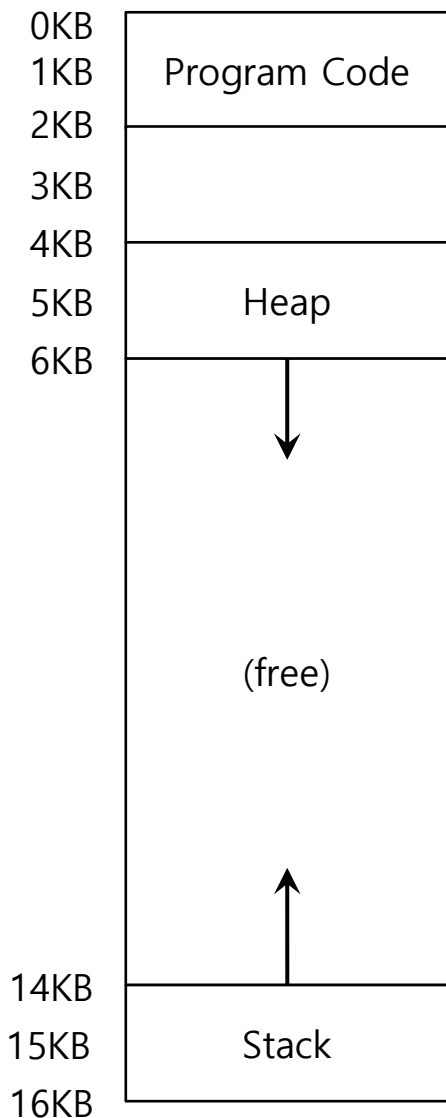
Summary of Hardware Requirements so far

Hardware Requirements	Notes
Privileged mode → USER MODE KERNEL MODE	Needed to prevent user-mode processes from executing privileged operations
Base/bound registers ↳ IN COMPACT REGISTERS	Need pair of registers per CPU to support address translation and bound checks
Ability to translate virtual addresses and check if within bounds	Circuitry to make translations and check limits, in this case quite simple
Privileged instruction(s) to update base/bounds	OS must be able to set these values before letting a user program run
Privileged instruction(s) to register exception handlers	OS must be able to tell hardware what code to run if exception occurs
Ability to raise exceptions	When processes try to access privileged instructions or out-of-bounds memory

16. Segmentation

Operating System: Three Easy Pieces

Inefficiency of the Base and Bound Approach

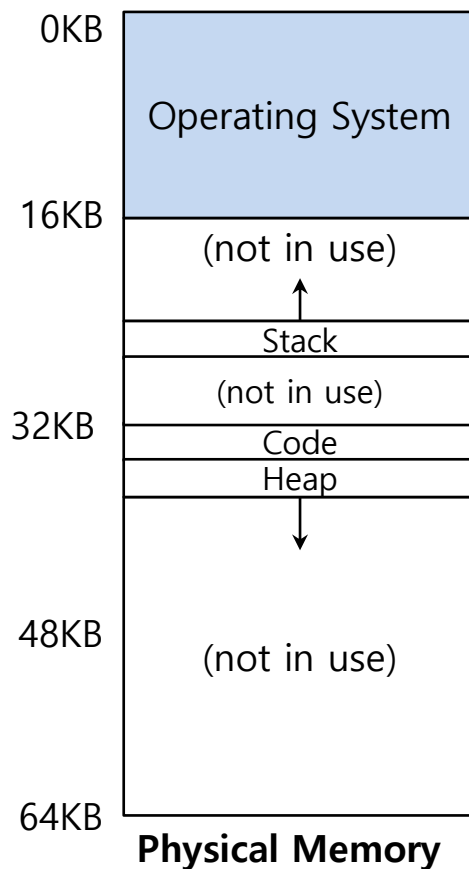


- **Big chunk of “free”** space
- “free” space **takes up** physical memory.
 - In this example the address space is only 16KB large. In a 32-bit architecture, it could be 4GB (in 32-bit Linux it is usually 3GB).
- Hard to run when an address space **does not fit** into physical memory

Segmentation

- A Segment is a **contiguous portion** of the address space of a particular length.
 - Logically-different segment: code, stack, heap
- Each segment can be **placed** in **different parts of physical memory**.
 - **Base** and **bounds** exist **per each segment**.

Placing Segments In Physical Memory



LO SPAZIO DEGLI INDIZZI È STATO POSIZIONATO
DALL SO IN PUNTI DIFFERENTI

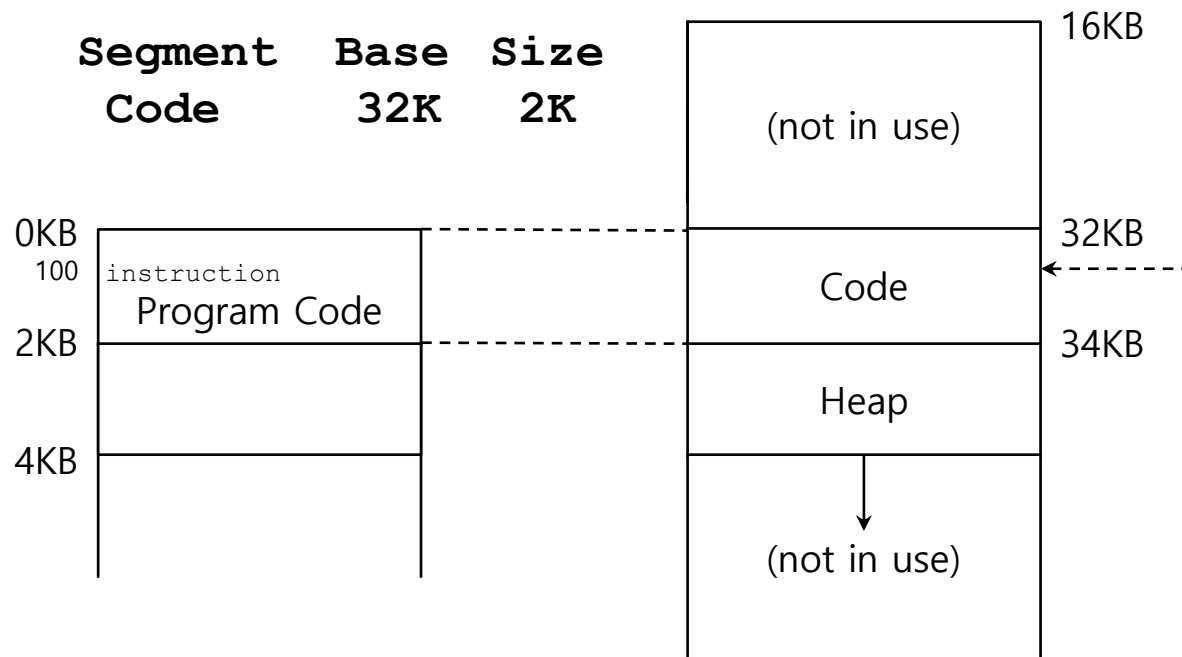
Segment	Base	Size
Code	32K	2K
Heap	34K	2K
Stack	28K	2K

Address Translation on Segmentation

$$\text{physical address} = \text{offset} + \text{base}$$

POSIZIONE
IN CUI MI
TROVO ORA
→ "DISTANZA
RISPETTO
A UNA BASE"

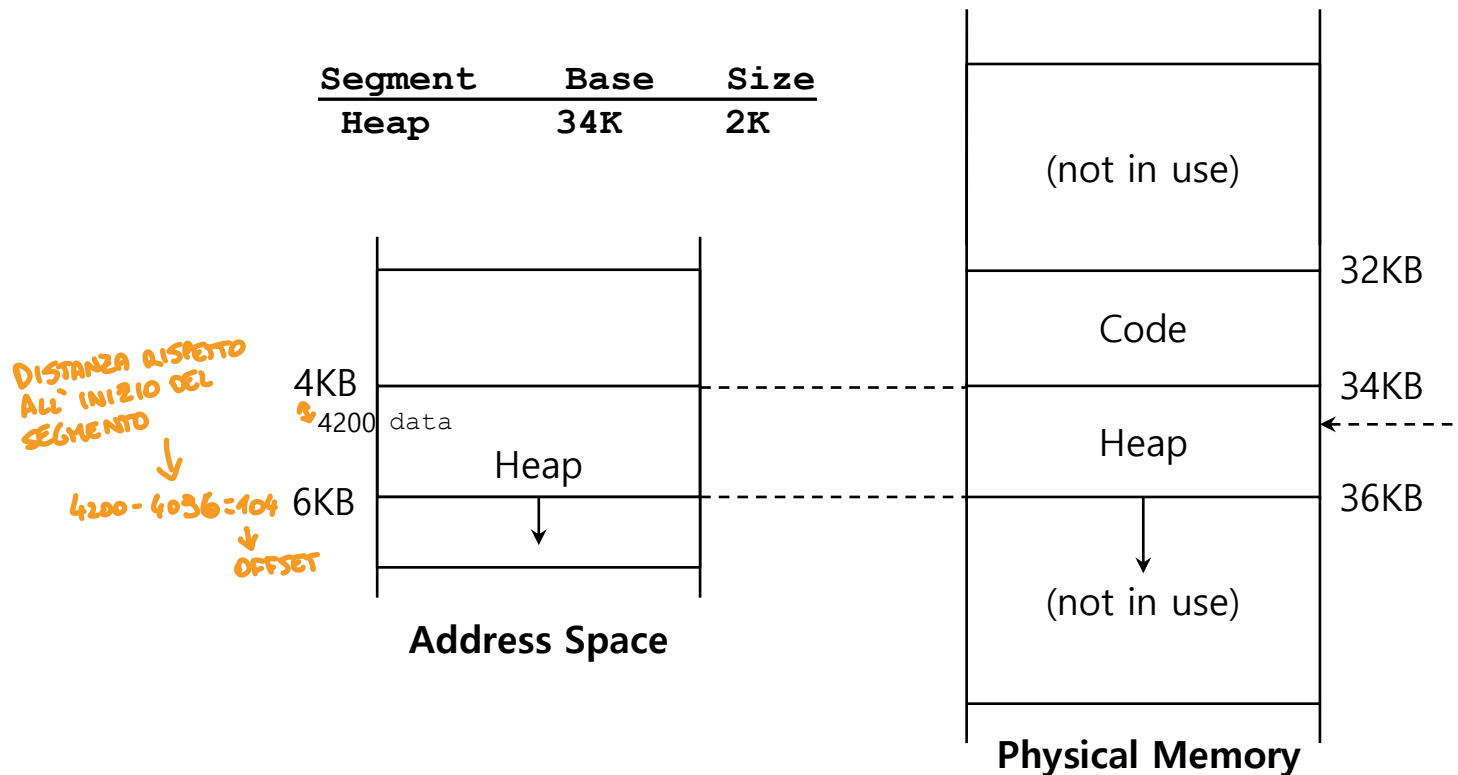
- Example: how does the instruction at byte 100 translate to physical memory?
- The `offset` of virtual address 100 is 100.
 - The code segment **starts at virtual address 0** in this address space.



1. Computation of the physical address: $32K + 100 = 32868$ VERIFICO L'OFFSET
2. Bound check: $100 < 2K$
3. Load instruction from physical memory

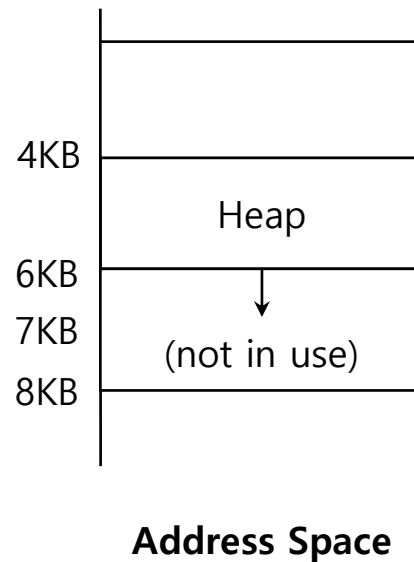
Address Translation on Segmentation (Cont.)

- The heap segment **starts at virtual address 4096** in the address space.
 - The `offset` of virtual address 4200 is 104.



Segmentation Fault or Violation

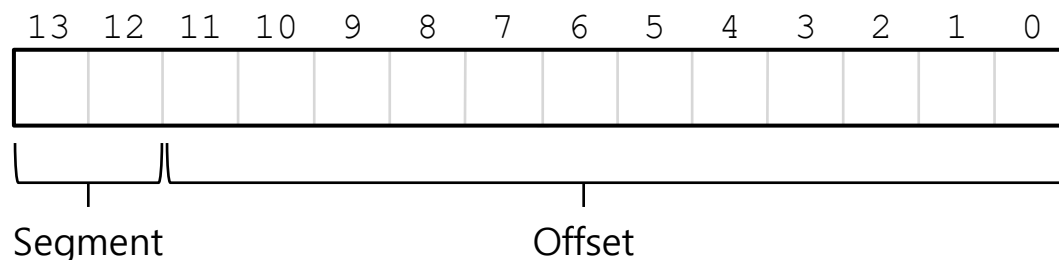
- If an **illegal address** is referenced, such as 7KB, which is beyond the end of the heap, the CPU raises an exception, and the OS terminates the process because of a **segmentation fault**.
 - The hardware detects that the address is **out of bounds**.



Referring to Segment

- **Explicit approach**

- Chop up the address space into segments based on the **top few bits** of virtual address.

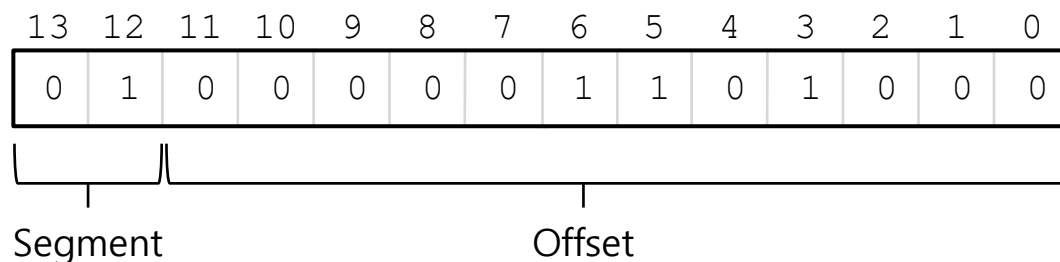


IL NUM. DI
SEGMENTI LO SAPPIAMO
A PRIORI

DATO UN INDIRIZZO 1024 BIT PIU' SIGNIFICATIVI
CI DICONO L'INDICE A CUI VOGLIAMO FARE
RIFERIMENTO

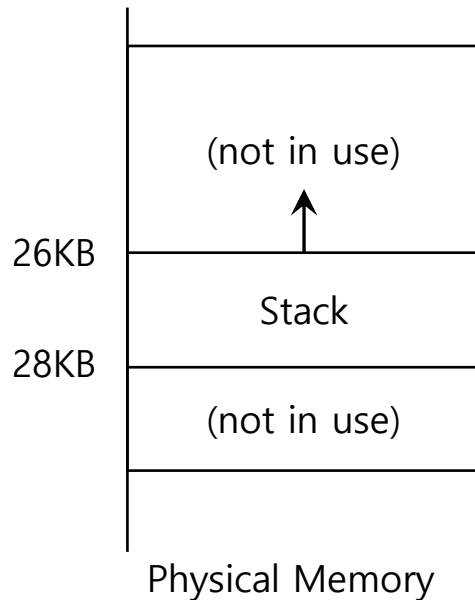
- Example: virtual address 4200 (010000001101000)

Segment	bits
Code	00
Heap	01
Stack	10
-	11



Referring to Stack Segment

- Stack grows **backward**.
- **Extra hardware support** is need.
 - The hardware checks which way the segment grows.
 - 1: positive direction, 0: negative direction



Segment Register (with Negative-Growth Support)

Segment	Base	Size	Grows Positive?
Code	32K	2K	1
Heap	34K	2K	1
Stack	28K	2K	0

MEMORIZA
SE O SEGMENTO
CRESCE POSITIVAMENTE
O NEGATIVAMENTE

Support for Sharing

- Segments can be **shared among address spaces**.
 - **Code sharing** is still in use in systems today.
 - Needs extra hardware support: **protection bits**
 - **A few more bits** per segment to indicate **permissions** of **read**, **write** and **execute**.

Segment Register Values(with Protection)

Segment	Base	Size	Grows	Positive?	Protection
Code	32K	2K		1	Read-Execute
Heap	34K	2K		1	Read-Write
Stack	28K	2K		0	Read-Write

SEGMENTO CONDIVISO O NON CONDIVISO
E CON CHE PRIVILEGI

Fine-Grained and Coarse-Grained

- **Coarse-Grained** means a small number of large segments (e.g., code, heap, stack.) *GRANA
GROSSA*
 - Advantage: easy to realize with some additional base/bound register
 - Drawback: subject to heavy **internal fragmentation** *Poco Ho
AGGIUNTO*
*HO DEI BUCCHI DI MEMORIA LIBERA
E DATI SPASTI* *NON LI POSSO UTILIZZARE
PERCHÉ NON SO COSA C'È
DENTRO*
- **Fine-Grained** segmentation means a large number of small segments
 - Advantage: limited internal fragmentation
 - Drawback: a pre-defined number of base/bound registers is not enough for its realization.

OS support: Fragmentation

SI CREA QUANDO ANDIAMO A RIALLOCARE PICCOLI SPAZI DI MEMORIA

- **External Fragmentation:** little holes of **free space** in physical memory that make it difficult to allocate new segments.

ESTERNA RISPETTO ALL'UNITA' DI SEGMENTO

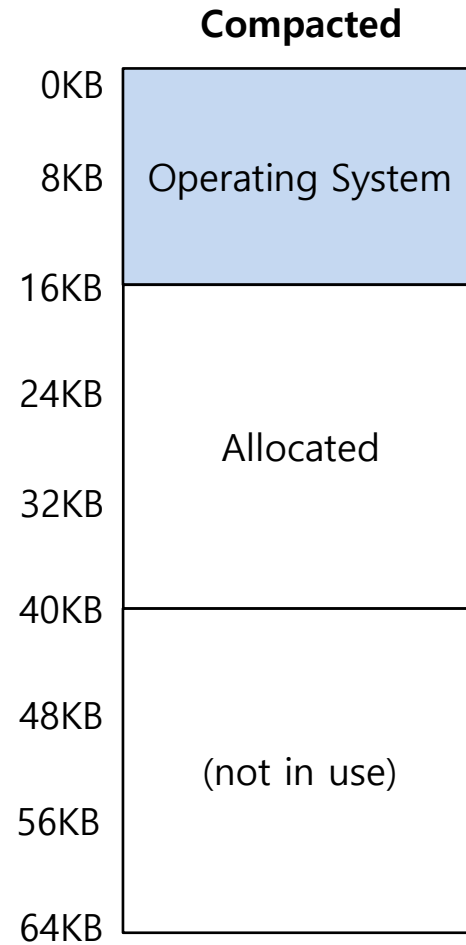
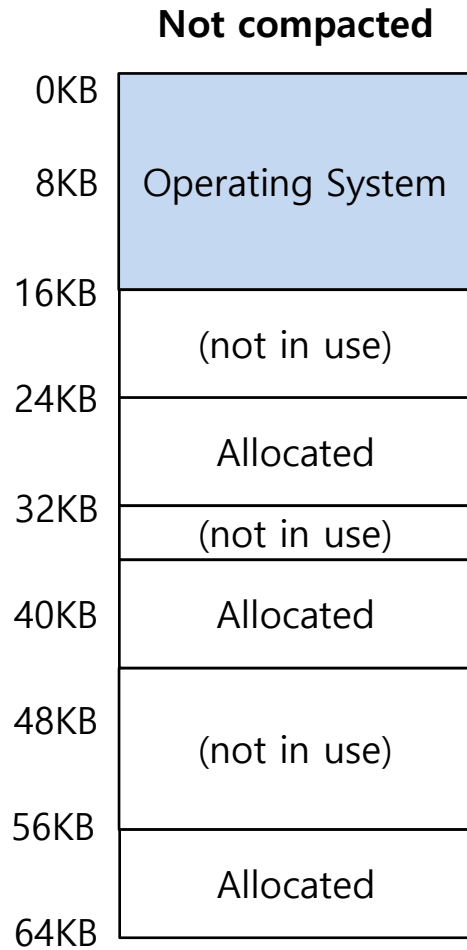
- For example, if there are **24KB of free memory**, but **not in one contiguous** segment, the OS **cannot** satisfy a **20KB request**.

- **Compaction:** rearranging the existing segments in physical memory.

PER MASSIMIZZARE LA MEMORIA DISPONIBILE IN UNO CONTIGUO

- Compaction is **costly**.
 - **Stop** running process.
 - **Copy** data to another position.
 - **Change** segment register value.

Memory Compaction

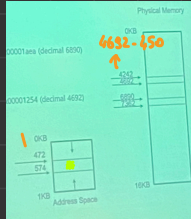


Esercizi di Traduzione degli indirizzi basati su segmentazione

Esercizi su repository Git-Hub OSTEP-Homework

1. FARE IL DISEGNO

- DISEGNARE GLI INDIRIZZI
- FACCIAMO DEI PAGAMENTI



Segmentation Exercise

Address Space: 1KB
Physical Memory: 16KB

Segment 0 base (grows positive): 0x00001000 (decimal 65536)
Segment 0 limit: 472 $\hookrightarrow$ CODE / HEAP

Segment 1 base (grows negative): 0x00001254 (decimal 4682)
Segment 1 limit: 450 $\hookrightarrow$ STACK

Virtual Address Trace

VA 0: 0x0000020b (decimal: 523) $\rightarrow$ PA or segmentation violation?

VA 1: 0x0000019e (decimal: 414) $\rightarrow$ PA or segmentation violation?

VA 2: 0x00000322 (decimal: 802) $\rightarrow$ PA or segmentation violation?

VA 3: 0x00000136 (decimal: 310) $\rightarrow$ PA or segmentation violation?

VA 4: 0x000001e8 (decimal: 488) $\rightarrow$ PA or segmentation violation?

$\hookrightarrow$ SEGMENTATION VIOLATION

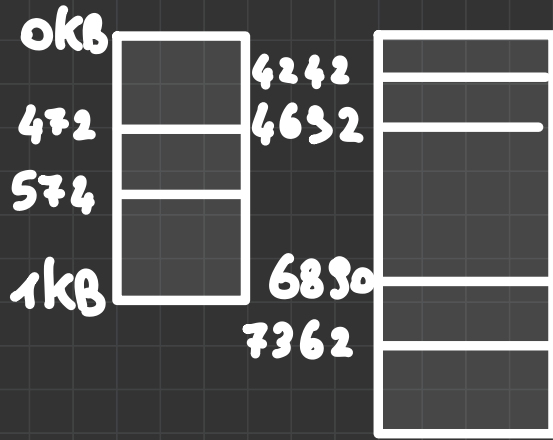
$\rightarrow$ CAPISCO A QUALE SEGMENTO APPARTIENE

$$414 + 65536 = 65950$$

$$1024 - 802 = 222$$

$$65536 - 222 = 65314$$

$$310 + 65536$$



1. INVALID

2. $6830 + 414 = 7244$

3. $1024 - 802 = 222$

$4632 - 222 = 4410$

4. $6830 + 310 = 7140$

5. INVALID

18. Paging: Introduction

Operating System: Three Easy Pieces

Concept of Paging

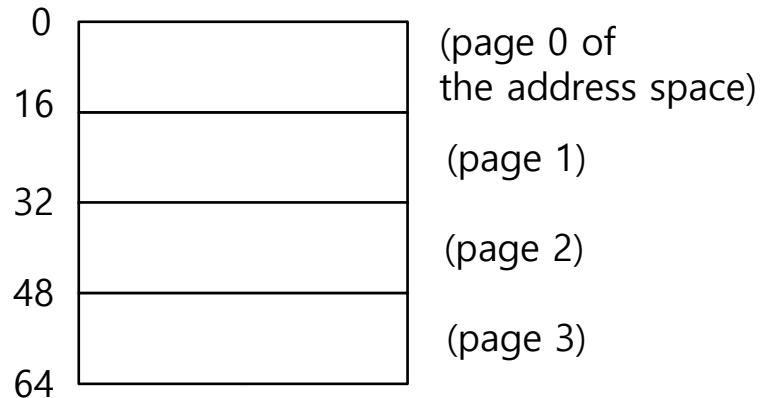
- **Segmentation** is used to divide the address space of an application in multiple pieces (e.g., code, stack, heap)
- Variable segments size
- One of the issues of segmentation is **memory fragmentation**.
- **Paging** splits up the address space into **fixed-sized** units called **pages**.
- With paging, **physical memory** is also **split** into some number of pages called **page frames**.
- The problem that we are trying to solve is *how to efficiently translate a **virtual page number (VPN)** into its corresponding **page frame number (PFN)***

Advantages Of Paging

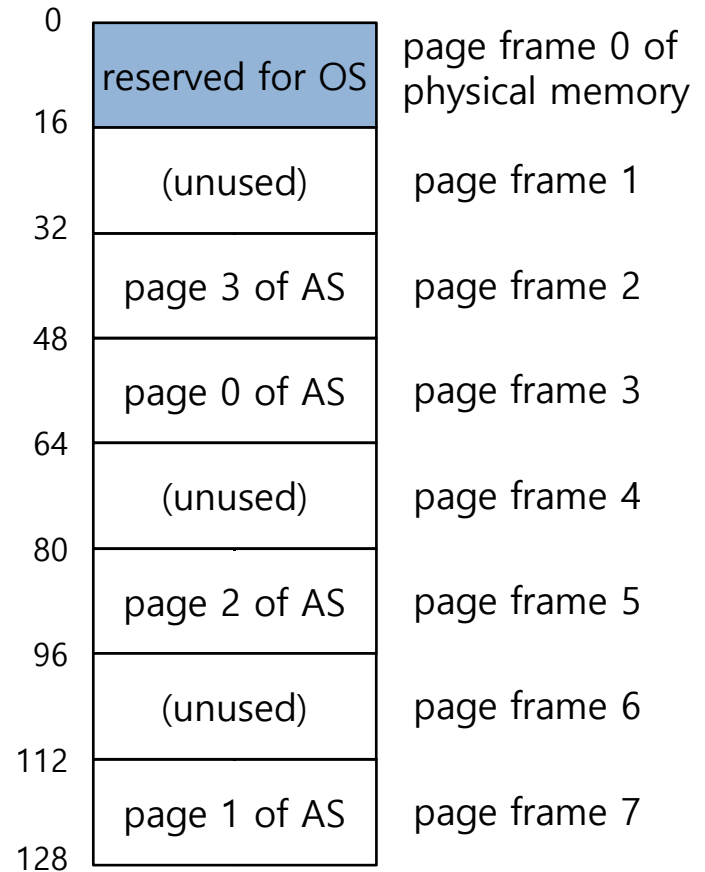
- **Flexibility:** Supports the abstraction of address space effectively
 - Does not need assumptions on how heap and stack grow and are used.
 - It is not constrained by the need of defining segments containing logically coherent data
- **Simplicity:** ease of free-space management
 - The page in address space and the page frame are the same size.

Example: Simple Paging

- 128-byte physical memory with 16 bytes page frames
- 64-byte address space with 16 bytes pages



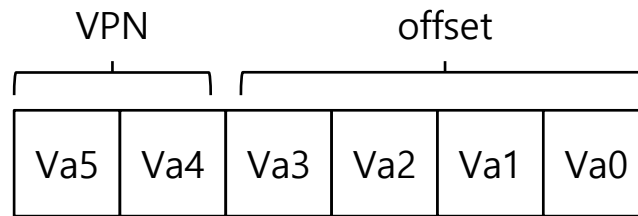
A Simple 64-byte Address Space



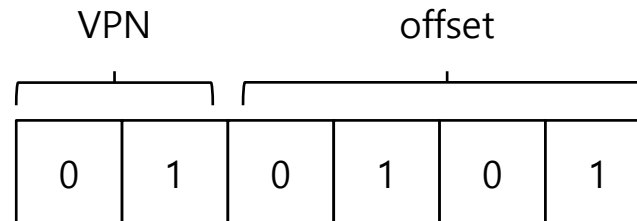
64-Byte Address Space Placed In Physical Memory

Address Translation

- Virtual address of 64 bytes → We need 6 bits to identify an address
 - VPN: virtual page number
 - Offset: offset within the page

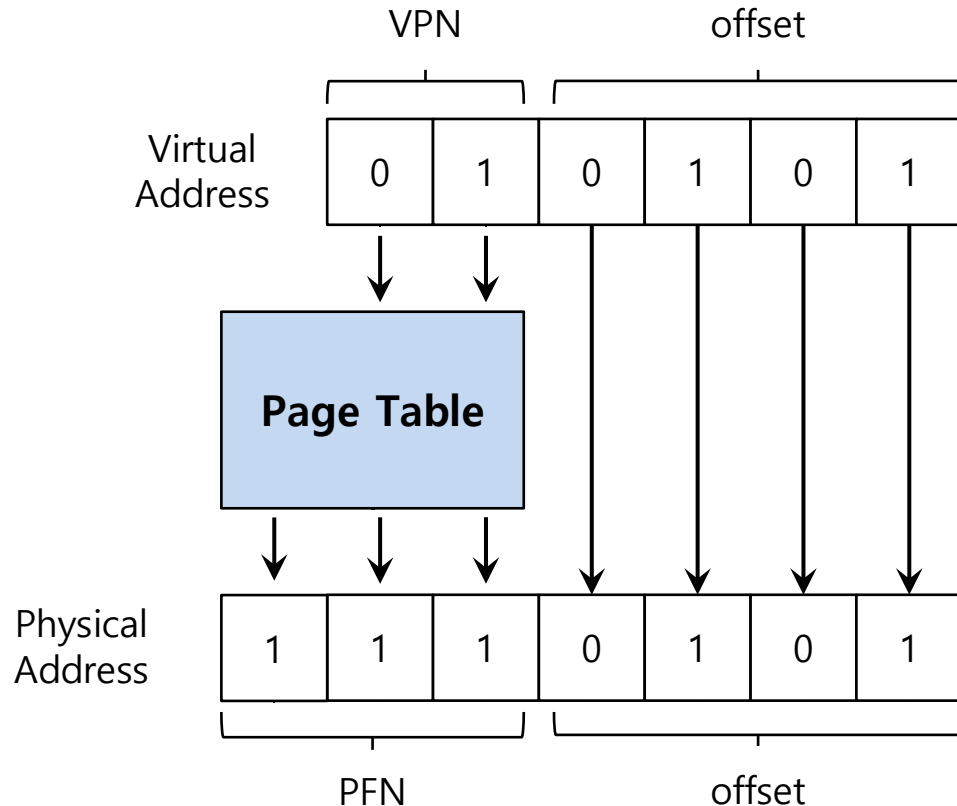


- Example: virtual address 21 in 64-byte address space



Example: Address Translation

- The virtual address 21 in 64-byte address space

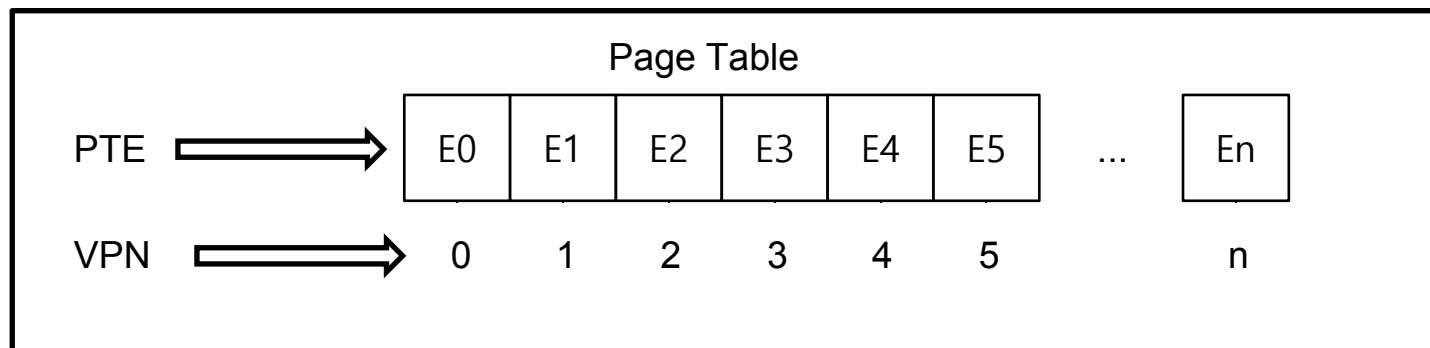


0	reserved for OS	page frame 0 of physical memory
16	(unused)	page frame 1
32	page 3 of AS	page frame 2
48	page 0 of AS	page frame 3
64	(unused)	page frame 4
80	page 2 of AS	page frame 5
96	(unused)	page frame 6
112	page 1 of AS	page frame 7
128		

64-Byte Address Space Placed In Physical Memory

What is the Page Table?

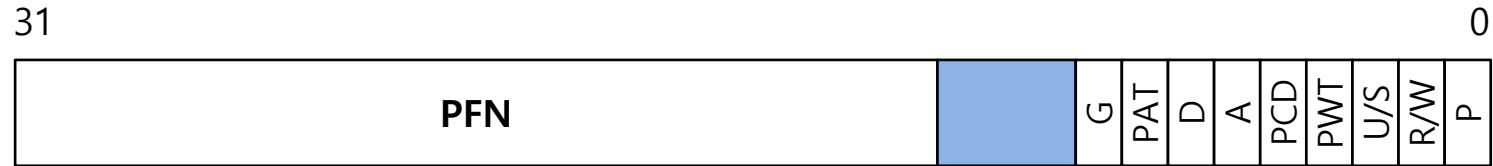
- The page table is a **data structure** that is used to map a virtual address to a physical address.
 - Simplest form: a linear page table, an array.
- The OS **indexes** the array by VPN, and looks up the page-table entry (PTE).



Content of a PTE

- **Page Frame Number:** the number of the page frame in physical memory
- **Valid Bit:** Indicates whether the particular translation is valid.
- **Protection Bits:** Indicates whether the page could be read from, written to, or executed from
- **Present Bit:** Indicates whether this page is in physical memory or on disk (swapped out, more on this later in the course)
- **Dirty Bit:** Indicates whether the page has been modified since it was taken into memory
- **Reference Bit (Accessed Bit):** Indicates that a page has been accessed

Example: x86 Page Table Entry (PTE)



A x86 Page Table Entry (PTE)

- P: present
- R/W: read/write bit
- U/S: supervisor
- A: accessed bit
- D: dirty bit
- PFN: the page frame number

Where Are Page Tables Stored?

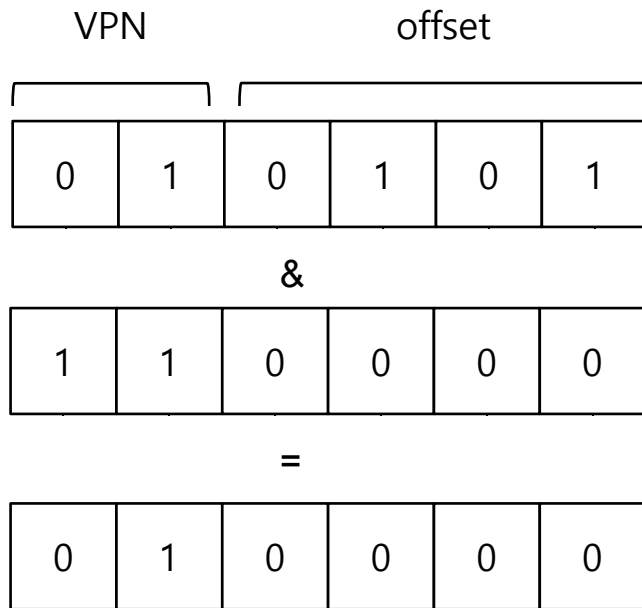
- Page tables can get very large
 - With segments, we used to have just a few of them (e.g., code, heap, stack)
 - A typical size for a page is around 4KB (in Linux you can find out the size by running `getconf PAGE_SIZE`)
 - Considering a 32-bit address space (4 GB) and 4KB page size, we need 20 bits to identify the page and 12 bits to identify the offset within the page itself.
 - Let's do some math: how much space do we need to store a page table?
 - our page table has 2^{20} entries = 1,048,576 entries = 1M
 - each entry must at least contain the 32-bit (4 bytes) address of the physical memory backing that page
 - the page table is 4 bytes * 1M large → 4MB
 - Now consider that you could have 100 processes or more running on your computer → more than 400MB just for the page tables!!!

Paging: Very Slow!

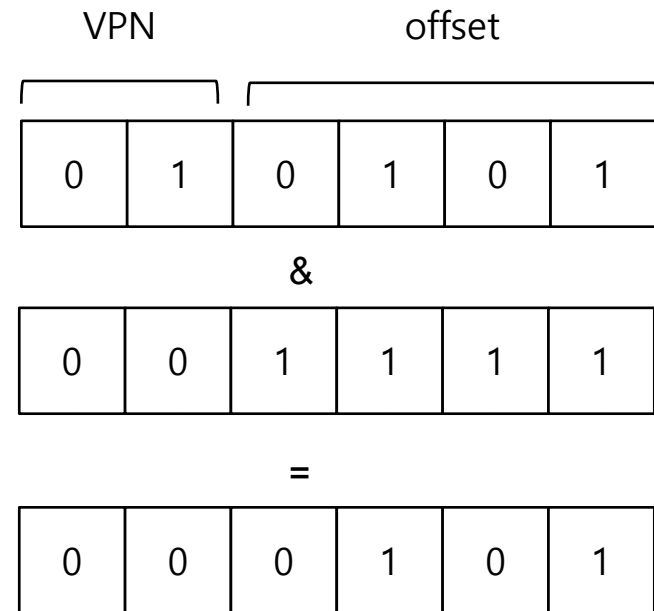
- To find a location of the desired PTE, the **starting location** of the page table is **needed**.
- For every memory reference, paging requires the OS to perform one **extra memory reference**.

VPN Mask and Offset Mask

Example of VPN Mask



Example of Offset Mask



Accessing Memory With Paging

Paging can be very slow. Let's see what happens with this simple instruction: `movl 21, %eax`

```
1      // Extract the VPN from the virtual address
2      VPN = (VirtualAddress & VPN_MASK) >> SHIFT
3
4      // Form the address of the page-table entry (PTE)
5      PTEAddr = PTBR + (VPN * sizeof(PTE))
6
7      // Fetch the PTE
8      PTE = AccessMemory(PTEAddr)
9
10     // Check if process can access the page
11     if (PTE.Valid == False)
12         RaiseException(SEGMENTATION_FAULT)
13     else if (CanAccess(PTE.ProtectBits) == False)
14         RaiseException(PROTECTION_FAULT)
15     else
16         // Access is OK: form physical address and
17         fetch it
18         offset = VirtualAddress & OFFSET_MASK
19         PhysAddr = (PTE.PFN << PFN_SHIFT) | offset
20         Register = AccessMemory(PhysAddr)
```

Accessing Memory with Paging - Step 1

1. We need to obtain the Virtual Page Number for byte 21. Let's begin:
 - a. Let's consider a 6-bit address space. 21 is then 010101 in binary
 - b. To obtain the Virtual Page Number, the first step is to perform an AND between the address and a mask (VPN_MASK), which "extracts" only the bits related to the virtual page number. In our case, VPN_MASK=110000 because we want to extract only the two leftmost bits. In summary, we perform: $010101 \& 110000 = 010000$
 - c. Unfortunately, 01000 is not the VPN yet. In order to obtain the VPN, we need to shift it right by 4 bits (the length of the offset field in bits). We obtain then: 000001, which is the finally the VPN

```
1      // Extract the VPN from the virtual address
2      VPN = (VirtualAddress & VPN_MASK) >> SHIFT
```

Accessing Memory with Paging - Step 2

2. Now we need to access the Page Table Entry (PTE) 01 (binary)
 - a. We suppose the page table is in memory, starting at address PTBR
 - b. The address of the page table entry containing the translation we are looking for is then: $PTEAddr = PTBR + (VPN * sizeof(PTE))$

```
4      // Form the address of the page-table entry (PTE)
5      PTEAddr = PTBR + (VPN * sizeof(PTE))
```

- c. Let's load the content of PTE: $PTE = \text{AccessMemory}(PTEAddr)$

```
7      // Fetch the PTE
8      PTE = AccessMemory(PTEAddr)
```

Accessing Memory with Paging – Step 3

3. We have now the Page Table Entry in PTE. Let's perform some checks. We check:
 - a. If the entry is valid
 - b. If we have the permission to access

```
// Check if process can access the page
11     if (PTE.Valid == False)
12         RaiseException(SEGMENTATION_FAULT)
13     else if (CanAccess(PTE.ProtectBits) == False)
14         RaiseException(PROTECTION_FAULT)
```

Accessing Memory with Paging - Step 4

4. If the checks we have performed previously were ok, we now need to build the physical address that we need to read the data from
 - a. Let's first of all obtain the offset. Similarly to VPN_MASK, we have another mask (OFFSET_MASK), which is its complement. In our case, OFFSET_MASK=001111, therefore $\text{offset} = \text{VirtualAddress} \& \text{OFFSET_MASK} = 000101$

17 `offset = VirtualAddress & OFFSET_MASK`

- b. Now let's build the physical address. We know the base address of the page frame: it is contained in PTE, and we identify it as: PTE.PFN. In our case, PFN = 1110000. To obtain the final physical address we can just OR it with the offset: $\text{PhysicalAddress} = \text{PFN} \mid \text{offset} = 1110101$

18 `PhysAddr = (PTE.PFN << PFN_SHIFT) | offset`

Accessing Memory with Paging - Step 5

5. Now we can finally populate %eax

```
19         Register = AccessMemory(PhysAddr)
```

Paging Exercise

Address space size: 16k

Phys mem size: 32K

Page size: 4k

Page Table (from entry 0 down to the max size)

The high-order (left-most) bit is the VALID bit.

If the bit is 1, the rest of the entry is the PFN.

If the bit is 0, the page is not valid.

VPN	PFN
[0]	0x8000000c
[1]	0x00000000
[2]	0x00000000
[3]	0x80000006

Paging Exercise

Address space size: 16k

Phys mem size: 32K

Page size: 4k

Page Table:

VPN	PFN
[0]	0x8000000c
[1]	0x00000000
[2]	0x00000000
[3]	0x80000006

PAGE TABLE
DEL PROCESSO

SE INIZIA CON 1 LO POSSIAMO
TRADURRE

Address Space	
0KB	VPN 0
4KB	VPN 1
8KB	VPN 2
12KB	VPN 3
16KB	

TRADUO DA LOGICO A FISICO

Virtual Address Trace

VA 0x00003229 (decimal: 12841) --> PA or invalid address?
 VA 0x00001369 (decimal: 4969) --> PA or invalid address?
 VA 0x00001e80 (decimal: 7808) --> PA or invalid address?
 VA 0x00002556 (decimal: 9558) --> PA or invalid address?
 VA 0x00003a1e (decimal: 14878) --> PA or invalid address?

IDENTIFICA CHE QUELLO CHE SEGA È SCRITTO IN
ESADECIMALE

QUESTI INFATTI
IDENTIFICANO LA
VPN

QUESTI
L'OFFSET

1. TRADUO IN BINARIO
~~00~~ 11 0010 0010 1001
PER NOI GLI INDIRIZZI VIRTUALI SONO
DI 44b QUINDI
2. IDENTIFICO LA VPN (3)
3. TRADUO IL PFN IN BINARIO
4. VEDO SE A SINISTRA AL PFN C'È 1
TRADUZIONE VALIDA
5. TRADUO PFN (GLI ULTIMI 4 BIT)
(6)
6. MOLTIPLICO PER LA DIMENSIONE DELLA
PAGE 6 · 4096
7. TRADUO L'OFFSET
E LO AGGIUNGO AL PUNTO 6
8. FINITO!

RAZIONAMENTO SEMPLIFICATO

So che la memoria virtuale è divisa in pag da 4096 quindi divido l'indirizzo per 4096, in questo caso scopro che l'indirizzo si trova nella oag n.3. Ora vedo immediatamente che è valida (8 più a sinistra), scopro che la traduzione per la pagina tre è il pfn 6, ora faccio 6×4096 e poi sommo l'offset

Paging Exercise

VPN	PFN
[0]	0x8000000c
[1]	0x00000000
[2]	0x00000000
[3]	0x80000006

Virtual Address Trace

VA 0x00003229 (decimal: 12841) -->

VPN: $\text{inf}(12841/4096) = 3$

Offset: $12841 - 3 * 4096 = 553$

PFN: 0x80000006 --> decimal: 6

Physical Address: $6 * 4096 + 553 = 25129$

VA 0x00001369 (decimal: 4969) -->

VA 0x00001e80 (decimal: 7808) -->

VA 0x00002556 (decimal: 9558) -->

VA 0x00003a1e (decimal: 14878) -->

4368

$4368 / 4096 = 1$ NON VALIDO

7808

$7808 / 4096 = 1$ NON VALIDO

$5558 / 4096 = 2$ NON VALIDO

$14848 / 4096 = 3$

OFFSET

$14848 - 3 \cdot 4096 = 2590$

$6 \cdot 4096 + 2590 = 27166$

Paging Exercise

VPN	PFN
[0]	0x8000000c
[1]	0x00000000
[2]	0x00000000
[3]	0x80000006

Virtual Address Trace

VA 0x00003229 (decimal: 12841) --> 25129
VA 0x00001369 (decimal: 4969) -->
VA 0x00001e80 (decimal: 7808) -->
VA 0x00002556 (decimal: 9558) -->
VA 0x00003a1e (decimal: 14878) -->

Paging Exercise

	VPN	PFN
[	0]	0x8000000c
[	1]	0x00000000
[	2]	0x00000000
[	3]	0x80000006

Virtual Address Trace

VA 0x00003229 (decimal: 12841) --> 25129
VA 0x00001369 (decimal: 4969) --> not valid
VA 0x00001e80 (decimal: 7808) -->
VA 0x00002556 (decimal: 9558) -->
VA 0x00003a1e (decimal: 14878) -->

Paging Exercise

VPN	PFN
[0]	0x8000000c
[1]	0x00000000
[2]	0x00000000
[3]	0x80000006

Virtual Address Trace

VA 0x00003229 (decimal: 12841) --> 25129
VA 0x00001369 (decimal: 4969) --> not valid
VA 0x00001e80 (decimal: 7808) --> not valid
VA 0x00002556 (decimal: 9558) -->
VA 0x00003a1e (decimal: 14878) -->

Paging Exercise

VPN	PFN
[0]	0x8000000c
[1]	0x00000000
[2]	0x00000000
[3]	0x80000006

Virtual Address Trace

VA 0x00003229 (decimal: 12841) --> 25129
VA 0x00001369 (decimal: 4969) --> not valid
VA 0x00001e80 (decimal: 7808) --> not valid
VA 0x00002556 (decimal: 9558) --> not valid
VA 0x00003a1e (decimal: 14878) -->

Paging Exercise

VPN	PFN
[0]	0x8000000c
[1]	0x00000000
[2]	0x00000000
[3]	0x80000006

Virtual Address Trace

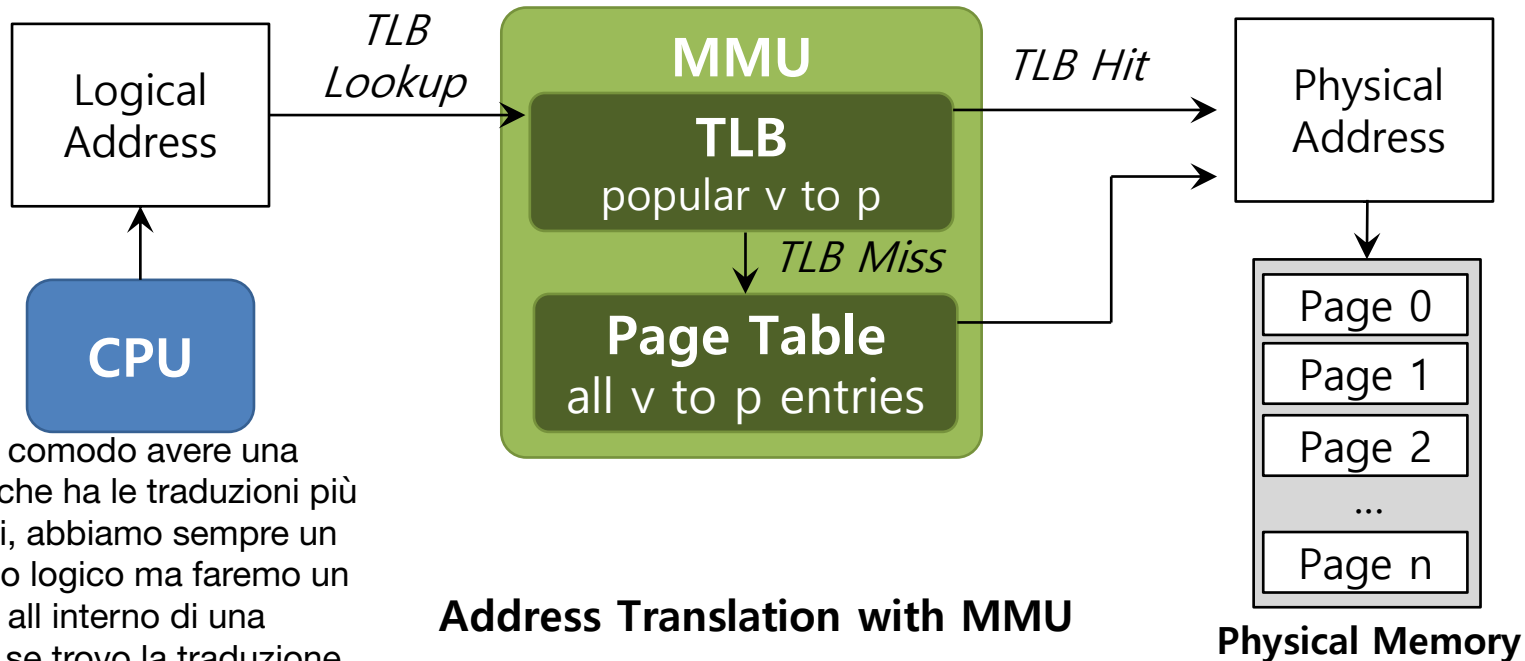
VA 0x00003229 (decimal: 12841) --> 25129
VA 0x00001369 (decimal: 4969) --> not valid
VA 0x00001e80 (decimal: 7808) --> not valid
VA 0x00002556 (decimal: 9558) --> not valid
VA 0x00003a1e (decimal: 14878) --> 27166

19. Translation Lookaside Buffer

Operating System: Three Easy Pieces

Translation Lookaside Buffer (TLB)

- Part of the chip's memory-management unit (MMU).
- A hardware cache of **popular** virtual-to-physical address translation.



Risulta comodo avere una cache che ha le traduzioni più comuni, abbiamo sempre un indirizzo logico ma faremo un lookup all'interno di una cache, se trovo la traduzione faccio la traduzione se non la trovo allora devo accedere alla page table e tradurre

TLB Basic Algorithms

```
1: VPN = (VirtualAddress & VPN_MASK ) >> SHIFT
2: (Success , TlbEntry) = TLB_Lookup(VPN) → LOOKUP ALL' INTERNO DELLA CACHE
3:     if (Success == True) { // TLB Hit
4:         if (CanAccess(TlbEntry.ProtectBit) == True ) {
5:             offset = VirtualAddress & OFFSET_MASK
6:             PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7:             AccessMemory( PhysAddr )
8:         } else RaiseException( PROTECTION_ERROR )
```

→ ALL' INTERNO HA UNA PAGE TABLE ENTRY

→ POSSIAMO ACCEDERE?

} COSTRUIRE L'INDIRIZZO

- ◆ (line 1) extract the virtual page number (VPN).
- ◆ (line 2) check if the TLB holds the translation for this VPN.
- ◆ (lines 5-8) extract the page frame number from the relevant TLB entry, and form the desired physical address and access memory.

TLB Basic Algorithms (Cont.)

```
11:      }else{ //TLB Miss
12:          PTEAddr = PTBR + (VPN * sizeof(PTE))
13:          PTE = AccessMemory(PTEAddr)
14:          (...)
15:          TLB_Insert(VPN , PTE.PFN , PTE.ProtectBits)
16:          RetryInstruction()
17:      }
18: }
```

- ◆ (11-12 lines) The hardware accesses the page table to find the translation.
- ◆ (15 lines) updates the TLB with the translation.

Example: Accessing An Array

- How a TLB can improve paging performance.

	OFFSET				
	00	04	08	12	16
VPN = 00					
VPN = 01					
VPN = 03					
VPN = 04					
VPN = 05					
VPN = 06		a[0]	a[1]	a[2]	
VPN = 07	a[3]	a[4]	a[5]	a[6]	
VPN = 08	a[7]	a[8]	a[9]		
VPN = 09					
VPN = 10					
VPN = 11					
VPN = 12					
VPN = 13					
VPN = 14					
VPN = 15					

```
0:      int sum = 0 ;
1:      for( i=0; i<10; i++) {
2:                  sum+=a[i];
3:      }
```

**The TLB improves performance
due to **spatial locality****

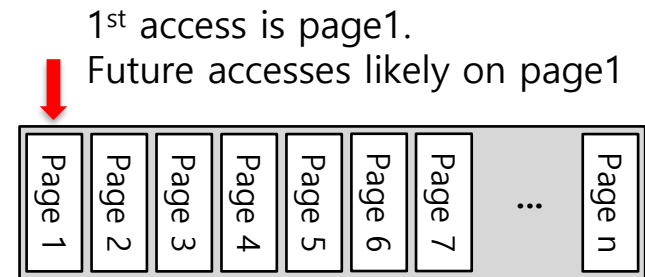
3 misses and 7 hits.
Thus **TLB hit rate** is 70%.

Ogni volta che andiamo a leggere un valore dall array dobbiamo ogni volta effettuare la traduzione, con la TLB è tutto più veloce, potremmo incorrere in cache miss, però quando ho un miss la traduzione viene caricata all'interno del TLB

Locality

- Temporal Locality

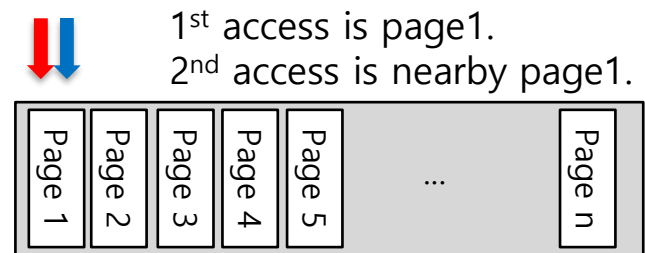
- An instruction or data item that has been recently accessed will likely be re-accessed soon in the future.



Virtual Memory

- Spatial Locality

- If a program accesses memory at address x , it will likely soon access memory near x .



Virtual Memory

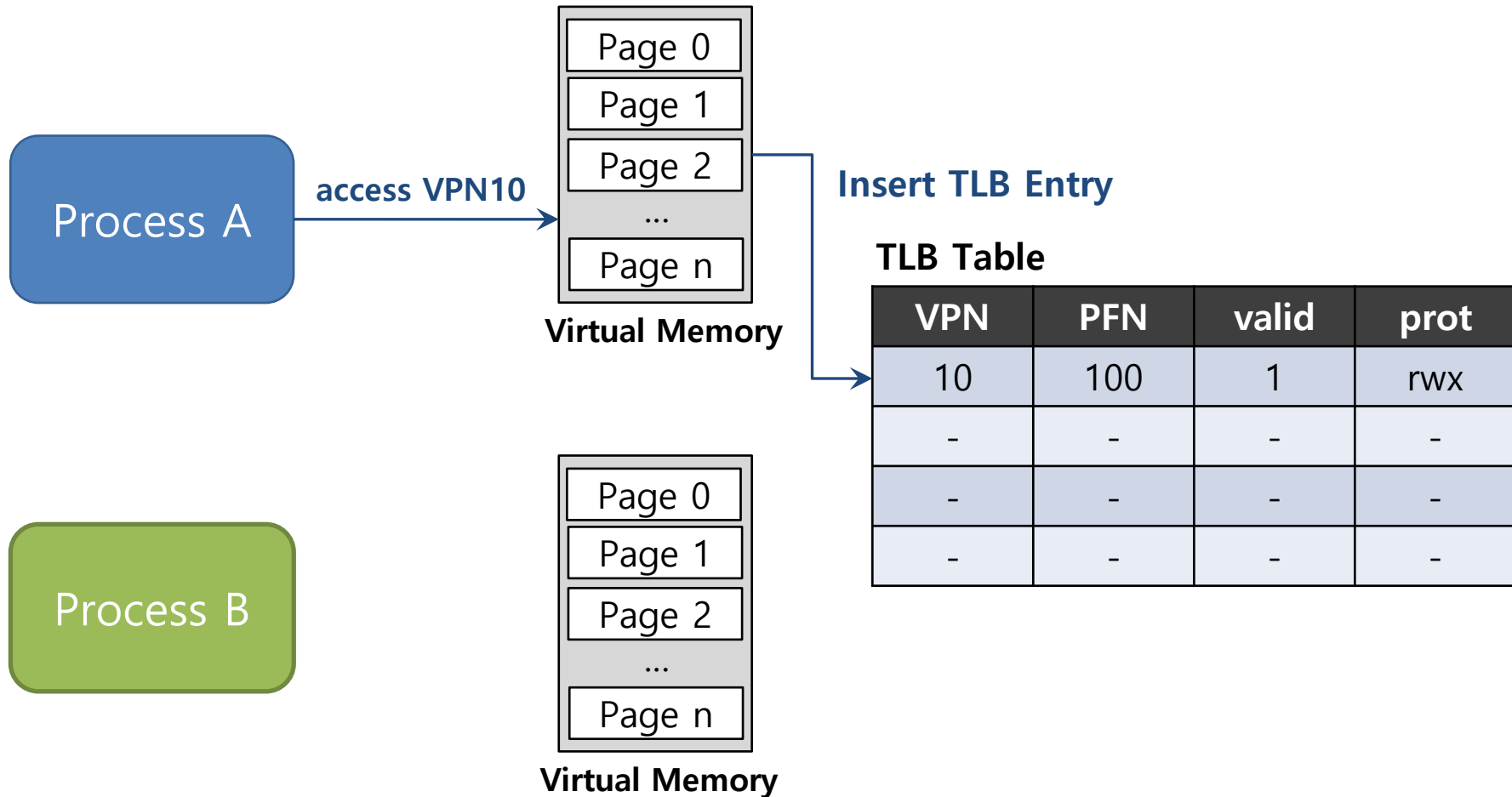
TLB entry

- A typical TLB might have 32,64, or 128 entries.
- Hardware searches the entire TLB in parallel to find the desired translation.
- other bits: valid bits, protection bits, address-space identifier, dirty bit

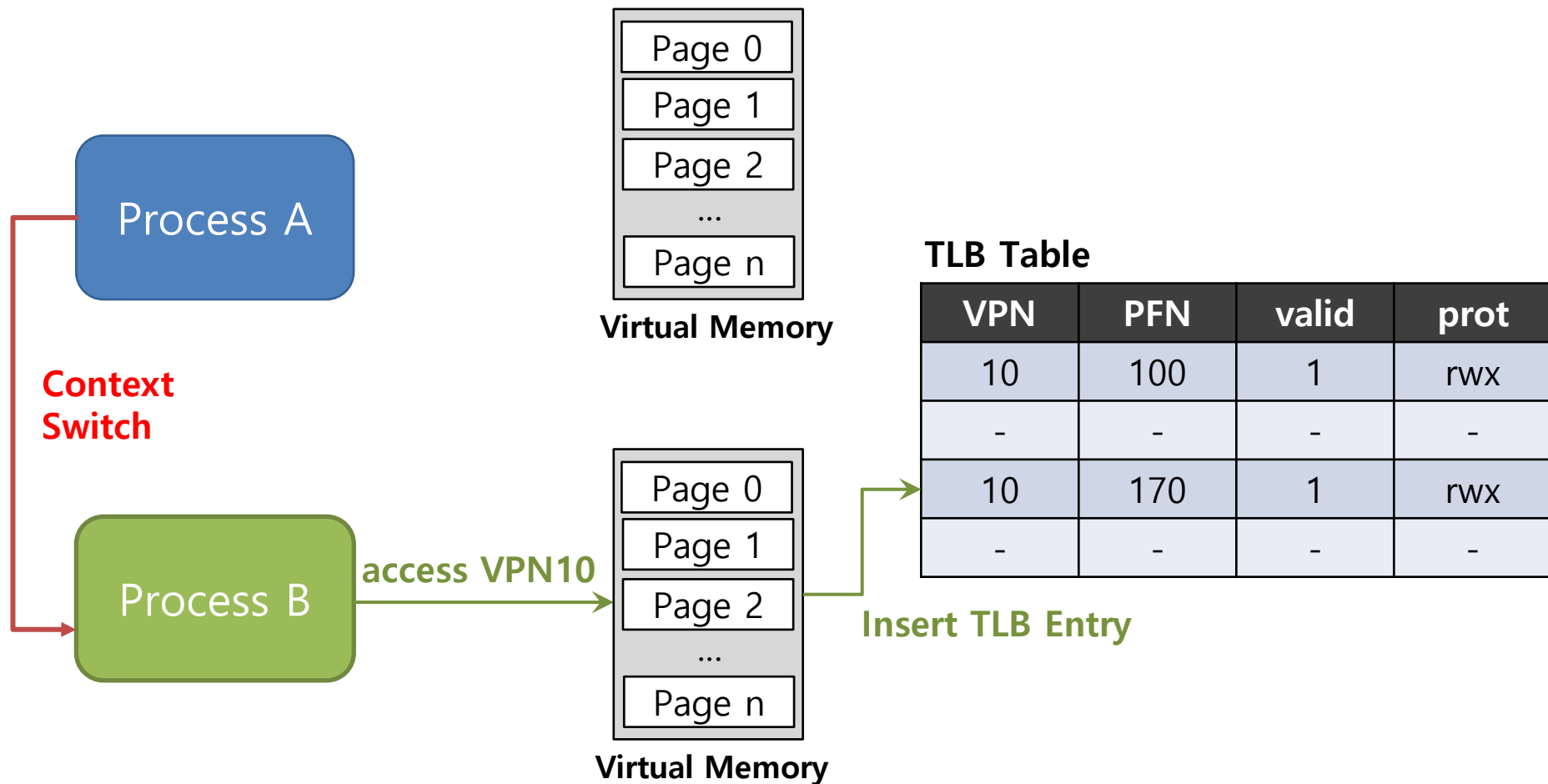


Typical TLB entry look like this

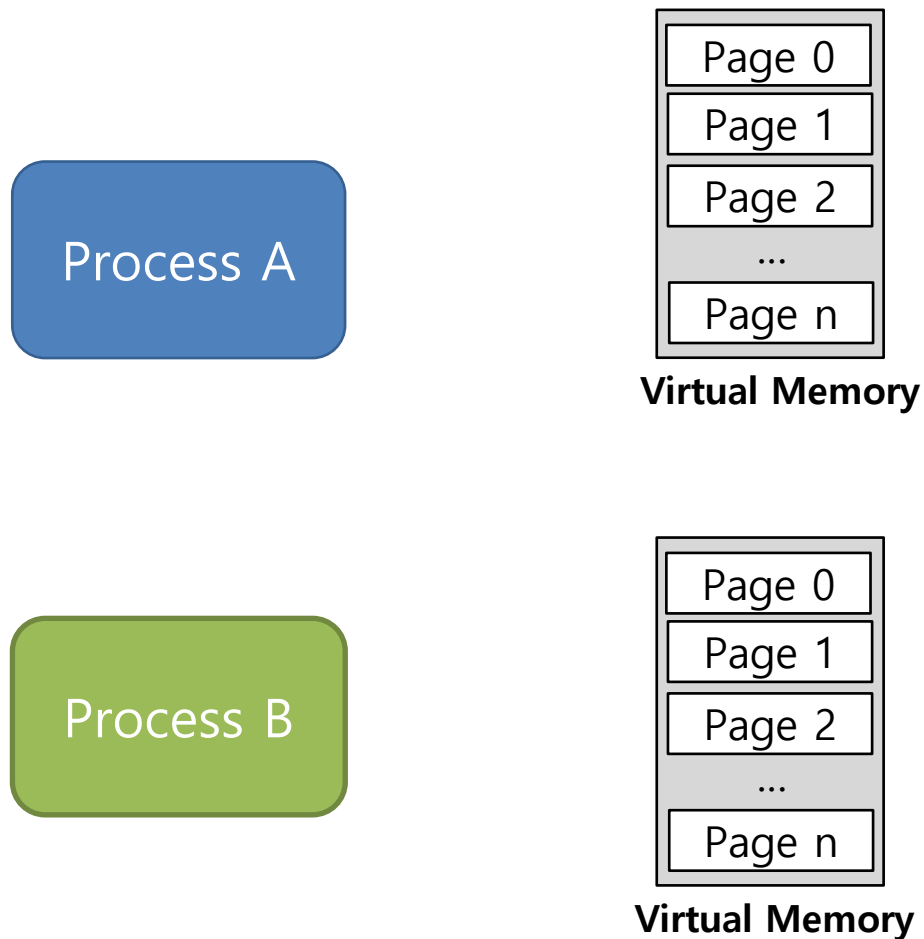
TLB Issue: Context Switching



TLB Issue: Context Switching



TLB Issue: Context Switching



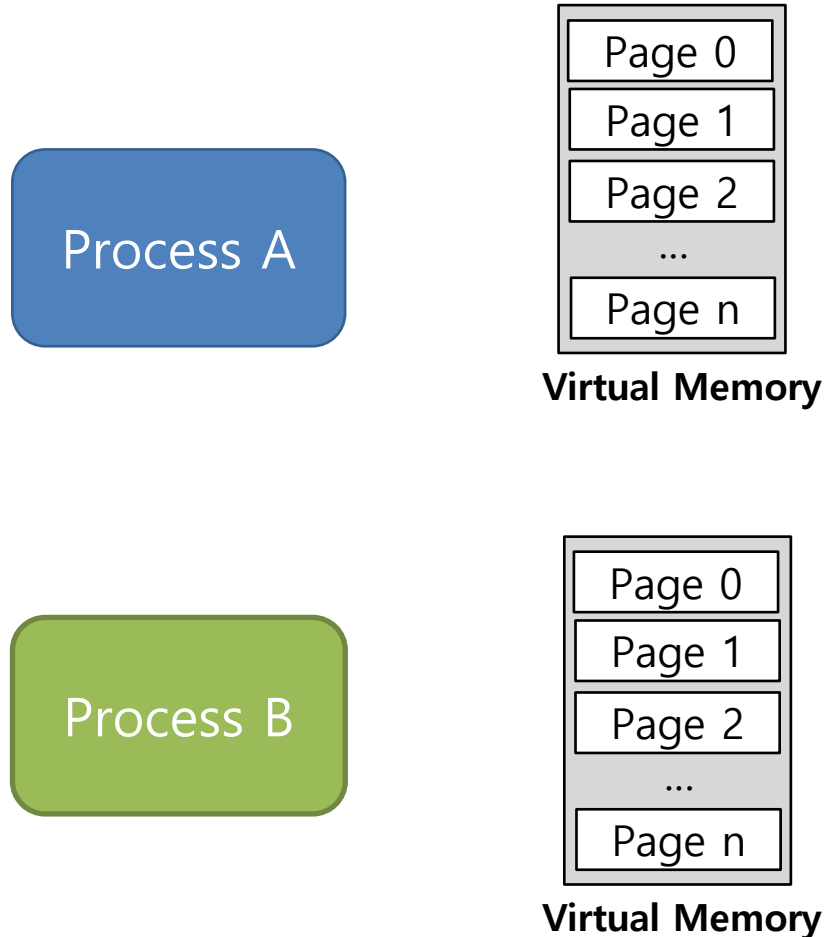
TLB Table

VPN	PFN	valid	prot
10	100	1	rwX
-	-	-	-
10	170	1	rwX
-	-	-	-

Can't Distinguish which entry is meant for which process

To Solve Problem

- Provide an address space identifier (ASID) field in the TLB.



TLB Table

VPN	PFN	valid	prot	ASID
10	100	1	rwX	1
-	-	-	-	-
10	170	1	rwX	2
-	-	-	-	-

Another Case

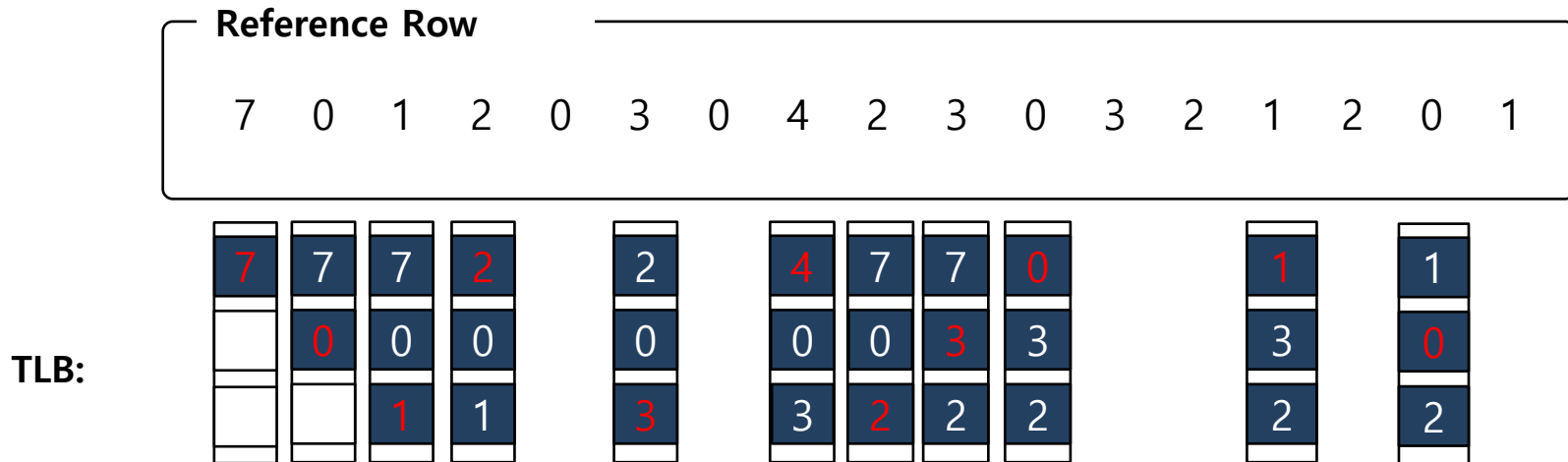
- Two processes **share a page**.
 - Process 1 is sharing physical page 101 with Process2.
 - P1 maps this page into the 10th page of its address space.
 - P2 maps this page to the 50th page of its address space.

VPN	PFN	valid	prot	ASID
10	101	1	rwX	1
-	-	-	-	-
50	101	1	rwX	2
-	-	-	-	-

Sharing of pages is **useful** as it reduces the number of physical pages in use.

TLB Replacement Policy → COSA SUCEDE QUANDO LA TLB È PIENA?

- LRU (Least Recently Used) → SCARTA IL VALORE → UTILIZZATO MENO RECENTEMENTE
 - Evict an entry that has not recently been used.



1 MISS

Se accedo a una sequenza di pagine di 1 più grande rispetto alla TLB questa politica sarà fallimentare e quindi provocherà solo cache miss

Total 11 TLB miss

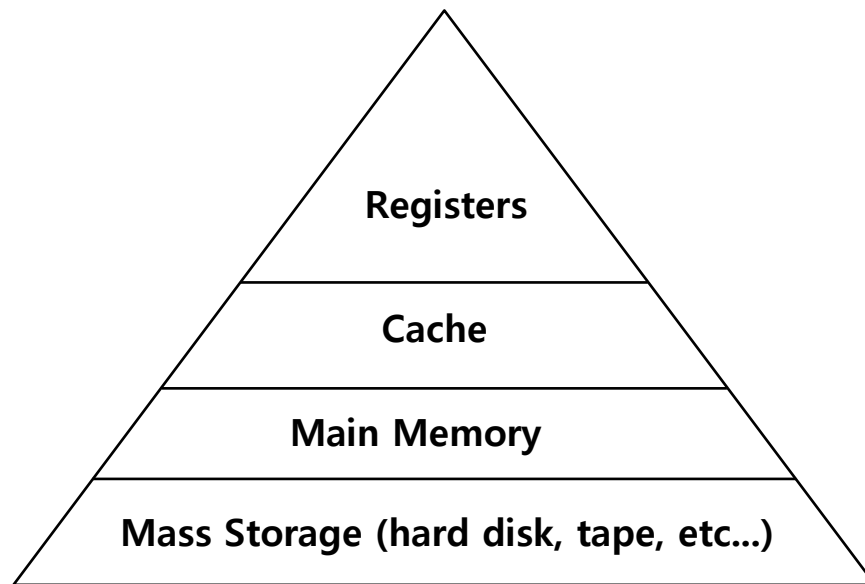
- Random. Useful because fast and easy to implement
 - Could perform better than LRU in extreme cases, e.g., loop over $n+1$ pages, with TLB size n

21. Swapping: Mechanisms

Operating System: Three Easy Pieces

Beyond Physical Memory: Mechanisms

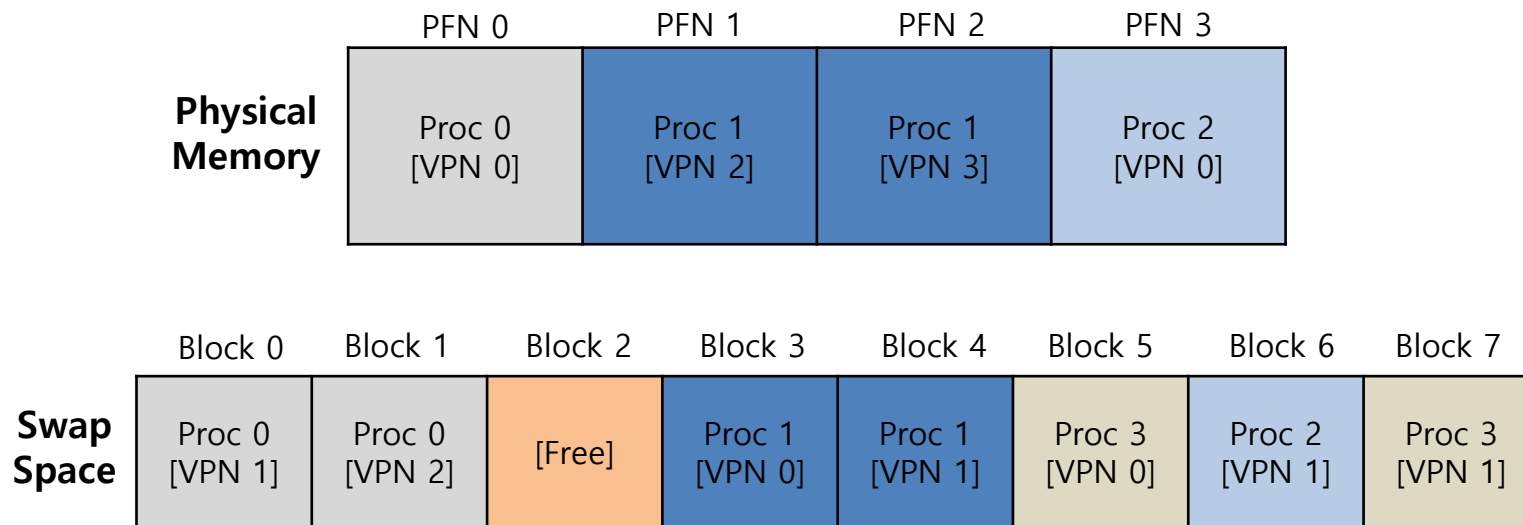
- Requires an additional level in the **memory hierarchy**.
 - OS needs a place to stash away portions of address space that currently are not in great demand.
 - In modern systems, this role is usually served by a **hard disk drive**



Memory Hierarchy in modern system

Swap Space

- Reserve some space on the disk for moving pages back and forth.
- OS splits the swap space in **page-sized unit**



Physical Memory and Swap Space

Present Bit

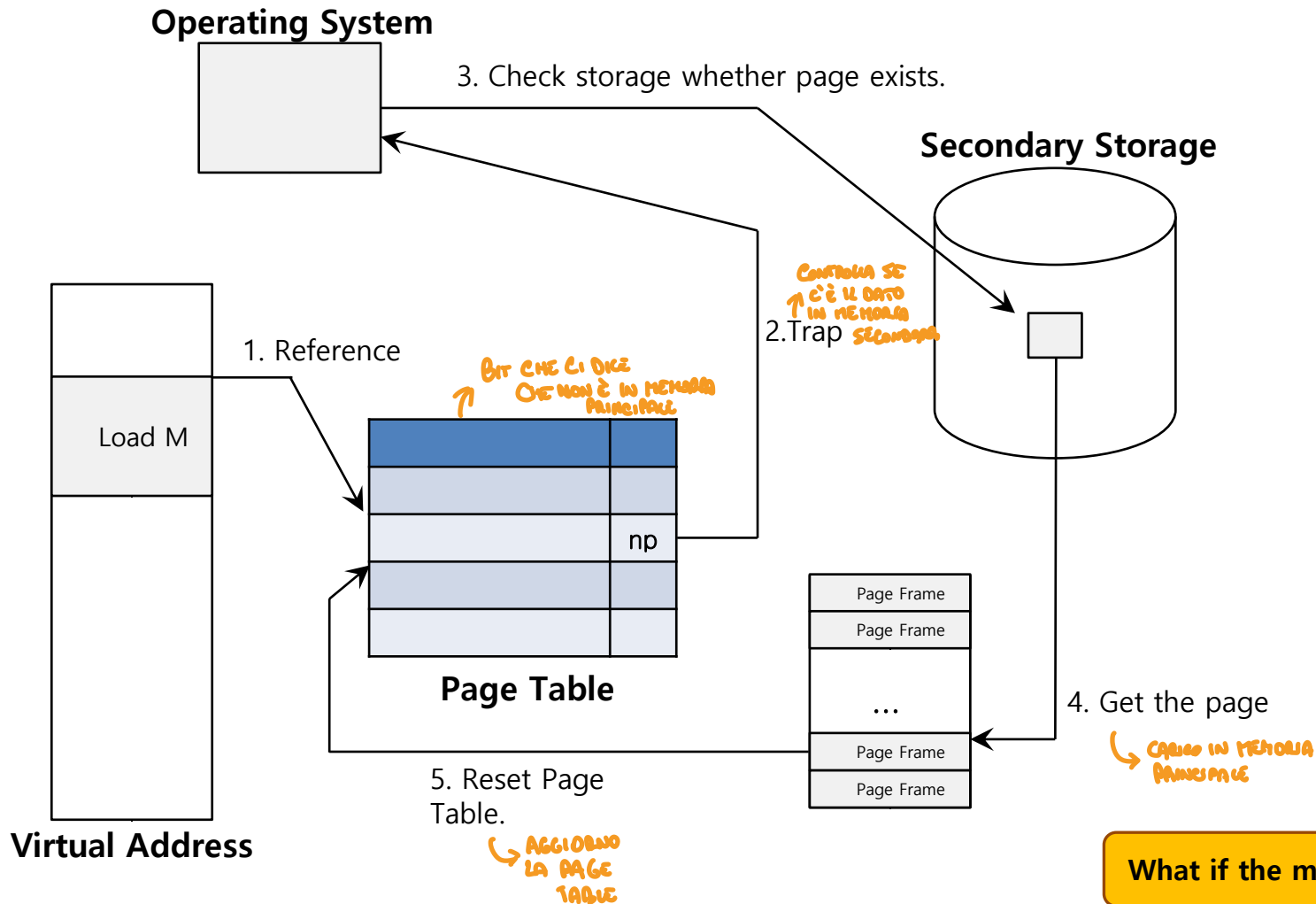
- Assuming a system with hardware TLB, we have seen that when a process tries to access a certain memory address, the following happens:
 - the VPN of the virtual address is found in the TLB → the PFN is retrieved and the physical memory accessed
 - the VPN of the virtual address is not found in the TLB → the page table is accessed to retrieve the PTE that contains the PFN and the TLB is updated
- Add some machinery higher up in the system in order to support swapping pages to and from the disk.
 - When the hardware looks in the PTE, it may find that the page is not present in physical memory.

Value	Meaning
1	page is present in physical memory
0	The page is not in memory but rather on disk.

The Page Fault

- Accessing page that is **not in physical memory**. **MA IN MEMORIA SECONDARIA**
 - If a page is not present and has been swapped to disk, the OS needs to swap the page into memory in order to serve the page fault.
 - In the PTE related to the page that must be swapped, the PFN field does not represent an address in the physical memory, rather an address on the hard drive
- Let's see a practical example

Page Fault Control Flow



Page Fault Control Flow

```
1:     VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2:     (Success, TlbEntry) = TLB_Lookup(VPN)
3:     if (Success == True) // TLB Hit
4:         if (CanAccess(TlbEntry.ProtectBits) == True)
5:             Offset = VirtualAddress & OFFSET_MASK
6:             PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7:             Register = AccessMemory(PhysAddr)
8:         else RaiseException(PROTECTION_FAULT)
```

Page Fault Control Flow – Hardware

```
9:         else // TLB Miss
10:             PTEAddr = PTBR + (VPN * sizeof(PTE))
11:             PTE = AccessMemory(PTEAddr)
12:             if (PTE.Valid == False)
13:                 RaiseException(SEGMENTATION_FAULT)
14:             else
15:                 if (CanAccess(PTE.ProtectBits) == False)
16:                     RaiseException(PROTECTION_FAULT)
17:                 else if (PTE.Present == True)
18:                     // assuming hardware-managed TLB
19:                     TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
20:                     RetryInstruction()
21:                 else if (PTE.Present == False)
22:                     RaiseException(PAGE_FAULT)
```

When Swapping Really Occurs

- OS waits until memory is entirely full, and only then replaces a page to make room for some other page
 - This is a little bit unrealistic, and there are many reason for the OS to keep a small portion of memory free more proactively.
- Swap Daemon
 - Two thresholds: High Watermark (HW) and Low Watermark (LW)
 - When the number of free pages goes under LW, the swap daemon swaps out some pages from memory to disk, until the free pages are above HW

VmStat per le informazioni relative allo stato del sistema operativo, vm perchè sono statistiche relative alla memoria virtuale. Le prime due colonne la b rappresenta i processi in stato block, la r rappresenta i processi che sono o running o ready, sotto memory, swpd quanto spazio su memoria secondaria sfo utilizzando per lo swap, free quantità di memoria disponibile, buff e cache, cache dati letti da file, buff quantità di memoria delle informazioni che devono essere ancora scritto, si so, swap in e swap out, sono a 0, vszlori che identificano la quantità in mb dalla memoria principale alla secondaria e viceversa. bi bo, dsti che ho portato da dispositivi di input output alla memoria principale e viceversa, in e cs, in sta per interrupt verificati in un determinato intervallo temporale e cs numero di context switch, id quanto il processore è libero. wa sta per wait tempo di cou sprecato in attesa di completamento di operazioni di I/O.

Altro comando utile TOP, shr shared quantità di memoria condivisa, virt e res, VIRT è altissimo su un programma che alloca una quantità di memoria elevata, RES crescerà quando effettivamente la memoria verrà utilizzata

26. Concurrency: An Introduction

Operating System: Three Easy Pieces

Introduction to Concurrency

- The Operating System introduces several abstractions
 - CPU virtualization → it creates the illusion of having multiple virtual CPU, which allows us to run multiple programs "at the same time"
 - Memory virtualization → each program believes and behaves as all the memory of the computer has been allocated just for itself
 - New abstraction: **thread** (FILO DI ESECUZIONE)
 - a single-thread program has a single Program Counter (PC)
 - Multi-threaded program
 - A multi-threaded program has more than one point of execution.
 - Multiple PCs (Program Counter)
 - Similar to having multiple processes, with the difference that threads share the same address space.
- ↓ DIVENTA PIU' SEMPLICE
CONDIZIONE DATI → AL CONTRARIO DI
QUANDO SI UTILIZZANO
DIVERSI SPAZI DEGLI INDIRIZZI

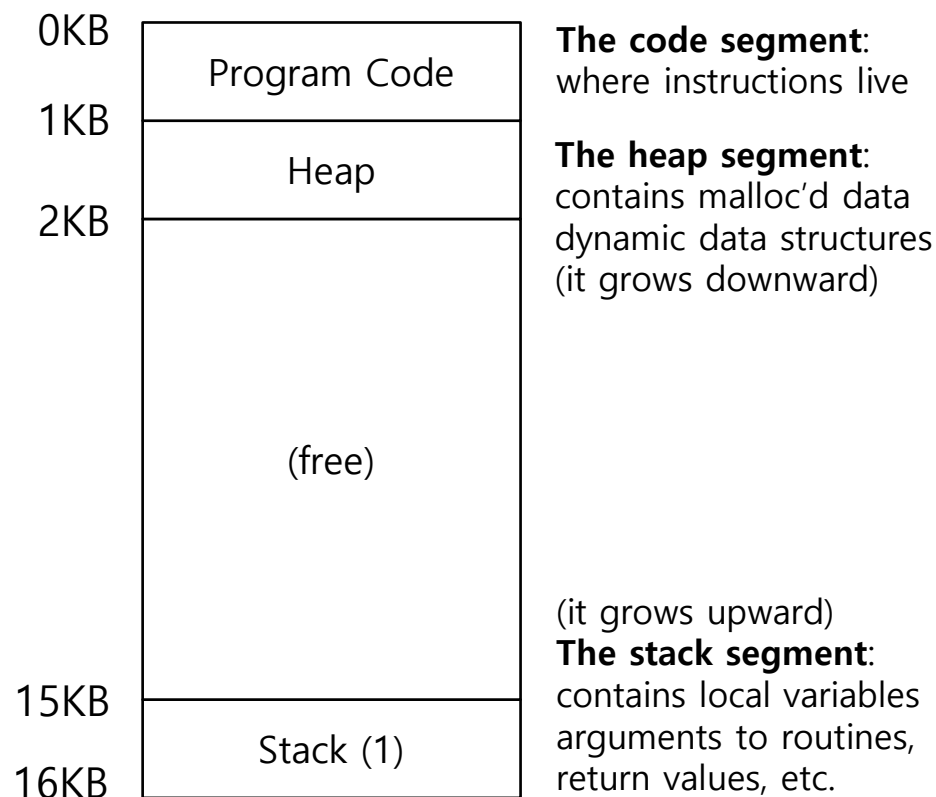
Context switch between threads

- Each thread has its own program counter and set of registers for the computation
 - With processes, during a context switch, the state of the process releasing the resources is saved in memory, in the **Process Control Block (PCB)**
 - With threads, the operation is similar, but their states are saved in memory in one or more **Thread Control Blocks (TCBs)**
- When switching from running one (T1) to running the other (T2),
 - The register state of T1 is saved.
 - The register state of T2 is restored.
 - The **address space remains** the same → no need to switch the page table we use

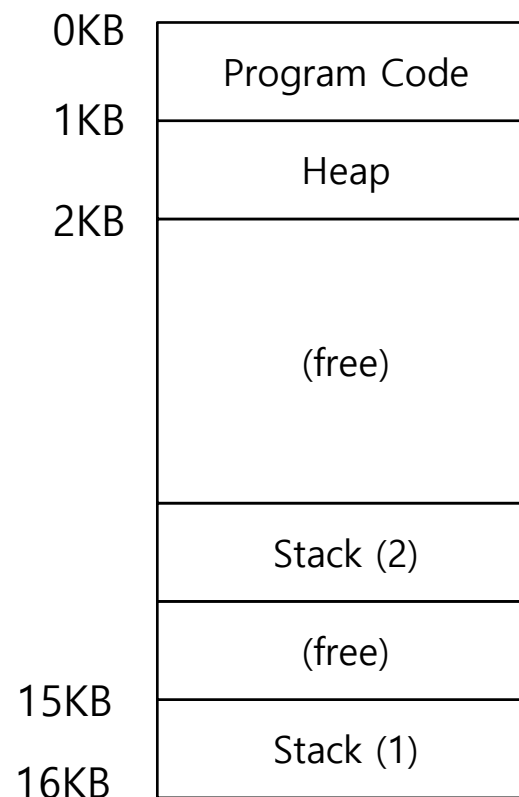
È PIÙ LEGGERO
SAPENDO CHE NON DOBBIAMO
↑ SWITCHARE LA PTB

The stack of a multithread process

- How does the address space change with multiple threads? Example with 2 threads



**A Single-Threaded
Address Space**



**Two threaded
Address Space**

Why Use Threads

- Multi-threading adds a lot of complexity, both from the perspective of the application that we write, and that of the operating system
 - So why should we use multi-threading?
- One simple answer: performance. However, we have two different angles
 - **Parallelism.** Modern computers have several CPUs or cores, each of which can execute a different thread at the same time
 - If we can write an application that distributes its computation among the available CPUs/cores, we increase the performance
 - **Efficient Programming.** You know better than anyone else the code of your applications. Sometimes, you can optimize their execution.
 - Example: an application that has to load a large amount of data from disk (slow operation) but does not need that data immediately might spawn a thread just for loading the data to avoid being blocked on the rest

An Example of Very Simple Multi-Threaded Application

```
1  #include <stdio.h>
2  #include <assert.h>
3  #include <pthread.h>
4
5  void *mythread(void *arg) {
6      printf("%s\n", (char *) arg);
7      return NULL;
8  }
9
10 int
11 main(int argc, char *argv[]) {
12     pthread_t p1, p2;
13     int rc;
14     printf("main: begin\n");
15     rc = pthread_create(&p1, NULL, mythread, "A"); assert(rc == 0);
16     rc = pthread_create(&p2, NULL, mythread, "B"); assert(rc == 0);
17     // join waits for the threads to finish
18     rc = pthread_join(p1, NULL); assert(rc == 0);
19     rc = pthread_join(p2, NULL); assert(rc == 0);
20     printf("main: end\n");
21     return 0;
22 }
```

Handwritten annotations:

- PUNTERO A UN ARGOMENTO DI CUI NON SAPPIAMO IL TIPO** (with arrow pointing to `void *` in line 5)
- IDENTIFICAZIONE DI UN THREAD CHE STO PER ANDARE A ESEGUIRE / CREARE** (with arrow pointing to `pthread_create` in line 15)
- QUELLO PRINCIPALE** (with arrow pointing to `main` in line 11)
- CREARE E FAR PARTIRE NUOVI THREAD IN ALGUNO A QUELLO CHE ATTIVO** (with arrow pointing to `pthread_create` in line 15)
- PUNTERO A UNA FUNZIONE** (with arrow pointing to `mythread` in line 15)
- "VOGLIAMO CHE VENGA ESEGUITA QUANDO IL THREAD VIENE CREATO"** (with arrow pointing to the third argument of `pthread_create` in line 15)
- PARAMETRO DELLA FUNZIONE** (with arrow pointing to the fourth argument of `pthread_create` in line 15)

Possible Outcomes (1)

main	Thread 1	Thread2
starts running		
prints "main: begin"		
creates Thread 1		
creates Thread 2		
waits for T1	runs	
	prints "A"	
	returns	
waits for T2		runs
		prints "B"
		returns
prints "main: end"		

Possible Outcomes (2)

main	Thread 1	Thread2
starts running prints "main: begin" creates Thread 1		
	runs prints "A" returns	
creates Thread 2		runs prints "B" returns
waits for T1 <i>returns immediately; T1 is done</i>		
waits for T2 <i>returns immediately; T2 is done</i>		
prints "main: end"		

Possible Outcomes (3)

main	Thread 1	Thread2
starts running prints "main: begin" creates Thread 1 creates Thread 2		
		runs prints "B" returns
waits for T1	runs prints "A" returns	
waits for T2 <i>returns immediately; T2 is done</i> prints "main: end"		

An Example of Multi-Threaded Application With Some Issues

```
static volatile int counter = 0;
void *
mythread(void *arg)
{
    printf("%s: begin\n", (char *) arg);
    int i;
    for (i = 0; i < 1e7; i++) {
        counter = counter + 1;
    }
    printf("%s: done\n", (char *) arg);
    return NULL;
}

int
main(int argc, char *argv[])
{
    pthread_t p1, p2;
    printf("main: begin (counter = %d)\n", counter);
    Pthread_create(&p1, NULL, mythread, "A");
    Pthread_create(&p2, NULL, mythread, "B");
    
    // join waits for the threads to finish
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("main: done with both (counter = %d)\n", counter);
    return 0;
}
```

Possible Outputs

```
prompt> gcc -o main main.c -Wall -pthread
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 20000000)
```

```
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 19345221)
```

```
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 19221041)
```

Critical Section and Race Condition

- Critical Section: a piece of code that is potentially executed by multiple threads, which **access a shared variable at the same time**
 - Multiple threads executing a critical section can result in a race condition.
- Race Condition: the result of a computation depends on the timing (or order) of execution of the code.

Race condition: an example

```

mov 0x8049a1c, %eax
add $0x1, %eax
mov %eax, 0x8049a1c

```

- Example with two threads

OS	Thread 1	Thread 2	(after instruction)		
			PC	%eax	counter
	<i>before critical section</i>		100	0	50
	mov 0x8049a1c, %eax		105	50	50
	add \$0x1, %eax		108	51	50
interrupt					
<i>save T1's state</i>					
<i>restore T2's state</i>			100	0	50
		mov 0x8049a1c, %eax	105	50	50
		add \$0x1, %eax	108	51	50
		mov %eax, 0x8049a1c	113	51	51
interrupt					
<i>save T2's state</i>					
<i>restore T1's state</i>			108	51	51
	mov %eax, 0x8049a1c		113	51	51

The Wish for Atomicity

- In other words, the problem is due to the fact that the “increment” operation is not atomic. Instead, it relies on three instructions.
- How to solve this problem?
 - We could introduce in the instruction set of the CPU an atomic instruction, say *memory-add*, which could be invoked like this:
memory-add 0x8049a1c, \$0x1
7 INCREMENT OF MEMORY 7 VALUE IN REGISTER
 - With this new instruction, in case an interrupt occurs, either the instruction has been executed or it has not. This is why we say that it is *atomic*
 - Unfortunately, this is only a very simple case. And the reality is much more complex
 - Think about updating a node in a B-Tree, or any other type of data in any other data structure. Intuitively, this cannot be realized in-hardware

Locks

- We need a more flexible mechanism
 - A mechanism that allows only one thread at a time to access a critical section
- This is the core of what we will see in the “Concurrency” part of this journey.

```
1  lock_t mutex;  
2  . . .  
3  lock(&mutex);  
4  balance = balance + 1;  
5  unlock(&mutex);
```

→ Critical section

28. Locks

Operating System: Three Easy Pieces

Locks: The Basic Idea

- Problem: multi-threaded applications could have critical sections, such as:

```
balance = balance + 1;
```

- Even though the previous seems to be a single instruction, it is actually composed of 3 instructions, which are not executed *atomically*
 - In other words, the execution of the application could be interrupted by any interrupt and another thread (perhaps executing the same code) could be run afterwards
- Need to find a way to “group together” an arbitrary portion of code (critical section) as it were a **single atomic instruction**.
- Solution is provided by locks. **A lock** is a *variable* that contains information regarding which thread can execute the critical section

Locks: The Basic Idea

- Lock variable holds the state of the lock.
 - **available** (or **unlocked** or **free**)
 - No thread holds the lock.
 - **acquired** (or **locked** or **held**)
 - Exactly one thread holds the lock and presumably is in a critical section.

IL THREAD
ACQUISISCE IL LOCK PRIMA
DI ENTRARE NELLA SEZIONE CRITICA
E NON PUO' ESSERE ACQUISITO DA NESSUN ALTRO THREAD

```
1  lock_t mutex; // some globally-allocated lock 'mutex'
2  ...
3  lock(&mutex);
4  balance = balance + 1;
5  unlock(&mutex);
```

MUTUA
ESCLUSIONE

Pthread Locks - mutex

- mutex (Mutual Exclusion) is the name that the POSIX library uses for a lock.
 - Used to provide mutual exclusion between threads.

MACRO CHE ATTRIBUISCE
IL VALORE DI LOCK DEL VALORE DI DEFAULT

```
1  pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;  
2  
3  pthread_mutex_lock(&lock); // wrapper for pthread_mutex_lock()  
4  balance = balance + 1;  
5  pthread_mutex_unlock(&lock);
```

Building A Lock

- Locks are pretty easy to use from the standpoint of a programmer, but how are they actually realized?
- Just like many other things we have seen in this course, building a lock needs some help from the **hardware** and the **OS**.

Evaluating locks – Basic criteria

In general, you might think about a high number of threads (even hundreds). What we have seen in the previous lecture (with 2 threads) was just an example

- **Metrics for the evaluation of a lock implementation:**

- **Mutual exclusion** → CI DICE SE IL LOCK STA FUNZIONANDO
SE NON C'È MUTUA ESCLUSIONE IL LOCK È INUTILE

- Does the lock work, preventing multiple threads from entering *a critical section* together?

- **Fairness** (EQUITA')

- Does each thread contending for the lock get a fair shot at acquiring it once it is free? (Starvation)

- **Performance**

- What is the time overhead added by using the lock?

A Very Simple Form of Lock - Controlling Interrupts

- **Disable Interrupts** for critical sections

→ TIRA OUT IL TIMER INTERRUPT

- One of the earliest solutions used to provide mutual exclusion

```
1  void lock() {  
2      DisableInterrupts();  
3  }  
4  void unlock() {  
5      EnableInterrupts();  
6  }
```

↳ PRESERVA IL CONTROLLO DEL PROCESSORE

- Problems:

- Requires too much *trust* in applications
 - Greedy (or malicious) program could monopolize the processor.
- Does not work on **multiprocessors**
 - If multiple threads are running on multiple CPUs, they will still be able to access the critical section despite the disabled interrupts
- Losing interrupts could lead to serious problems

A Software-Only Attempt at the Realization of a Lock

- **First attempt:** Using a *flag* denoting whether the lock is held or not.
 - The code below has problems.

```
1  typedef struct __lock_t { int flag; } lock_t;
2
3  void init(lock_t *mutex) {
4      // 0 -> lock is available, 1 -> held
5      mutex->flag = 0;
6  }
7
8  void lock(lock_t *mutex) {
9      while (mutex->flag == 1) // TEST the flag
10         ; // spin-wait (do nothing)
11     mutex->flag = 1; // now SET it !
12 }
13
14 void unlock(lock_t *mutex) {
15     mutex->flag = 0;
16 }
```

A Software-Only Attempt at the Realization of a Lock (Cont.)

- **Problem 1:** No Mutual Exclusion (assume `flag=0` to begin)

Thread1

Thread2

```
call lock()
while (flag == 1)
interrupt: switch to Thread 2
```

```
call lock()
while (flag == 1)
flag = 1;
interrupt: switch to Thread 1
```

```
flag = 1; // set flag to 1 (too!) DUE THREAD LEAK SEIZURE CONCERN
```

- **Problem 2:** Spin-waiting wastes time waiting for another thread.
- So, we need an atomic instruction supported by **Hardware!**
 - *test-and-set* instruction, also known as *atomic exchange*

Test And Set (Atomic Exchange)

- *Test and Set* is an atomic hardware instruction. The following is its description in C

↑ VALUE MEMORIZED

```
1  int TestAndSet(int *ptr, int new) {  
2      int old = *ptr;    // fetch old value at ptr  
3      *ptr = new;        // store 'new' into ptr  
4      return old;        // return the old value  
5  }
```

- This sequence of operations is **performed atomically**.
- The instruction takes two parameter: a pointer to an integer in memory and a new int value
 - It retrieves the int value pointed by the pointer
 - It stores in the area of memory pointed by the pointer the new int value
 - It returns the old int value

A Simple Spin Lock using test-and-set

```
1  typedef struct __lock_t {
2      int flag;
3  } lock_t;
4
5  void init(lock_t *lock) {
6      // 0 indicates that lock is available,
7      // 1 that it is held
8      lock->flag = 0;
9  }
10
11 void lock(lock_t *lock) {
12     while (TestAndSet(&lock->flag, 1) == 1)
13         ;           // spin-wait
14 }
15
16 void unlock(lock_t *lock) {
17     lock->flag = 0;
18 }
```

↳ uso
eccessivo
di processore

- **Note:** To work correctly on *a single processor*, it requires a preemptive scheduler.

Evaluating Spin Locks

- **Correctness:** yes
 - The spin lock only allows a single thread to enter the critical section.
- **Fairness:** no
 - Spin locks don't provide any fairness guarantees.
 - Indeed, a thread spinning may spin *forever*.
- **Performance:**
 - With one CPU, performance overheads can be quite *painful*.
 - If the number of threads roughly equals the number of CPUs, spin locks work *reasonably well*.
 - (as long as you don't have other applications running)

Fetch-And-Add

- Atomically increment a value while returning the old value at a particular address.

```
1  int FetchAndAdd(int *ptr) {  
2      int old = *ptr;  
3      *ptr = old + 1;  
4      return old;  
5  }
```

Handwritten notes: "Fetch d'un valeur" with an arrow pointing to line 2, and "GLI AGGIUNGE UN UNITÀ" with an arrow pointing to the "+ 1" in line 3.

Fetch-And-Add Hardware atomic instruction (C-style)

Ticket Lock

- **Ticket lock** can be built with fetch-and add.
 - Ensure progress for all threads → **fairness**

```
1  typedef struct __lock_t {
2      int ticket;
3      int turn;
4  } lock_t;
5
6  void lock_init(lock_t *lock) {
7      lock->ticket = 0;
8      lock->turn = 0;
9  }
10
11 void lock(lock_t *lock) {
12     int myturn = FetchAndAdd(&lock->ticket);
13     while (lock->turn != myturn)
14         ; // spin
15 }
16 void unlock(lock_t *lock) {
17     FetchAndAdd(&lock->turn);
18 }
```

Immaginiamo il lock come una biglietteria che vende ticket, ticket mi dice quanti ticket ho venduto (quanti thread hanno provato a fare lock) il turn stabilisce di chi è il turno di entrare nella sezione critica. Quando facciamo lock la fetch and add ci restituisce il mio turno quest'ultima infatti va a prendere il numero di ticket venduti fino a quel momento e mi restituisce il mio turno. Ora devo semplicemente aspettare che sia il mio turno. Unlock invece va ad aumentare il valore della variabile del turno, quindi tocca al thread i+unesimo

So Much Spinning

- Hardware-based spin locks are **simple** and they work.
- In some cases, these solutions can be quite **inefficient**.
 - Any time a thread gets caught *spinning*, it **wastes an entire time slice** doing nothing but checking a value.

How To Avoid *Spinning*?
We'll need OS Support too!

A Simple Approach: Just Yield

- Instead of spinning, **give up the CPU** to another thread.
 - OS system call moves the caller from the *running state* to the *ready state*.

```
1  void init() {
2      flag = 0;
3  }
4
5  void lock() {
6      while (TestAndSet(&flag, 1) == 1)
7          yield(); // give up the CPU
8  }
9
10 void unlock() {
11     flag = 0;
12 }
```

Lock with Test-and-set and Yield